

Calcolabilità e complessità

Alessandro Carella

Indice

I	Automi e linguaggi	3
1	Automi e linguaggi	4
1.1	Automi finiti	4
1.2	Non determinismo	8
1.3	Espressioni regolari	16
1.3.1	Definizione formale di espressione regolare	16
1.3.2	Equivalenza con gli automi finiti	18
1.4	Linguaggi non regolari	24
1.4.1	Il pumping lemma per i linguaggi regolari	24
2	Linguaggi context-free	28
2.1	Grammatiche context-free	28
2.1.1	Ambiguità	29
2.1.2	Forma normale di Chomsky	30
2.1.3	Membership problem	33
2.1.4	Proprietà dei linguaggi context-free	36
2.2	Automi a pila	39
2.2.1	Definizione formale di automa a pila	40
2.2.2	Equivalenza con le grammatiche context-free	43
2.3	Linguaggi non context-free	51
2.3.1	Il pumping lemma per i linguaggi context-free	51
2.4	Linguaggi context-free deterministici	55
2.4.1	Proprietà dei DCFL	57
II	Teoria della computabilità	59
3	La tesi di Church-Turing	60

3.1	Macchine di Turing	60
3.1.1	Definizione formale di macchina di Turing	60
3.1.2	Esempi di macchine di Turing	63
3.2	Varianti di macchine di Turing	66
3.2.1	Macchina di Turing multinastro	66
3.2.2	Macchina di Turing non deterministica	68
3.2.3	Enumeratori	70
3.3	La definizione di algoritmo	71
3.3.1	I problemi di Hilbert	71
4	Decidibilità	74
4.1	Linguaggi decidibili	74
4.1.1	Problemi decidibili relativi a linguaggi regolari	74
4.1.2	Problemi decidibili relativi a linguaggi context-free	78
4.2	Indecidibilità	81
4.2.1	Il metodo della diagonalizzazione	81
4.2.2	Un linguaggio indecidibile	84
4.2.3	Un linguaggio non Turing-riconoscibile	85
5	Riducibilità	87
5.1	Problemi indecidibili dalla teoria dei linguaggi	87
III	Teoria della complessità	92
6	Complessità di tempo	93
6.1	Misure di complessità	93
6.1.1	Notazione O-grande ed o-piccola	94
6.1.2	Analisi degli algoritmi	95
6.2	La classe P	96
6.2.1	Tempo polinomiale	96
6.3	La classe NP	101
6.3.1	Esempi di problemi in NP	104
6.3.2	$P = NP$	106
6.4	NP-completezza	107
6.4.1	Riducibilità in tempo polinomiale	107
6.4.2	Definizione di NP-completezza	110
6.4.3	Il Teorema di Cook e Levin	110

Parte I

Automati e linguaggi

Capitolo 1

Automi e linguaggi

1.1 Automi finiti

Gli automi finiti sono un buon modello per computer una quantità estremamente di memoria che permette di fare molte cose utili. Il sistema di controllo per una porta automatica è un esempio di tale dispositivo.

Gli automi finiti e la loro controparte probabilistica, le *catene di Markov* sono utili strumenti quando cerchiamo di riconoscere pattern in dati. Questi dispositivi sono usati nell'elaborazione vocale e nel riconoscimento ottico dei caratteri. Le catene di Markov sono state perfino usate per modellare e predire i cambiamenti dei prezzi nei mercati finanziari.

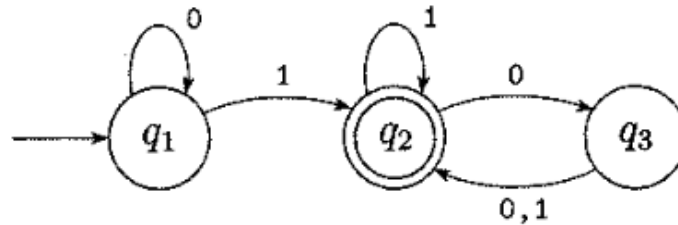
Definizione formale di automa finito

DEFINIZIONE 1.1: AUTOMA FINITO

Un *automa finito* è una quintupla $(Q, \Sigma, \delta, q_0, F)$ dove:

1. Q è un insieme finito chiamato l'insieme degli *stati*;
2. Σ è un insieme finito chiamato *alfabeto*;
3. $\delta : Q \times \Sigma \rightarrow Q$ è la *funzione di transizione*;
4. $q_0 \in Q$ è lo *stato iniziale*;
5. $F \subseteq Q$ è l'*insieme degli stati accettanti*.

Consideriamo il seguente automa M_1 :



Possiamo descrivere M_1 formalmente ponendo $M_1 = (Q, \Sigma, \delta, q_1, F)$ ove:

1. $Q = \{q_1, q_2, q_3\}$;
2. $\Sigma = \{0, 1\}$;
3. δ è descritta come segue:

	0	1
q_1	q_1	q_2
q_2	q_3	q_2
q_3	q_2	q_2

4. q_1 è lo stato iniziale;
5. $F = \{q_2\}$

Se A è l'insieme di tutte le stringhe che la macchina M accetta, diciamo che A è il *linguaggio della macchina* M e scriviamo $L(M) = A$. Diciamo che M *riconosce* A o che M *accetta* A . Poichè il termine *accetta* ha significati diversi quando ci riferiamo a macchine che accettano stringhe e macchine che accettano linguaggi, per evitare confusione preferiamo per i linguaggi il termine *riconosce*.

Una macchina può accettare diverse stringhe, ma riconosce sempre soltanto un linguaggio. Se la macchina non accetta alcuna stringa, essa riconosce ancora un linguaggio, il linguaggio vuoto $\emptyset$.

Nel nostro esempio, sia:

$$A = \{w \mid w \text{ contiene almeno un } 1 \text{ e un numero pari di } 0 \text{ segue l'ultimo } 1\}$$

Definizione formale di computazione

Finora abbiamo descritto gli automi finiti informalmente, usando diagrammi di stato, e con una definizione formale, come una quintupla. La descrizione informale è più facile da comprendere all'inizio, ma la definizione formale è più utile per rendere la notazione precisa, risolvendo ogni ambiguità.

Facciamo lo stesso per la computazione di un automa finito.

Sia $M = (Q, \Sigma, \delta, q_0, F)$ un automa finito e sia $w = w_1 w_2 \cdots w_n$ una stringa dove ogni w_i è un elemento dell'alfabeto Σ . Allora M accetta w se esiste una sequenza di stati $r_0, r_1, \dots, r_n$ in Q con tre condizioni:

1. $r_0 = q_0$;
2. $\delta(r_i, w_{i+1}) = r_{i+1}$ per $i = 0, \dots, n-1$;
3. $r_n \in F$.

La prima condizione afferma che la macchina inizia nello stato iniziale. La seconda condizione afferma che la macchina passa da stato a stato in base alla funzione di transizione. La terza condizione afferma che la macchina accetta il suo input se termina la lettura in uno stato accettante. Diciamo che M riconosce il linguaggio A se $A = \{w \mid M \text{ accetta } w\}$.

DEFINIZIONE 1.2: LINGUAGGIO REGOLARE

Un linguaggio è chiamato un *linguaggio regolare* se un automa finito lo riconosce.

Le operazioni regolari

Dopo aver introdotto e definito gli automi finiti e i linguaggi regolari, iniziamo a studiarne le proprietà. In aritmetica, gli oggetti di base sono i numeri e gli strumenti sono le operazioni per trattarli, come $+$ e $\times$. Nella teoria della computazione, gli oggetti sono i linguaggi e gli strumenti includono operazioni specificatamente progettate per trattarli. Definiamo tre operazioni sui linguaggi, chiamate *operazioni regolari*, e le usiamo per studiare le proprietà dei linguaggi regolari.

DEFINIZIONE 1.3: OPERAZIONI REGOLARI

Siano A e B linguaggi. Definiamo le operazioni regolari *unione*, *concatenazione* e *star* come segue:

- **Unione:** $A \cup B = \{x \mid x \in A \text{ o } x \in B\}$
- **Concatenazione:** $A \circ B = \{xy \mid x \in A \text{ e } y \in B\}$

- **Star:** $A^* = \{x_1x_2 \dots x_k \mid k \geq 0 \text{ e ogni } x_i \in A\}$

Esempio

Sia Σ l'alfabeto standard a 26 lettere $\{a, b, \dots, z\}$. Se $A = \{good, bad\}$ e $B = \{boy, girl\}$, allora:

$$A \cup B = \{good, bad, boy, girl\}$$

$$A \circ B = \{goodboy, goodgirl, badboy, badgirl\}$$

$$A^* = \{\epsilon, good, bad, goodgood, goodbad, badgood, badbad, goodgoodgood, goodgoodbad, \dots\}$$

Sia $\mathbb{N} = \{1, 2, 3, \dots\}$ l'insieme dei numeri naturali. Quando diciamo che $\mathbb{N}$ è *chiuso rispetto alla moltiplicazione*, intendiamo che per ogni x e per ogni y in $\mathbb{N}$, il prodotto $x \times y \in \mathbb{N}$. Invece $\mathbb{N}$ non è chiuso rispetto alla divisione, poichè 1 e 2 sono in $\mathbb{N}$ ma $1/2 \notin \mathbb{N}$.

In generale, una classe di oggetti è *chiusa* rispetto a un'operazione se l'applicazione di questa operazioni a elementi della classe restituisce un oggetto ancora nella classe. Mostriamo che la classe dei linguaggi regolari è chiusa rispetto a tutte e tre le operazioni regolari.

Teorema 1.1: Chiusura unione

La classe dei linguaggi regolari è chiusa rispetto all'operazioni di unione.

In altre parole, se A_1 e A_2 sono linguaggi regolari, lo è anche $A_1 \cup A_2$.

Quindi, abbiamo due linguaggi regolari, A_1 e A_2 , e vogliamo mostrare che anche $A_1 \cup A_2$ è regolare. Poichè A_1 e A_2 sono regolari, sappiamo che sono riconosciuti da degli automi finiti. Quindi per provare che $A_1 \cup A_2$ è regolare, dobbiamo esibire un automa finito, M , che riconosce $A_1 \cup A_2$.

Dimostrazione. Supponiamo che M_1 riconosca A_1 , dove $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ ed M_2 riconosca A_2 , dove $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$. Costruiamo M che riconosce $A_1 \cup A_2$ dove $M = (Q, \Sigma, \delta, q_0, F)$.

1. $Q = \{(r_1, r_2) \mid r_1 \in Q_1 \text{ e } r_2 \in Q_2\}$.
Questo insieme è il *prodotto cartesiano* degli insiemi Q_1 e Q_2 ed è denotato con $Q_1 \times Q_2$.
2. Σ , l'alfabeto è lo stesso di M_1 e M_2 . In questo teorema e in quelli successivi, assumiamo per semplicità che sia M_1 che M_2 abbiano stesso alfabeto di simboli. Tuttavia, se così non fosse, $\Sigma = \Sigma_1 \cup \Sigma_2$.

3. δ , la funzione di transizione, è definita come segue. Per ogni $(r_1, r_2) \in Q$ e ogni $a \in \Sigma$, sia

$$\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$$

4. q_0 è la coppia (q_1, q_2) .
5. F è l'insieme delle coppie in cui l'uno o l'altro elemento è uno stato accettante di M_1 o M_2 . Possiamo scriverlo come

$$F = \{(r_1, r_2) \mid r_1 \in F_1 \text{ o } r_2 \in F_2\}$$

Questa espressione è uguale a $F = (F_1 \times Q_2) \cup (Q_1 \times F_2)$

□

Abbiamo mostrato che l'unione di due linguaggi regolari è regolare, provando che la classe dei linguaggi regolari è chiusa rispetto all'operazione di unione. Ora mostriamo che la classe dei linguaggi regolari è chiusa rispetto all'operazione di concatenazione.

Teorema 1.2: Chiusura concatenazione

La classe dei linguaggi regolari è chiusa rispetto all'operazione di concatenazione.

In altre parole, se A_1 e A_2 sono linguaggi regolari, allora lo è anche $A_1 \circ A_2$.

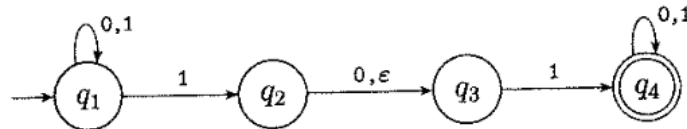
Per provare questo teorema, possiamo iniziare con gli automi finiti M_1 ed M_2 che riconoscono i linguaggi regolari A_1 ed A_2 . Ma ora, invece di costruire M che accetta l'input se M_1 o M_2 lo accetta, esso deve accettare se il suo input può essere diviso in due parti, tali che M_1 accetta la prima parte ed M_2 accetta la seconda parte. Il problema è che M non sa dividere il suo input. Per risolvere questo problema, introduciamo una tecnica chiamata *non determinismo*.

1.2 Non determinismo

Negli esempi visti precedentemente, ogni passo di una computazione seguiva univocamente dal passo precedente. Quando la macchina è in un dato stato e legge il simbolo di input successivo, sappiamo qual è lo stato successivo, esso è univocamente determinato. In questo caso parliamo di computazione *deterministica*. In una macchina *non deterministica*, possono esistere diverse scelte per lo

stato successivo in ogni punto.

Il non determinismo è una generalizzazione del determinismo, quindi ogni automa finito deterministico è automaticamente un automa finito non deterministico. Consideriamo il seguente automa:



La differenza tra un automa finito deterministico, DFA, e un automa finito non deterministico, indicato con NFA, è evidente. In primo luogo, ogni stato di un DFA ha sempre esattamente un arco uscente per ogni simbolo nell'alfabeto. L'NFA viola questa regola, infatti, lo stato q_1 dell'automata della figura precedente, ha un arco uscente per 0, ma ne ha due per 1. In un NFA, uno stato può avere zero, uno, o più archi uscenti per ogni simbolo dell'alfabeto.

Inoltre, in un DFA, le etichette sugli archi di transizione sono simboli dell'alfabeto. Questo NFA ha un arco con etichetta ϵ .

Durante la computazione di un NFA, quando si giunge ad uno stato con più modi per procedere, la macchina si divide in più copie di sè stessa e segue tutte le possibilità in parallelo. Infine, se una qualunque di queste copie della macchina è in uno stato accettante alla fine dell'input, l'NFA accetta la stringa di input.

Il non determinismo può essere visto come una specie di computazione parallela dove "processi" multipli indipendenti o "threads" possono essere eseguiti contemporaneamente. Un altro modo per considerare una computazione non deterministica è vederla come un albero di possibilità. La radice dell'albero corrisponde all'inizio della computazione. Considerando sempre l'automata precedente, l'albero di computazione è il seguente:

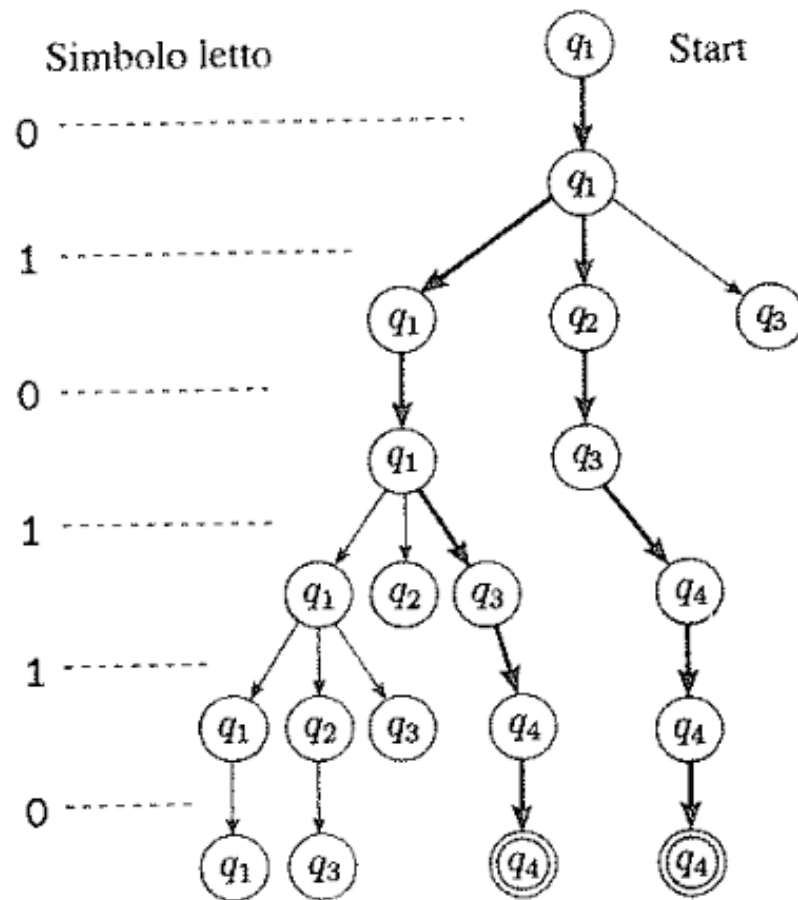


Figura 1.1: La computazione dell'automa precedente su 010110

Definizione formale di automa finito non deterministico

La definizione formale di automa finito non deterministico è simile a quella di automa finito deterministico. Essi differiscono nel tipo di funzione di transizione. In un DFA, la funzione di transizione prende uno stato e un simbolo di input e produce lo stato successivo. In un NFA, la funzione di transizione prende uno stato e un simbolo di input o *la stringa vuota* e produce l'*insieme dei possibili stati successivi*.

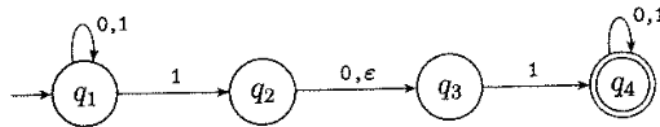
DEFINIZIONE 1.4: AUTOMA FINITO NON DETERMINISTICO

Un *automa finito non deterministico* è una quintupla $(Q, \Sigma, \delta, q_0, F)$ dove

1. Q è un insieme finito di stati,
2. Σ è un alfabeto finito,
3. $\delta : Q \times \Sigma_\epsilon \rightarrow \mathcal{P}(Q)$ è la funzione di transizione,
4. $q_0 \in Q$ è lo stato iniziale,
5. $F \subseteq Q$ è l'insieme degli stati di accettazione.

Esempio

Riconsideriamo il seguente NFA N_1 :



La descrizione formale di N_1 è $(Q, \Sigma, \delta, q_1, F)$ dove

1. $Q = \{q_1, q_2, q_3, q_4\}$
2. $\Sigma = \{0, 1\}$
3. δ è data da

	0	1	ϵ
q_1	$\{q_1\}$	$\{q_1, q_2\}$	$\emptyset$
q_2	$\{q_3\}$	$\emptyset$	$\{q_3\}$
q_3	$\emptyset$	$\{q_4\}$	$\emptyset$
q_4	$\{q_4\}$	$\{q_4\}$	$\emptyset$

4. q_1 è lo stato iniziale
5. $F = \{q_4\}$

La definizione formale di computazione per un NFA è simile a quella per un DFA. Sia $N = (Q, \Sigma, \delta, q_0, F)$ un NFA e sia w una stringa sull'alfabeto Σ . Diciamo

che N accetta w se possiamo scrivere w come $w = y_1 y_2 \dots y_m$, dove ciascun y_i è un elemento di $\sum_\epsilon$ ed esiste una sequenza di stati $r_0, r_1, \dots, r_n$ in Q con tre condizioni:

1. $r_0 = q_0$
2. $r_{i+1} \in \delta(r_i, y_{i+1})$ per $i = 0, \dots, m-1$
3. $r_m \in F$

Equivalenza tra gli NFA e i DFA Gli automi finiti deterministici e quelli non deterministici riconoscono la stessa classe di linguaggi. Diciamo che due macchine sono *equivalenti* se esse riconoscono lo stesso linguaggio.

Teorema 1.3: Equivalenza NFA e DFA

Per ogni automa finito non deterministico esiste un automa finito deterministico equivalente.

Se un linguaggio è riconosciuto da un NFA, allora dobbiamo mostrare l'esistenza di un DFA che lo riconosce. L'idea è trasformare l'NFA in un DFA equivalente che simula l'NFA.

Dimostrazione. Sia $N = (Q, \sum, \delta, q_0, F)$ l'NFA che riconosce un linguaggio A . Costruiamo un DFA $M = (Q', \sum, \delta', q'_0, F')$ che riconosce A . Prima di fare la costruzione completa, consideriamo inizialmente il caso più semplice in cui N non ha ϵ -archi.

1. $Q' = \mathcal{P}(Q)$
Ogni stato di M è un insieme di stati di N .
2. Per $R \in Q'$ e $a \in \sum$, sia $\delta'(R, a) = \{q \in Q \mid q \in \delta(r, a)\}$ per qualche $r \in R$
Se R è uno stato di M , esso è anche un insieme di stati di N . Quando M legge un simbolo a nello stato R . Poichè da ogni stato si può andare in un insieme di stati, prendiamo l'unione di tutti questi insiemi. Un altro modo per scrivere quest'espressione è:

$$\delta'(R, a) = \bigcup_{r \in R} \delta(r, a)$$

3. $q'_0 = \{q_0\}$

4. $F' = \{R \in Q' \mid R \text{ contiene uno stato accettante di } N\}$

Ora dobbiamo considerare gli ϵ -archi. Per farlo introduciamo qualche ulteriore notazione. Per ogni stato R di M , definiamo $E(R)$ come la collezione di stati che possono essere raggiunti dagli elementi di R proseguendo solo con ϵ -archi, includendo gli stessi elementi di R . Formalmente, per $R \subseteq Q$ sia

$$E(R) = \{q \mid q \text{ può essere raggiunto da } R \text{ attraverso 0 o più } \epsilon\text{-archi}\}$$

Modifichiamo la funzione di transizione di M ponendo le dita su tutti gli stati che possono essere raggiunti proseguendo attraverso ϵ -archi dopo ogni passo. Sostituendo $\delta(r, a)$ con $E(\delta(r, a))$ realizziamo questo effetto. La funzione di transizione è la seguente.

$$\delta'(R, a) = \{q \in Q \mid q \in E(\delta(r, a)) \text{ per qualche } r \in R\}$$

Inoltre, dobbiamo modificare lo stato iniziale di M per muovere inizialmente le dita su tutti i possibili stati che possono essere raggiunti dallo stato iniziale di N attraverso gli ϵ -archi. Cambiare q'_0 in $E(\{q_0\})$ realizza questo risultato. Abbiamo così completato la costruzione del DFA M che simula l'NFA N . $\square$

Corollario 1.1

Un linguaggio è regolare se e solo se qualche automa finito non deterministico lo riconosce.

Una direzione della condizione “se e solo se” afferma che un linguaggio è regolare se qualche NFA lo riconosce. Tuttavia, abbiamo dimostrato che ogni NFA può essere trasformato in un DFA. Conseguentemente, se un NFA riconosce un linguaggio, lo fa anche qualche DFA, e quindi il linguaggio regolare. L'altra direzione della condizione “se e solo se” afferma che un linguaggio è regolare solo se qualche NFA lo riconosce. Questa condizione è vera perché un linguaggio regolare ha un DFA che lo riconosce e un DFA è anche un NFA.

Chiusura rispetto alle operazioni regolari

Torniamo alla chiusura della classe dei linguaggi regolari rispetto alle operazioni regolari. Dobbiamo provare che l'unione, la concatenazione e lo star di linguaggi regolari sono ancora regolari.

Teorema 1.4

La classe dei linguaggi regolari è chiusa rispetto all'operazione di unione.

Abbiamo i linguaggi regolari A_1 ed A_2 e vogliamo provare che $A_1 \cup A_2$ è regolare. L'idea è provare che due NFA, N_1 ed N_2 per A_1 ed A_2 , e comporli in un nuovo NFA, N . La macchina N deve accettare il suo input se N_1 o N_2 accetta questo input. La nuova macchina ha uno stato iniziale che si dirama negli stati iniziali delle vecchie macchine con ϵ archi.

Dimostrazione. Sia $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ che riconosce A_1 ed $N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ che riconosce A_2 .

Costruiamo $N = (Q, \Sigma, \delta, q_0, F)$ per riconoscere $A_1 \cup A_2$.

1. $Q = \{q_0\} \cup Q_1 \cup Q_2$
2. Lo stato q_0 è lo stato iniziale di N
3. L'insieme degli stati accettanti è $F = F_1 \cup F_2$
4. Definiamo δ in modo che per ogni $q \in Q$ e $a \in \Sigma_\epsilon$

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \\ \delta_2(q, a) & q \in Q_2 \\ \{q_1, q_2\} & q = q_0 \text{ e } a = \epsilon \\ \emptyset & q = q_0 \text{ e } a \neq \epsilon \end{cases}$$

□

Teorema 1.5: L

classe dei linguaggi regolari è chiusa rispetto all'operazione di concatenazione.

L'idea è prendere due NFA, N_1 ed N_2 , per A_1 ed A_2 , e combinarli in un nuovo NFA N . Poniamo come stato iniziale di N lo stato iniziale N_1 . Gli stati accettanti di N_1 hanno degli ulteriori ϵ -archi che non deterministicamente permettono di diramarsi in N_2 ogni volta che N_1 è in uno stato accettore, indicando che ha trovato un pezzo iniziale dell'input che costituisce una stringa in A_1 . Gli stati accettanti di N sono gli stati accettanti di N_2 .

Dimostrazione. Sia $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ che riconosce A_1 ed $N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ che riconosce A_2 .

Costruiamo $N = (Q, \Sigma, \delta, q_1, F_2)$ per riconoscere $A_1 \circ A_2$.

1. $Q = Q_1 \cup Q_2$

2. Lo stato q_1 è uguale allo stato iniziale di N_1
3. Gli stati accettanti F_2 sono uguale agli stati accettanti di N_2
4. Definiamo δ in modo che per ogni $q \in Q$ e ogni $a \in \sum_\epsilon$

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \text{ e } q \notin F_1 \\ \delta_1(q, a) & q \in F_1 \text{ e } a \neq \epsilon \\ \delta_1(q, a) \cup \{q_2\} & q \in F_1 \text{ e } a = \epsilon \\ \delta_2(q, a) & q \in Q_2 \end{cases}$$

□

Teorema 1.6

La classe dei linguaggi regolari è chiusa rispetto all'operazione star.

Prendiamo un NFA N_1 per A_1 e lo modifichiamo per riconoscere A_1^* . L'NFA N risultante accetterà il suo input quando esso può essere diviso in varie parti ed N_1 accetta ogni parte. Possiamo costruire N come N_1 con ϵ -archi supplementari che dagli stati accettanti ritornano allo stato iniziale. In questo modo, quando l'elaborazione giunge alla fine di una parte che N_1 accetta, la macchina N ha la scelta di tornare indietro allo stato iniziale per provare a leggere un'altra parte che N_1 accetta. Inoltre, dobbiamo modificare N in modo che accetti ϵ , che è sempre un elemento di A_1^* .

Dimostrazione. Sia $N_1 = (Q_1, \sum, \delta_1, q_1, F_1)$ che riconosce A_1 . Costruiamo $N = (Q, \sum, \delta, q_0, F)$ per riconoscere A_1^* .

1. $Q = \{q_0\} \cup Q_1$
2. Lo stato q_0 è il nuovo stato iniziale.
3. $F = \{q_0\} \cup F_1$
4. Definiamo δ in modo che per ogni $q \in Q$ e ogni $a \in \sum_\epsilon$

$$\delta(q, a) = \begin{cases} \delta_1(qa) & q \in Q_1 \text{ e } q \notin F_1 \\ \delta_1(q, a) & q \in F_1 \text{ e } a \neq \epsilon \\ \delta_1(q, a) \cup \{q_1\} & q \in F_1 \text{ e } a = \epsilon \\ \{q_1\} & q = q_0 \text{ e } a = \epsilon \\ \emptyset & q = q_0 \text{ e } a \neq \epsilon \end{cases}$$



1.3 Espressioni regolari

Possiamo usare le operazioni regolari per costruire espressioni che descrivono linguaggi, che sono chiamate *espressioni regolari*. Ad esempio:

$$(0 \cup 1)0^*$$

Il valore di un'espressione regolare è un linguaggio. In questo caso, il valore è il linguaggio che consiste di tutte le stringhe che iniziano con 0 o un 1 seguito da un qualsiasi numero di simboli uguali a 0.

1.3.1 Definizione formale di espressione regolare

DEFINIZIONE 1.5: ESPRESSIONE REGOLARE

Diciamo che R è un'espressione regolare se R è

1. a per qualche a nell'alfabeto Σ ;
2. ϵ ;
3. $\emptyset$;
4. $(R_1 \cup R_2)$ dove R_1 ed R_2 sono espressioni regolari;
5. $(R_1 \circ R_2)$ dove R_1 ed R_2 sono espressioni regolari;
6. $(R_1)^*$ dove R_1 è un'espressione regolare.

È bene non confondere le espressioni regolari ϵ e $\emptyset$. L'espressione regolare ϵ rappresenta il linguaggio che contiene la sola stringa vuota, mentre $\emptyset$ rappresenta il linguaggio che non contiene alcuna stringa.

Inoltre, per comodità, usiamo R^+ come abbreviazione per RR^* e possiamo scrivere R^* come $R^+ \cup \epsilon$.

Quando vogliamo distinguere un'espressione regolare R e il linguaggio che descrive, denotiamo con $L(R)$ il linguaggio di R .

Analizziamo alcune proprietà.

1. $a^* \emptyset = \emptyset$
Concatenando l'insieme vuoto a un qualsiasi insieme si ottiene l'insieme vuoto.
2. $\emptyset^* = \{\epsilon\}$
L'operazione star concatena un numero qualsiasi di stringhe del linguaggio per ottenere una stringa nel risultato. Se il linguaggio è vuoto, l'operazione star può concatenare 0 stringhe, dando solo la stringa vuota.
3. $R \cup \emptyset = R$
Aggiungere il linguaggio vuoto a un qualsiasi altro linguaggio non lo cambia.
4. $R \circ \epsilon = R$
Concatenare la stringa vuota a una qualsiasi stringa non la cambia.

Tuttavia, scambiare $\emptyset$ e ϵ nelle precedenti identità può far sì che le uguaglianze non siano vere.

1. $R \cup \epsilon$ può non essere uguale a R
Per esempio, se $R = \emptyset$, allora $L(R) = \{\emptyset\}$ ma $L(R \cup \epsilon) = \{\emptyset, \epsilon\}$.
2. $R \circ \emptyset$ può non essere uguale a R .
Per esempio, se $R = \emptyset$, allora $L(R) = \{\emptyset\}$ ma $L(R \circ \emptyset) = \emptyset$

Le espressioni regolari sono un utile strumento nel progetto dei compilatori per i linguaggi di programmazione. Gli oggetti elementari in un linguaggio di programmazione, chiamati *token*, come i nomi delle variabili e le costanti, possono essere descritti con espressioni regolari. Ad esempio, una costante numerica può avere una parte decimale e/o un segno e può essere descritta come un elemento del linguaggio

$$(+ \cup - \cup \epsilon)(D^+ \cup D^+.D^* \cup D^*.D^+)$$

Una volta che la sintassi di un linguaggio di programmazione è stata descritta con un'espressione regolare in termini dei suoi token, dei sistemi automatici possono generare l'*analizzatore lessicale*, la parte di un compilatore che elabora inizialmente il programma input.

1.3.2 Equivalenza con gli automi finiti

Le espressioni regolari e gli automi finiti sono equivalenti rispetto alla loro potenza descrittiva. Ogni espressione regolare può essere trasformata in un automa finito che riconosce il linguaggio che essa descrive, e viceversa.

Teorema 1.7

Un linguaggio è regolare se e solo se qualche espressione regolare lo descrive.

Questo teorema deve essere dimostrato in entrambe le direzioni. Enunciamo e proviamo ciascuna direzione in un lemma separato.

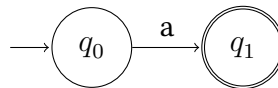
LEMMA 1.1

Se un linguaggio è descritto da un'espressione regolare, allora esso è regolare.

Supponiamo di avere un'espressione regolare R che descrive un linguaggio A . Mostriamo come trasformare R in un NFA che riconosce A .

Dimostrazione. Trasformiamo R in un NFA N . Consideriamo i sei casi nella definizione di **espressione regolare**.

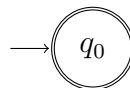
1. $R = a$ per qualche $a \in \Sigma$. Allora $L(R) = \{a\}$ e il seguente NFA riconosce $L(R)$.



Questa macchina soddisfa la definizione di NFA ma non quella di DFA perchè ha qualche stato con nessun arco uscente per ogni possibile simbolo di input.

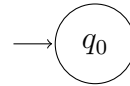
Formalmente, $N = (\{q_0, q_1\}, \Sigma, \sigma, q_0, \{q_1\})$ dove descriviamo σ dicendo che $\sigma(q_0, a) = \{q_1\}$ e $\sigma(r, b) = \emptyset$ per $r \neq q_0$ o $b \neq a$.

2. $R = \epsilon$. Allora $L(R) = \{\epsilon\}$ e il seguente NFA riconosce $L(R)$.



Formalmente $N = (\{q_0\}, \Sigma, \sigma, q_0, \{q_0\})$, dove $\sigma(r, b) = \emptyset$ per ogni r e b .

3. $R = \emptyset$. Allora $L(R) = \emptyset$ e il seguente NFA riconosce $L(R)$.



Formalmente, $N = (\{q_0\}, \Sigma, \sigma, q_0, \emptyset)$ dove $\sigma(r, b) = \emptyset$ per ogni r e b .

4. $R = R_1 \cup R_2$
 5. $R = R_1 \circ R_2$
 6. $R = R_1^*$

Per gli ultimi tre casi, usiamo le costruzioni date nelle prove che la classe dei linguaggi regolari è chiusa rispetto alle operazioni regolari. $\square$

Questo termina la prima parte del teorema dando la direzione più facile della condizione “se e solo se”.

LEMMA 1.2

Se un linguaggio è regolare, allora è descritto da un'espressione regolare.

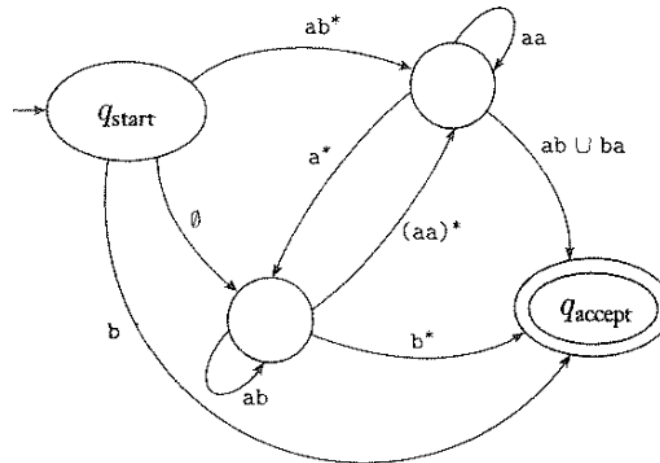
Dobbiamo mostrare che se un linguaggio A è regolare, allora un'espressione regolare lo descrive. Poiché A è regolare, esso è accettato da un DFA. Descriviamo una procedura per trasformare i DFA in espressioni regolari equivalenti.

Dividiamo questa procedura in due parti, usando un nuovo tipo di automa finito chiamato *automa finito non deterministico generalizzato*, GNFA.

Prima mostriamo come trasformare un DFA in un GNFA e poi come trasformare un GNFA in un'espressione regolare.

Gli automi finiti non deterministici generalizzati sono automi finiti non deterministici nei quali gli archi delle transizioni possono avere espressioni regolari come etichette, invece che solo elementi dell'alfabeto o ϵ . Un GNFA è non deterministico e quindi può avere diversi modi di elaborare la stessa stringa di input.

La figura seguente presenta un esempio di GNFA.



Per comodità, richiediamo che i GNFA abbiano sempre una forma speciale che soddisfi le seguenti condizioni.

- Lo stato iniziale ha archi di transizione uscenti verso un qualsiasi altro stato ma nessun arco entrante proveniente da un qualsiasi altro stato.
- Esiste un solo stato accettante, ed esso ha archi entranti provenienti da un qualsiasi altro stato ma nessun arco uscente verso un qualsiasi altro stato. Inoltre, lo stato accettante non è uguale allo stato iniziale.
- Eccetto che per lo stato iniziale e lo stato accettante, un arco va da ogni stato ad ogni altro stato e anche da ogni stato a sé stesso.

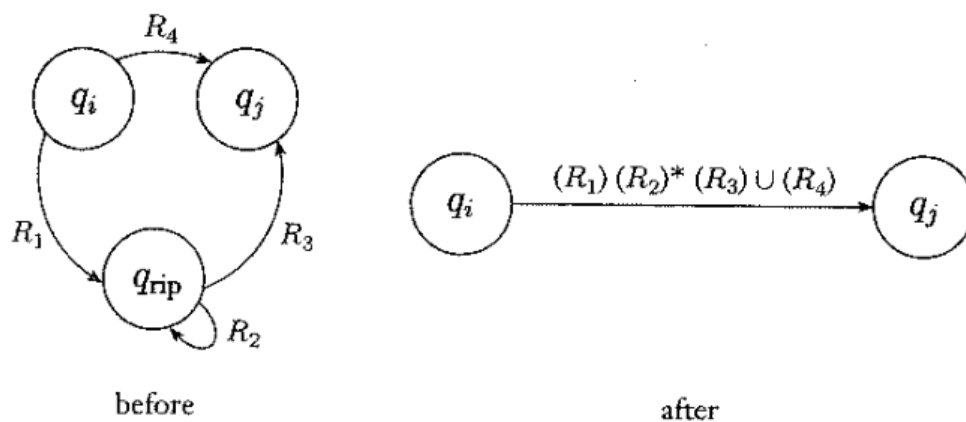
Possiamo facilmente trasformare un DFA in un GNFA. Aggiungiamo semplicemente un nuovo stato iniziale con un ϵ -arco che entra nel vecchio stato iniziale e un nuovo stato accettante con ϵ -archi entranti, provenienti dai vecchi stati accettanti. Se alcuni archi hanno più etichette (o se ci sono più archi che collegano gli stessi due stati nella stessa direzione), sostituiamo ognuno di essi con un solo arco la cui etichetta è l'unione delle precedenti etichette. Infine, aggiungiamo archi con etichetta $\emptyset$ tra stati che non hanno archi. Quest'ultimo non cambierebbe il linguaggio riconosciuto perchè una transizione etichettata con $\emptyset$ non può mai essere usata.

Ora mostriamo come trasformare un GNFA in un'espressione regolare. Supponiamo che il GNFA abbia k stati. Allora, poichè un GNFA deve avere uno stato

iniziale e uno stato accettante ed essi devono essere diversi tra loro, sappiamo che $k \geq 2$. Se $k > 2$, costruiamo un GNFA equivalente con $k - 1$ stati. Questo passo può essere ripetuto sul nuovo GNFA fino a quando esso è ridotto a 2 stati. Se $k = 2$, il GNFA ha un solo arco che va dallo stato iniziale allo stato accettante. L'etichetta di questo arco è l'espressione regolare equivalente.

Il passo cruciale è costruire un GNFA equivalente con uno stato in meno quando $k > 2$. Lo facciamo scegliendo uno stato, estraendolo dalla macchina, e modificando il resto in modo che sia riconosciuto lo stesso linguaggio. Ogni stato può essere scelto, purché non sia lo stato iniziale o lo stato accettante. Chiamiamo q_{rip} lo stato rimosso.

Dopo aver rimosso q_{rip} modifichiamo la macchina cambiando le espressioni regolari che etichettano ciascuno dei restanti archi. Le nuove etichette controbilanciano l'assenza di q_{rip} reintegrando le computazioni perse. La nuova etichetta che va da uno stato q_i a uno stato q_j è un'espressione regolare che descrive tutte le stringhe che porterebbero la macchina da q_i a q_j o direttamente o tramite q_{rip} . Analizziamo tale procedimento nella seguente figura.



Nella vecchia macchina, se

1. q_i va a q_{rip} con un arco etichettato R_1 ;
2. q_{rip} va in sé stesso con un arco etichettato R_2 ;
3. q_{rip} va a q_j con un arco etichettato R_3 ;
4. q_i va a q_j con un arco etichettato R_4 ,

allora nella nuova macchina, l'arco da q_i a q_j riceve l'etichetta

$$(R_1)(R_2)^*(R_3) \cup (R_4)$$

Facciamo questa modifica per ogni arco che va da un qualsiasi stato q_i a un qualsiasi stato q_j , includendo il caso in cui $q_i = q_j$. La nuova macchina riconosce il linguaggio di partenza.

Dimostrazione. Formalizziamo quest'idea. In primo luogo, definiamo formalmente l'automa GNFA. Un GNFA è simile ad un automa finito non deterministico tranne che per la funzione di transizione, che ha la forma

$$\delta : (Q - \{q_{accept}\}) \times (Q - \{q_{start}\}) \rightarrow R$$

Il simbolo R è la collezione di tutte le espressioni regolari sull'alfabeto Σ e q_{start} e q_{accept} sono gli stati iniziale e accettante.

Il dominio della funzione di transizione è $(Q - \{q_{accept}\}) \times (Q - \{q_{start}\})$ perchè un arco collega ogni stato a un qualsiasi altro stato, tranne che nessun arco è uscente da q_{accept} o è entrante in q_{start} . $\square$

DEFINIZIONE 1.6: AUTOMA FINITO NON DETERMINISTICO GENERALIZZATO

Un *automa finito non deterministico generalizzato* è una quintupla $(Q, \Sigma, \delta, q_{start}, q_{accept})$, dove

1. Q è l'insieme finito degli stati;
2. Σ è l'alfabeto di input;
3. $\delta : (Q - \{q_{accept}\}) \times (Q - \{q_{start}\}) \rightarrow R$ è la funzione di transizione;
4. q_{start} è lo stato iniziale;
5. q_{accept} è lo stato accettante.

Un GNFA accetta una stringa w in Σ^* se $w = w_1 w_2 \cdots w_k$, dove ogni w_i è in Σ^* ed esiste una sequenza di stati $q_0, q_1, \dots, q_k$ tale che

1. $q_0 = q_{start}$ è lo stato iniziale,
2. $q_k = q_{accept}$ è lo stato accettante e
3. per ogni i , risulta $w_i \in L(R_i)$, dove $R_i = \delta(q_{i-1}, q_i)$

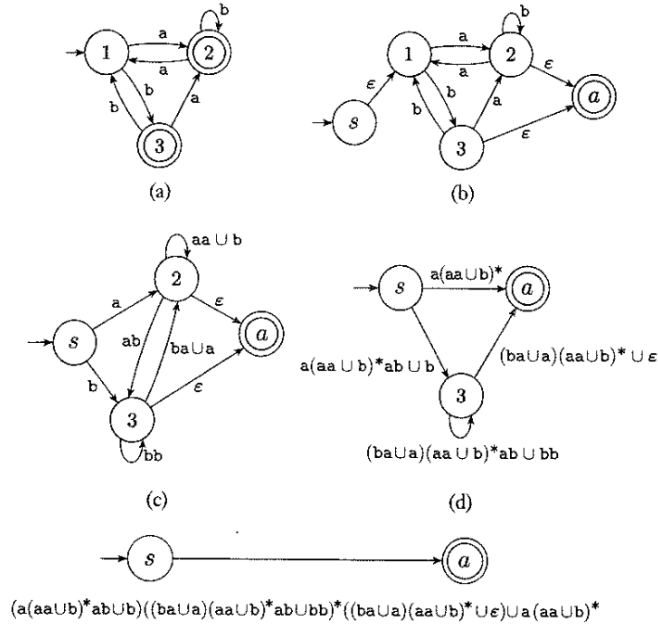
Torniamo alla dimostrazione del **lemma 1.2**, sia M il DFA per il linguaggio A . Allora trasformiamo M in un GNFA G aggiungendo un nuovo stato iniziale un nuovo stato accettante e i necessari archi di transizione supplementari. Usiamo la procedura $\text{CONVERT}(G)$, che prende un GNFA e restituisce un'espressione regolare equivalente. Questa procedura usa la *ricorsione*.

```

1 CONVERT(G) :
2   if k = 2 then return R;
3   if k > 2 then scelto  $q_{rip} \neq q_{accept}$  e  $q_{start}$ 
4     CONVERT(G')
5    $G' = (Q', \Sigma, \delta, q_{start}, q_{accept})$ 
6    $Q' = Q - \{q_{rip}\}$ 
7   foreach  $q_i \in Q' - \{q_{accept}\}$  and  $q_j \in Q' - \{q_{start}\}$ 
8      $\delta'(q_i, q_j) = (R_1)(R_2)^*(R_3) \cup (R_4)$ 
9      $R_1 = \delta(q_i, q_{rip})$ 
10     $R_2 = \delta(q_{rip}, q_{rip})$ 
11     $R_3 = \delta(q_{rip}, q_j)$ 
12     $R_4 = \delta(q_i, q_j)$ 
13   endfor

```

Analizziamo un esempio, iniziamo con un DFA con tre stati. I passi nella trasformazione sono mostrati nella figura seguente.



1.4 Linguaggi non regolari

In questa sezione, mostriamo come provare che alcuni linguaggi non possono essere riconosciuti da alcun automa finito. Consideriamo il linguaggio $B = \{0^n 1^n \mid n \geq 0\}$. Se cerchiamo di trovare un DFA che riconosca B , scopriamo che la macchina sembra aver bisogno di ricordare quanti simboli uguali a 0 ha visto fin quando legge l'input. Poichè il numero di simboli uguali a 0 non è limitato, la macchina dovrà tenere traccia di un numero infinito di possibilità. Ma non può farlo con un numero finito di stati.

Non è detto che se un linguaggio sembra richiedere memoria non limitata allora è un linguaggio non regolare. Per esempio, consideriamo i due linguaggi sull'alfabeto $\Sigma = \{0, 1\}$:

- $C = \{w \mid w \text{ ha lo stesso numero di simboli uguali a 0 e simboli uguali a 1}\}$
- $D = \{w \mid w \text{ ha un numero uguale di occorrenze di 01 e 10 come sottostringhe}\}$

A prima vista, una macchina che riconosce sembra aver bisogno di contare in ciascun caso,, e quindi nessuno dei due linguaggi sembra essere regolare.

Sappiamo che C non è regolare, ma D è regolare.

1.4.1 Il pumping lemma per i linguaggi regolari

La tecnica per provare la regolarità deriva da un teorema sui linguaggi regolari, chiamato il *pumping lemma*. Questo teorema afferma che tutti i linguaggi regolari hanno una proprietà speciale. Se noi possiamo mostrare che un linguaggio non ha questa proprietà, siamo sicuri che esso non è regolare. La proprietà afferma che tutte le stringhe nel linguaggio possono essere “replicate” se la loro lunghezza raggiunge almeno uno specifico valore speciale, chiamato la *lunghezza del pumping*.

Teorema 1.8: Pumping lemma per linguaggi regolari

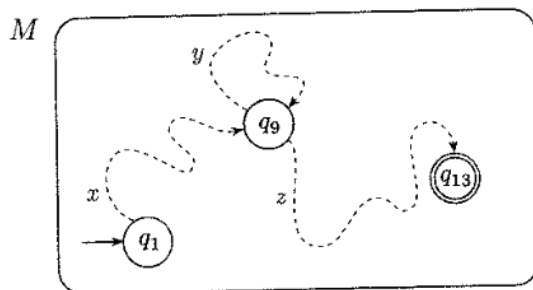
Se A è un linguaggio regolare, allora esiste p (lunghezza del pumping) tale che se s è una qualsiasi stringa in A di lunghezza almeno p , allora s può essere divisa in tre parti, $s = xyz$, soddisfacenti le seguenti condizioni:

1. $\forall i \geq 0, xy^i z \in A$
2. $|y| > 0$
3. $|xy| \leq p$

Sia $M = (Q, \Sigma, \delta, q_1, F)$ un DFA che riconosce A . Assegniamo alla lunghezza del pumping p il numero degli stati di M . Mostriamo che per ogni stringa s in A di lunghezza almeno p può essere divisa nelle tre parti xyz , soddisfacenti le tre condizioni.

Se $s \in A$ ha lunghezza almeno p , consideriamo la sequenza di stati che M attraversa nella computazione con input s . Inizia con lo stato iniziale q_1 , poi va per esempio in q_3 , poi in q_{20} , e così via, finché raggiunge la fine di s nello stato q_{13} . Se s è in A , sappiamo che M accetta s , quindi q_{13} è uno stato accettante. Se supponiamo che n sia la lunghezza di s , la sequenza di stati $q_1 q_3 q_{20} \dots q_{13}$ ha lunghezza $n + 1$. Poiché n è almeno p , sappiamo che $n + 1 > p$, il numero di stati di M . Quindi, la sequenza deve contenere uno stato che si ripete. Questo risultato è un esempio del *principio della piccionaia*.

Di seguito viene mostrato un esempio di un DFA con uno stato che si ripete.



Ora dividiamo s nelle tre componenti x , y e z . La componente x è la parte di s che compare prima di q_9 , la componente y è la parte tra le due occorrenze di q_9 e la componente z è la parte restante di s , che viene dopo la seconda occorrenza di q_9 .

Vediamo perché questa suddivisione di s soddisfa le tre condizioni. Supponiamo di eseguire M sull'input $xyyz$. Sappiamo che x porta M da q_1 a q_9 , e poi il primo y lo riporta da q_9 a q_9 , come fa il secondo y e poi z lo porta in q_{13} che accetta $xyyz$ in quanto stato accettante. Analogamente, esso accetterà $xy^i z \forall i > 0$.

Per verificare la condizione 2, vediamo che $|y| > 0$, poiché era la parte di s tra due diverse occorrenze dello stato q_9 . Per ottenere la condizione 3, ci assicuriamo che q_9 sia la prima ripetizione nella sequenza. Per il principio della piccionaia, i primi $p + 1$ stati nella sequenza devono contenere una ripetizione. Quindi $|xy| \leq p$.

Dimostrazione. Sia $M = (Q, \Sigma, \delta, q_1, F)$ un DFA che riconosce A e sia p il numero di stati di M .

Sia $s = s_1 s_2 \dots s_n$ una stringa in A di lunghezza n , dove $n \geq p$. Sia $r_1, \dots, r_{n+1}$ la sequenza di stati attraversati da M mentre elabora s , quindi $r_{i+1} = \delta(r_i, s_i)$ per $1 \leq i \leq n$. Questa sequenza ha lunghezza $n + 1$, che è almeno $p + 1$. Due stati tra i primi $p + 1$ elementi nella sequenza devono essere lo stesso stato, per il principio della piccioni. Chiamiamo il primo di questi r_j e il secondo r_l . Poichè r_l si presenta tra le prime $p + 1$ posizioni in una sequenza che inizia in r_1 , abbiamo $l \leq p + 1$. Ora sia $x = s_1 \dots s_{j-1}$, $y = s_j \dots s_{l-1}$ e $z = s_l \dots s_n$.

Poichè x porta M da r_1 a r_j , y porta M da r_j a r_j e z porta M da r_j a r_{n+1} , che è uno stato accettante, M deve accettare $xy^i z$ per $i \geq 0$. Sappiamo che $j \neq l$, perciò $|y| > 0$ e $l \leq p + 1$, perciò $|xy| \leq p$. Quindi tutte le condizioni del pumping lemma sono rispettate. $\square$

Per usare il pumping lemma per provare che un linguaggio B non è regolare, in primo luogo si assuma che B sia regolare per ottenere una contraddizione. Poi si usi il pumping lemma per assicurare l'esistenza di una lunghezza del pumping p tale che tutte le stringhe di lunghezza maggiore o uguale a p in B possano essere iterate. In seguito, si trovi una stringa s in B che ha una lunghezza maggiore o uguale a p , ma che non può essere iterata. Infine, si dimostri che s non può essere iterata considerando tutti i modi di dividere s in x, y e z e, per ogni tale divisione, trovando un valore i tale che $xy^i z \notin B$.

Esempio

Sia $B = \{0^n 1^n \mid n \geq 0\}$. Usiamo il pumping lemma per provare che B non è regolare.

Assumiamo che B sia regolare. Sia p la lunghezza del pumping data dal pumping lemma. Scegliamo $s = 0^p 1^p$. Poichè $s \in B$ ed $|s| \geq p$, il pumping lemma assicura che s può essere divisa in tre parti, $s = xyz$, tali che per ogni $i \geq 0$ la stringa $xy^i z \in B$. Consideriamo tre casi per mostrare che questo risultato è impossibile.

1. La stringa y consiste solo di simboli uguali a 0. In questo caso, la stringa $xyyz$ ha più simboli uguali a 0 che simboli uguali a 1 e quindi non è un elemento di B , violando la condizione 1 del pumping lemma.
2. La stringa y consiste solo di simboli uguali a 1. Anche questo caso porta ad una contraddizione.

3. La stringa y consiste solo di simboli uguali a 0 che di simboli uguali a 1. In questo caso, la stringa $xyyz$ può avere lo stesso numero di simboli uguali a 0 e simboli uguali a 1, ma essi non saranno nell'ordine corretto, con qualche 1 prima di qualche 0. Quindi non è un elemento di B , il che è una contraddizione.

Pertanto una contraddizione è inevitabile se assumiamo che B sia regolare, quindi B non è regolare.

Esempio

Sia $C = \{w \mid w \text{ ha lo stesso numero di simboli uguali a 0 e simboli uguali a 1}\}$. Usiamo il pumping lemma per mostrare che C non è regolare.

Assumiamo al contrario che C sia regolare. Sia p la lunghezza del pumping data dal pumping lemma. Sia s la stringa $0^p 1^p$. Poichè $s \in C$ che ha lunghezza maggiore di p , il pumping lemma assicura che s può essere divisa in tre parti, $s = xyz$, tali che per ogni $i \geq 0$ la stringa $xy^i z \in C$. Se prendiamo x e z uguali alla stringa vuota e y uguale alla stringa $0^p 1^p$, allora $xy^i z$ ha sempre lo stesso numero di simboli uguali a 0 e simboli uguali a 1 e quindi è in C .

Qui la condizione 3 del pumping lemma è utile. Essa stabilisce che quando iteriamo s , essa deve essere divisa in modo che $|xy| \leq p$. Questa restrizione sul modo in cui s può essere divisa rende più facile mostrare che la stringa $s = 0^p 1^p$ che abbiamo scelto non può essere iterata. Se $|xy| \leq p$, allora y deve consistere solo di simboli uguali a 0, perciò $xyyz \notin C$. Quindi, s non può essere iterata.

Scegliere la stringa s in questo esempio richiede più attenzione. Se avessimo scelto $s = (01)^p$, avremmo avuto dei problemi perchè abbiamo bisogno di una stringa che **non può essere iterata** e quella stringa può essere iterata, perfino prendendo in considerazione la condizione 3.

Un metodo alternativo per provare che C non è regolare segue dal fatto che sappiamo che B non è regolare. Se C fosse regolare, anche $C \cap 0^* 1^*$ sarebbe regolare. Questo perchè il linguaggio $0^* 1^*$ è regolare e la classe dei linguaggi regolari è chiusa rispetto all'intersezione.

Capitolo 2

Linguaggi context-free

2.1 Grammatiche context-free

Un esempio di grammatica context-free, che chiamiamo G_1 , è il seguente:

$$A \rightarrow 0A1$$

$$B \rightarrow B$$

$$B \rightarrow \#$$

Una grammatica consiste di un insieme di *regole di sostituzione*, anche chiamate *produzioni*. Tutte le stringhe generate da una grammatica costituiscono il *linguaggio della grammatica*. Denoteremo con $L(G_1)$ il linguaggio della grammatica G_1 .

Ogni linguaggio che può essere generato da una grammatica context-free è chiamato un *linguaggio context-free* (CFL).

DEFINIZIONE 2.1: GRAMMATICA CONTEXT-FREE

Una *grammatica context-free* è una quadrupla (V, Σ, R, S) , dove

1. V è un insieme finito i cui elementi sono chiamati *variabili*,
2. Σ è un insieme finito, disgiunto da V , i cui elementi sono chiamati *terminali*,
3. R è un insieme finito di *regole*, dove ciascuna regola è una variabile e una stringa di variabili e terminali,
4. $S \in V$ è la variabile iniziale.

Se u, v e w sono stringhe di variabili e terminali e $A \rightarrow w$ è una regola della grammatica, diciamo che uAv *produce* uwv , e lo denotiamo con $uAv \Rightarrow uwv$. Diciamo che u *deriva* v , e lo denotiamo con $u \Rightarrow^* v$, se $u = v$ o se esiste una sequenza $u_1, u_2, \dots, u_k$ con $k \geq 0$ e

$$u \Rightarrow u_1 \Rightarrow u_2 \Rightarrow \dots \Rightarrow u_k \Rightarrow v.$$

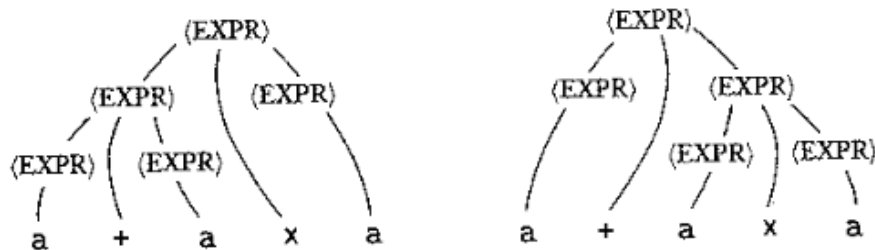
Il *linguaggio della grammatica* è $\{w \in \Sigma^* \mid S \Rightarrow^* w\}$

2.1.1 Ambiguità

Qualche volta una grammatica può generare la stessa stringa in più modi diversi. Una tale stringa avrà diversi alberi sintattici e quindi diversi significati. Questo risultato può essere indesiderabile per alcune applicazioni, come i linguaggi di programmazione, dove un programma dovrebbe avere un'unica interpretazione. Se una grammatica genera la stessa stringa in più modi diversi, diciamo che la stringa è derivata *ambiguamente* in quella grammatica. Se una grammatica genera alcune stringhe ambiguamente, diciamo che la grammatica è *ambigua*. Per esempio, consideriamo la grammatica G_5 :

$$\langle \text{EXPR} \rangle \rightarrow \langle \text{EXPR} \rangle + \langle \text{EXPR} \rangle \mid \langle \text{EXPR} \rangle \times \langle \text{EXPR} \rangle \mid (\langle \text{EXPR} \rangle) \mid a$$

Questa grammatica genera la stringa $a + a \times a$ ambiguamente.



Questa grammatica non tiene conto delle usuai regole di precedenza e quindi può raggruppare $+$ prima di $\times$ o viceversa. Al contrario la seguente grammatica G_4 genera esattamente lo stesso linguaggio, ma ogni stringa generata ha un solo albero sintattico. $G_4 = (V, \Sigma, R, \langle \text{EXPR} \rangle)$ ove $V = \{\langle \text{EXPR} \rangle, \langle \text{TERM} \rangle, \langle \text{FACTOR} \rangle\}$

e $\Sigma = \{a, +, \times, (,)\}$. Le regole sono

$$\begin{aligned}\langle \text{EXPR} \rangle &\rightarrow \langle \text{EXPR} \rangle + \langle \text{TERM} \rangle \mid \langle \text{TERM} \rangle \\ \langle \text{TERM} \rangle &\rightarrow \langle \text{TERM} \rangle \times \langle \text{FACTOR} \rangle \mid \langle \text{FACTOR} \rangle \\ \langle \text{FACTOR} \rangle &\rightarrow (\langle \text{EXPR} \rangle) \mid a\end{aligned}$$

Quando diciamo che una grammatica genera ambigualmente una stringa, intendiamo che la stringa ha due diversi alberi sintattici, non due differenti derivazioni. Due derivazioni possono differire solo nell'ordine in cui esse sostituiscono le variabili ma non nella loro struttura complessiva. Una derivazione di una stringa w in una grammatica G è una *derivazione a sinistra* se a ogni passo la variabile sostituita è quella che si trova più a sinistra.

DEFINIZIONE 2.2: GRAMMATICA AMBIGUA

Una stringa w è derivata *ambiguamente* in una grammatica context-free G se essa ha due o più diverse derivazioni a sinistra. Una grammatica G è *ambigua* se essa genera qualche stringa ambigualmente.

Quando abbiamo una grammatica ambigua, possiamo trovare una grammatica non ambigua che genera lo stesso linguaggio. Tuttavia, alcuni linguaggi context-free possono essere generati solo da grammatiche ambigue. Tali linguaggi sono chiamati *inerentemente ambigui*.

2.1.2 Forma normale di Chomsky

DEFINIZIONE 2.3: FORMA NORMALE DI CHOMSKY

Una grammatica context-free è in *forma normale di Chomsky* se ogni regola è della forma

$$\begin{aligned}A &\rightarrow BC \\ A &\rightarrow a\end{aligned}$$

dove a è un terminale e A, B e C sono variabili qualsiasi tanne che B e C non possono essere la variabile iniziale.

Inoltre, permettiamo la regola $S \rightarrow \epsilon$, dove S è la variabile iniziale.

Teorema 2.1

Ogni linguaggio context-free è generato da una grammatica context-free in forma normale di Chomsky.

Possiamo trasformare ogni grammatica G in forma normale di Chomsky. La trasformazione ha diversi passi nei quali le regole che violano le condizioni sono rimpiazzate con regole equivalenti che sono adeguate. Innanzitutto, aggiungiamo una nuova variabile iniziale. Poi, eliminiamo tutte le ϵ -regole della forma $A \rightarrow \epsilon$. Eliminiamo anche tutte le *regole unitarie* della forma $A \rightarrow B$. In entrambi i casi, modifichiamo la grammatica in modo da essere sicuri che generi ancora lo stesso linguaggio. Infine, trasformiamo le restanti regole nella forma adeguata.

Dimostrazione. In primo luogo, aggiungiamo una nuova variabile S_0 e la regola $S_0 \rightarrow S$, dove S era la variabile iniziale di partenza. Questo cambiamento garantisce che la variabile iniziale non compare sul lato destro di una regola. In secondo luogo, ci occupiamo di tutte le ϵ -regole. Eliminiamo una ϵ -regola $A \rightarrow \epsilon$, dove A non è la variabile iniziale. Poi, per ogni occorrenza di A sul lato destro di una regola, aggiungiamo una nuova regola con quell'occorrenza cancellata. In altre parole, se $R \rightarrow uAv$ è una regola in cui u e v sono stringhe di variabili e terminali, aggiungiamo la regola $R \rightarrow uv$. Facciamo questo per ogni occorrenza di A , in modo che la regola $R \rightarrow uAvAw$ faccia sì che vengano aggiunte $R \rightarrow uvAw$, $R \rightarrow uAvw$ ed $R \rightarrow uvw$. Se abbiamo la regola $R \rightarrow A$, aggiungiamo $R \rightarrow \epsilon$ a meno che non avevamo precedentemente rimosso la regola $R \rightarrow \epsilon$.

Inoltre, ci occupiamo delle regole unitarie. Eliminiamo una regola unitaria $A \rightarrow B$. Poi, per ogni regola $B \rightarrow u$, aggiungiamo la regola $A \rightarrow u$ a meno che questa non sia una regola unitaria precedentemente cancellata.

Infine, trasformiamo tutte le restanti regole nella forma appropriata. Rimpiazziamo ogni regola $A \rightarrow u_1u_2 \dots u_k$, dove $k \geq 3$ e ciascun u_i è una variabile o un simbolo terminale, con le regole $A \rightarrow u_1A_1$, $A_1 \rightarrow u_2A_2$, $A_2 \rightarrow u_3A_3, \dots$ e $A_{k-2} \rightarrow u_{k-1}u_k$. Le A_i sono nuove variabili. Rimpiazziamo ogni terminale u_i nelle precedenti regole con la nuova variabile U_i e aggiungiamo la regola $U_i \rightarrow u_i$. $\square$

Esempio

Consideriamo la seguente CFG G_6 e trasformiamola in forma normale di Chomsky usando la procedura di conversione appena fornita.

1. La CFG iniziale G_6 è mostrata di seguito. Il risultato dell'applicazione

del primo passo è mostrato sotto.

$$\begin{array}{l} S \rightarrow ASA \mid aB \\ A \rightarrow B \mid S \\ B \rightarrow b \mid \epsilon \end{array}$$

che diventa

$$\begin{array}{l} S_0 \rightarrow S \\ S \rightarrow ASA \mid aB \\ A \rightarrow B \mid S \\ B \rightarrow b \mid \epsilon \end{array}$$

2. Eliminiamo le ϵ -regole $B \rightarrow \epsilon$.

$$\begin{array}{l} S_0 \rightarrow S \\ S \rightarrow ASA \mid aB \mid a \\ A \rightarrow B \mid S \mid \epsilon \\ B \rightarrow b \mid \epsilon \end{array}$$

Eliminando la regola $A \rightarrow \epsilon$ otteniamo:

$$\begin{array}{l} S_0 \rightarrow S \\ S \rightarrow ASA \mid aB \mid a \mid SA \mid AS \mid S \\ A \rightarrow B \mid S \mid \epsilon \\ B \rightarrow b \end{array}$$

3. Eliminiamo la regola unitaria $S \rightarrow S$.

$$\begin{array}{l} S_0 \rightarrow S \\ S \rightarrow ASA \mid aB \mid a \mid SA \mid AS \\ A \rightarrow B \mid S \mid \epsilon \\ B \rightarrow b \end{array}$$

Eliminiamo la regola unitaria $S_0 \rightarrow S$.

$$\begin{array}{l} S_0 \rightarrow ASA \mid aB \mid a \mid SA \mid AS \\ S \rightarrow ASA \mid aB \mid a \mid SA \mid AS \\ A \rightarrow B \mid S \mid \epsilon \\ B \rightarrow b \end{array}$$

Eliminiamo le regole unitarie $A \rightarrow B$ e $A \rightarrow S$.

$$\begin{aligned}S_0 &\rightarrow ASA \mid aB \mid a \mid SA \mid AS \\S &\rightarrow ASA \mid aB \mid a \mid SA \mid AS \\A &\rightarrow b \mid ASA \mid aB \mid a \mid SA \mid AS \\B &\rightarrow b\end{aligned}$$

4. Trasforiamo le rimanenti regole nella forma corretta, aggiungendo variabili e regole supplementari.

$$\begin{aligned}S_0 &\rightarrow AA_1 \mid UB \mid a \mid SA \mid AS \\S &\rightarrow AA_1 \mid UB \mid a \mid SA \mid AS \\A &\rightarrow b \mid AA_1 \mid UB \mid a \mid SA \mid AS \\A_1 &\rightarrow SA \\U &\rightarrow a \\B &\rightarrow b\end{aligned}$$

La CNF è utile per risolvere alcuni problemi dei linguaggi context-free, ad esempio il cosiddetto *membership problem* ed è anche utile per provare che alcuni linguaggi sono decidibili.

2.1.3 Membership problem

Il membership problem viene risolto mediante l'algoritmo CYK che permette di stabilire se una stringa w appartiene ad una data grammatica $G = (V, \Sigma, P, S)$. L'algoritmo CYK è un algoritmo di programmazione dinamica in quanto la soluzione al problema $[i, j]$ è costruita dalla combinazione della soluzione al problema $[i, k]$ e la soluzione al problema $[k, j]$.

La grammatica deve essere in CNF in quanto ogni problema è suddiviso in due sottoproblemi e non di più, per limitare la complessità.

Viene usata una tecnica chiamata "*table-filling algorithm*". Viene costruita una tabella triangolare ove per ogni $X_{i,j}$, i rappresenta la lunghezza della stringa e j l'indice di partenza.

$x_{5,1}$				
$x_{4,1}$	$x_{4,2}$			
$x_{3,1}$	$x_{3,2}$	$x_{3,3}$		
$x_{2,1}$	$x_{2,2}$	$x_{2,3}$	$x_{2,4}$	
$x_{1,1}$	$x_{1,2}$	$x_{1,3}$	$x_{1,4}$	$x_{1,5}$
w_1	w_2	w_3	w_4	w_5

Esempio

Sia G la grammatica context-free in CNF con le seguenti regole:

$$S \rightarrow AB \mid BC$$

$$A \rightarrow BA \mid a$$

$$B \rightarrow CC \mid b$$

$$C \rightarrow AB \mid a$$

e sia $w = baaba$.

$\{B\}$	$\{A, C\}$	$\{A, C\}$	$\{B\}$	$\{A, C\}$
b	a	a	b	a

Adesso troviamo:

$$X_{2,1} = (X_{1,1}, X_{1,2}) \rightarrow \{B\}\{A, C\} = \{BA, BC\}$$

$$X_{2,2} = (X_{1,2}, X_{1,3}) \rightarrow \{A, C\}\{A, C\} = \{AA, AC, CA, CC\}$$

$$X_{2,3} = (X_{1,3}, X_{1,4}) \rightarrow \{A, C\}\{B\} = \{AB, CB\}$$

$$X_{2,4} = (X_{1,4}, X_{1,5}) \rightarrow \{B\}\{A, C\} = \{BA, BC\}$$

Quindi, dobbiamo trovare le regole che producono BA e BC , AA , AC , CA , CC e così via.

$\{S, A\}$	$\{B\}$	$\{S, C\}$	$\{S, A\}$	
$\{B\}$	$\{A, C\}$	$\{A, C\}$	$\{B\}$	$\{A, C\}$
b	a	a	b	a

Adesso troviamo:

$$X_{3,1} = (X_{1,1}, X_{2,2}), (X_{2,1}, X_{1,3}) \rightarrow \{B\}\{B\} \cup \{S, A\}\{A, C\} = \{BB, SA, SC, AA, AC\}$$

$$X_{3,2} = (X_{1,2}, X_{2,3}), (X_{2,2}, X_{1,4}) \rightarrow \{A, C\}\{S, C\} \cup \{B\}\{B\} = \{AS, AC, CS, CC, BB\}$$

$$X_{3,3} = (X_{1,3}, X_{2,4}), (X_{2,3}, X_{1,5}) \rightarrow \{A, C\}\{S, A\} \cup \{S, C\}\{A, C\} = \{AS, AA, CS, CA, SA, SC, C$$

Quindi, dobbiamo trovare le regole che producono BB, SA, SC, AA e così via.

$\emptyset$	$\{B\}$	$\{B\}$		
$\{S, A\}$	$\{B\}$	$\{S, C\}$	$\{S, A\}$	
$\{B\}$	$\{A, C\}$	$\{A, C\}$	$\{B\}$	$\{A, C\}$
b	a	a	b	a

Adesso troviamo:

$$X_{4,1} = (X_{1,1}, X_{3,2}), (X_{2,1}, X_{2,3}), (X_{3,1}, X_{1,4}) = \{BB, SS, SC, AS, AC, B\}$$

$$X_{4,2} = (X_{1,2}, X_{3,3}), (X_{2,2}, X_{2,4}), (X_{3,2}, X_{1,5}) = \{AB, CB, BS, BA, BC\}$$

$\{S, A, C\}$				
$\emptyset$	$\{S, A, C\}$			
$\emptyset$	$\{B\}$	$\{B\}$		
$\{S, A\}$	$\{B\}$	$\{S, C\}$	$\{S, A\}$	
$\{B\}$	$\{A, C\}$	$\{A, C\}$	$\{B\}$	$\{A, C\}$
b	a	a	b	a

2.1.4 Proprietà dei linguaggi context-free

Teorema 2.2: Chiusura rispetto all'operazione di unione

I linguaggi context-free sono chiusi rispetto all'operazione di unione.

Se L_1 e L_2 sono linguaggi context-free allora $L_1 \cup L_2$ è context-free.

Esempio

Sia $L_1 = \{a^n b^n\}$ e sia $L_2 = \{ww^R\}$ ove le grammatiche sono rispettivamente:

$$\begin{aligned} S_1 &\rightarrow aS_1b \mid \lambda \\ S_2 &\rightarrow aS_2a \mid bS_2b \mid \lambda \end{aligned}$$

e l'unione è:

$$S \rightarrow S_1 \mid S_2$$

Teorema 2.3: Chiusura rispetto all'operazione di concatenazione

I linguaggi context-free sono chiusi rispetto all'operazione di concatenazione.

Se L_1 e L_2 sono linguaggi context-free allora $L_1 L_2$ è context-free.

Esempio

Sia $L_1 = \{a^n b^n\}$ e sia $L_2 = \{ww^R\}$ ove le grammatiche sono rispettivamente:

$$\begin{aligned} S_1 &\rightarrow aS_1b \mid \lambda \\ S_2 &\rightarrow aS_2a \mid bS_2b \mid \lambda \end{aligned}$$

e la concatenazione è:

$$S \rightarrow S_1 S_2$$

Teorema 2.4: Chiusura rispetto all'operazione *

I linguaggi context-free sono chiusi rispetto all'operazione *.

Se L è un linguaggio context-free allora L^* è context-free.

Esempio

Sia $L = \{a^n b^n\}$ ove la grammatica è:

$$S \rightarrow aSb \mid \lambda$$

e la grammatica di L^* è:

$$\begin{aligned} S_1 &\rightarrow SS_1 \mid \lambda \\ S &\rightarrow aSb \mid \lambda \end{aligned}$$

I linguaggi context-free invece **non** sono chiusi rispetto all'operazione di intersezione.

Esempio

Sia $L_1 = \{a^n b^n c^m\}$ e sia $L_2 = \{a^n b^m c^m\}$ ove le grammatiche sono rispettivamente:

$$\begin{aligned} S &\rightarrow AC \\ A &\rightarrow aAb \mid \lambda \\ C &\rightarrow cC \mid \lambda \end{aligned}$$

e

$$\begin{aligned} S &\rightarrow AB \\ A &\rightarrow aA \mid \lambda \\ B &\rightarrow bBc \mid \lambda \end{aligned}$$

L'intersezione è $L_1 \cap L_2 = \{a^n b^n c^n\}$ che non è context-free.

Inoltre, i linguaggi context-free **non** sono chiusi rispetto all'operazione di complemento.

Esempio

Sia $L_1 = \{a^n b^n c^m\}$ e sia $L_2 = \{a^n b^m c^m\}$ ove le grammatiche sono rispettivamente:

$$\begin{aligned} S &\rightarrow AC \\ A &\rightarrow aAb \mid \lambda \\ C &\rightarrow cC \mid \lambda \end{aligned}$$

e

$$\begin{aligned} S &\rightarrow AB \\ A &\rightarrow aA \mid \lambda \\ B &\rightarrow bBc \mid \lambda \end{aligned}$$

Il complemento è pari a:

$$\overline{\overline{L_1} \cup \overline{L_2}} = L_1 \cap L_2 = \{a^n b^n c^n\}$$

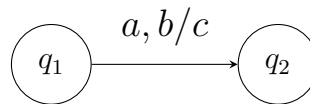
che non è context-free.

Intersezione tra linguaggi context-free e linguaggi regolari

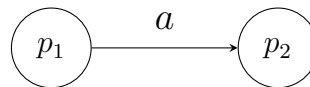
Teorema 2.5

L'intersezione tra linguaggi context-free e linguaggi regolari è un linguaggio context-free.

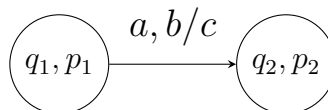
Se L_1 è un linguaggio context-free e L_2 è un linguaggio regolare allora $L_1 \cap L_2$ è context-free. Il linguaggio L_1 è riconosciuto da un PDA e il linguaggio L_2 è riconosciuto da un DFA. Quindi vogliamo costruire un PDA che riconosce $L_1 \cap L_2$. Il PDA per M_1 è il seguente:



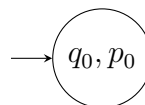
Il DFA per L_2 è il seguente:



e il PDA per $L_1 \cap L_2$ è il seguente:



Per gli stati iniziali invece, se q_0 è lo stato iniziale del PDA che riconosce L_1 e p_0 è lo stato iniziale del DFA che riconosce L_2 allora lo stato iniziale del PDA che riconosce $L_1 \cap L_2$ è:



Per gli stati finali, invece, se q_1 è uno stato finale del PDA che riconosce L_1 e p_1 e p_2 sono stati finali del DFA che riconosce L_2 allora gli stati finali del PDA che riconosce $L_1 \cap L_2$ sono:



Il PDA M che riconosce $L_1 \cap L_2$ simula in parallelo il PDA M_1 che riconosce L_1 e il DFA M_2 che riconosce L_2 e accetta una stringa solo se M_1 accetta la stringa e M_2 accetta la stringa, ovvero:

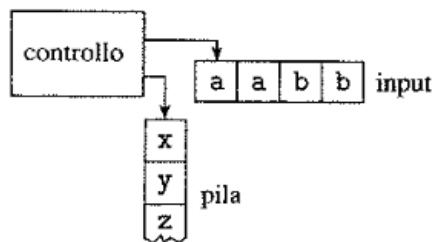
$$L(M) = L(M_1) \cap L(M_2)$$

Quindi, dato che M è PDA, $L_1 \cap L_2$ è context-free.

2.2 Automi a pila

Gli *automi a pila* (*pushdown automata*) sono come gli automi finiti non deterministici ma hanno una componente in più chiamata *pila* (*stack*). La pila fornisce memoria aggiuntiva oltre alla quantità finita di essa disponibile nel controllo. La pila consente a tali automi di riconoscere alcuni linguaggi non regolari.

Gli automi a pila sono computazionalmente equivalenti alle grammatiche context-free. Questa equivalenza è utile poichè essa ci dà due alternative per provare che un linguaggio è context-free. La figura seguente è una rappresentazione schematica di un automa finito. Il controllo rappresenta gli stati e la funzione di transizione, il nastro contiene la stringa di input, e la freccia rappresenta la testina sull'input, che indica il successivo simbolo di input da leggere.



Un PDA può scrivere simboli nella pila e rileggerli in seguito. Scrivere un simbolo “spinge giù” tutti gli altri simboli nella pila. In un qualunque momento il simbolo sulla cima (*top*) della pila può essere letto e rimosso. I rimanenti simboli allora

ritornano più in alto. L'operazione di scrivere un simbolo dalla pila è spesso chiamata *push*, quella di eliminare un simbolo dalla pila è spesso chiamata *pop*. Una pila è molto importante perchè può mantenere una quantità non limitata di informazioni. Ricordiamo che un automa finito non può riconoscere il linguaggio $\{0^n 1^n \mid n \geq 0\}$ poichè non può memorizzare numeri molto grandi nella sua memoria finita. Un PDA è in grado di riconoscere questo linguaggio poichè può usare la sua pila per memorizzare il numero di simboli uguali a 0 che ha visto. Gli automi a pila possono essere non deterministici. Automi a pila non deterministici e deterministici non sono computazionalmente equivalenti. Gli automi a pila non deterministici riconoscono alcuni linguaggi che nessun automa a pila deterministico può riconoscere. Gli automi a pila non deterministici sono computazionalmente equivalenti alle grammatiche context-free.

2.2.1 Definizione formale di automa a pila

La definizione di automa a pila è simile a quella di un automa finito, tranne che per la pila. La macchina può usare differenti alfabeti per il suo input e la sua pila, quindi ora specifichiamo sia un alfabeto di input Σ sia un alfabeto di pila Γ .

Nel cuore di ogni definizione di automa c'è la funzione di transizione, che descrive il suo comportamento. Ricordiamo che $\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$ e $\Gamma_\epsilon = \Gamma \cup \{\epsilon\}$. Il dominio della funzione di transizione è $Q \times \Sigma_\epsilon \times \Gamma_\epsilon$. Quindi lo stato corrente, il prossimo simbolo di input letto, e il simbolo sulla cima della pila determinano la mossa seguente di un automa a pila. L'uno o l'altro simbolo può essere ϵ , il che determina che la macchina si muove senza leggere un simbolo di input o senza leggere un simbolo dalla pila.

Per quanto riguarda il codominio della funzione di transizione, dobbiamo considerare cosa può fare l'automa quando è in una specifica situazione. Esso può trovarsi in un nuovo stato ed eventualmente scrivere un simbolo sulla cima della pila. La funzione δ può indicare quest'azione restituendo un elemento di Q insieme a un elemento di Γ_ϵ , cioè un elemento di $Q \times \Gamma_\epsilon$. Poichè permettiamo il non determinismo in questo modello, una situazione può avere diverse mosse successive lecite. La funzione di transizione ingloba il non determinismo nel modo usuale, restituendo un insieme di elementi di $Q \times \Gamma_\epsilon$, cioè un elemento di $\mathcal{P}(Q \times \Gamma_\epsilon)$. Quindi la funzione di transizione δ assume la forma: $\delta : Q \times \Sigma_\epsilon \times \Gamma_\epsilon \rightarrow \mathcal{P}(Q \times \Gamma_\epsilon)$

DEFINIZIONE 2.4: AUTOMA A PILA

Un *automa a pila* è una sestupla $(Q, \Sigma, \Gamma, \delta, q_0, F)$ dove, Q, Σ, Γ ed F sono tutti insiemi finiti, e

1. Q è l'insieme degli stati,
2. Σ è l'alfabeto dell'input,
3. Γ è l'alfabeto della pila,
4. $\delta : Q \times \Sigma_\epsilon \times \Gamma_\epsilon \rightarrow \mathcal{P}(Q \times \Gamma_\epsilon)$ è la funzione di transizione,
5. $q_0 \in Q$ è lo stato iniziale,
6. $F \subseteq Q$ è l'insieme degli stati accettanti.

Un automa a pila $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$ computa nel seguente modo. Accetta un input w se w può essere scritto come $w = w_1 w_2 \dots w_m$ dove ciascun $w_i \in \Sigma_\epsilon$ ed esistono sequenze di stati $r_0, r_1, \dots, r_m \in Q$ e di stringhe $s_0, s_1, \dots, s_m \in \Gamma^*$ che soddisfano le tre condizioni seguenti. Le stringhe s_i rappresentano la sequenza del contenuto della pila che M ha su un ramo accettante della computazione.

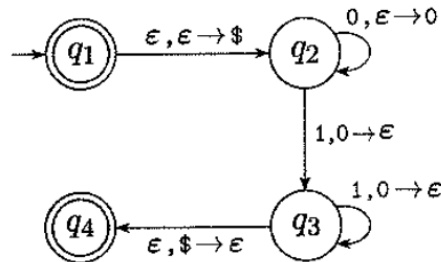
1. $r_0 = q_0$ e $s_0 = \epsilon$. Questa condizione significa che M inizia correttamente, nello stato iniziale e con una pila vuota.
2. Per $i = 0, \dots, m - 1$ abbiamo $(r_{i+1}, b) \in \delta(r_i, w_{i+1}, a)$ dove $s_i = at$ ed $s_{i+1} = bt$ per qualche $a, b \in \Gamma_\epsilon$ e $t \in \Gamma^*$. Questa condizione afferma che M si muove correttamente in base allo stato, al simbolo di pila, e al prossimo simbolo in input.
3. $r_m \in F$. Questa condizione afferma che alla fine dell'input M si trova in uno stato accettante.

Esempio

Quello che segue è la descrizione formale del PDA che riconosce il linguaggio $\{0^n 1^n \mid n \geq 0\}$. Sia M_1 la sestupla $(Q, \Sigma, \Gamma, \delta, q_1, F)$, dove

- $Q = \{q_1, q_2, q_3, q_4\}$
- $\Sigma = \{0, 1\}$
- $\Gamma = \{0, \$\}$
- $F = \{q_1, q_4\}$

L'automata è il seguente.



La definizione di un PDA non contiene alcun meccanismo esplicito per permettere al PDA di controllare se la pila è vuota. Questo PDA è in grado di ottenere lo stesso effetto inserendo inizialmente un simbolo speciale \$ nella pila. Allora, nel caso in cui veda il \$ di nuovo, sa che la pila è di fatto vuota. Analogamente, i PDA non possono controllare esplicitamente se hanno raggiunto la fine della stringa di input. Questo PDA è in grado di ottenere lo stesso risultato poichè lo stato accettante potrebbe essere eventualmente raggiunto solo quando la macchina è alla fine dell'input. Quindi possiamo assumere che i PDA possono controllare la fine dell'input e sappiamo che possiamo implementarlo nello stesso modo.

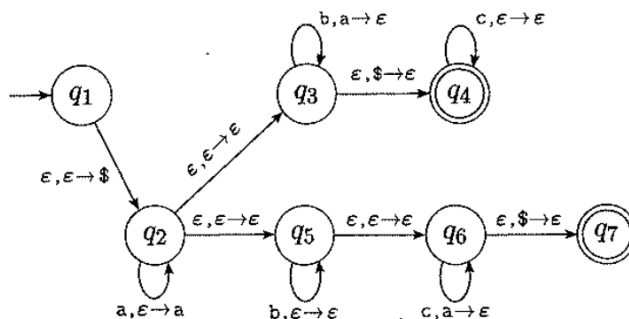
Esempio

Questo esempio illustra un PDA che riconosce il linguaggio:

$$\{a^i b^j c^k \mid i, j, k \geq 0 \text{ e } i = j \text{ oppure } i = k\}$$

Informalmente, il PDA per questo linguaggio opera dapprima leggendo e inserendo le a nella pila. Quando le a sono terminate, la macchina le ha tutte nella pila, quindi può abbinarle con le b o con le c . Quest'operazione è complicata in quanto la macchina non sa se far corrispondere le a con le b o con le c . In questo caso il non determinismo risulta utile.

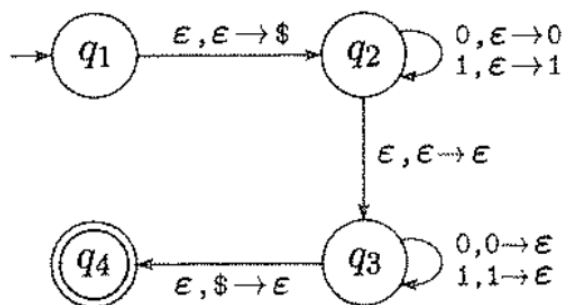
Usando il non determinismo, il PDA può ipotizzare di far corrispondere le a con le b o con le c . Il PDA è il seguente.



Esempio

In questo esempio diamo un PDA M_3 che riconosce il linguaggio $\{ww^R \mid w \in \{0, 1\}^*\}$.

Il PDA inizia inserendo nella pila i simboli letti. In qualunque momento ipotizza non deterministicamente di aver raggiunto la prima metà della stringa e quindi cambia azione, eliminando dalla pila un simbolo per ciascun simbolo letto, se essi coincidono. Se essi coincidono sempre e la pila si suota nello stesso momento in cui l'input è terminato, accetta; altrimenti rifiuta.



2.2.2 Equivalenza con le grammatiche context-free

Le grammatiche context-free e i PDA sono computazionalmente equivalenti e entrambi sono in grado di descrivere la classe dei linguaggi context-free.

Teorema 2.6: Equivalenza delle CFG con i PDA

Un linguaggio è context free se e solo se esiste un automa a pila che lo riconosce.

Per i teoremi “se e solo se”, abbiamo due enunciati da provare, nelle due direzioni.

LEMMA 2.1

Se un linguaggio è context-free, allora esiste un automa a pila che lo riconosce.

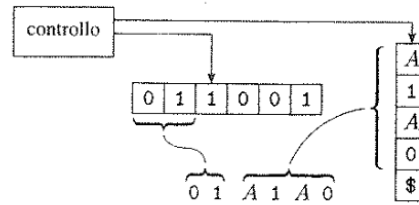
Sia A un CFL. Dalla definizione sappiamo che esiste una CFG, G , che genera A . Mostriamo come trasformare G in un PDA equivalente, che chiamiamo P .

Il PDA P che ora descriviamo, opererà accettando il suo input w , se G genera tale input, determinando se esiste una derivazione per w . Noi progettiamo P in modo che possa stabilire una serie di sostituzioni che usano le regole di G possa condurre dalla variabile iniziale a w .

Una delle difficoltà nell'esaminare se esiste una derivazione per w consiste nel capire quali sostituzioni fare. Il non determinismo di un PDA gli consente di ipotizzare la sequenza di sostituzioni giuste. In ogni passo della derivazione, una delle regole per una particolare variabile è scelta non deterministicamente e usata per sostituire quella variabile. Il PDA P inizia scrivendo la variabile iniziale sulla sua pila. Esso passa attraverso una serie di stringhe intermedie, facendo una sostituzione dietro l'altra. Infine può giungere a una stringa che contiene solo simboli terminali, il che significa che usato la grammatica per derivare una stringa. Allora P accetta se questa stringa è identica alla stringa che ha ricevuto in input.

Implementare questa strategia su un PDA richiede un'idea supplementare. Dobbiamo capire come il PDA immagazzina le stringhe intermedie quando passa dall'una all'altra. Usare semplicemente la pila per immagazzinare ciascuna stringa intermedia è allettante. Tuttavia non funziona del tutto poiché il PDA ha bisogno di trovare le variabili nelle stringhe intermedie e fare sostituzioni. Il PDA può accedere solo al simbolo sulla cima della pila e questo potrebbe essere un simbolo terminale e non una variabile. Il modo per aggirare questo problema è mantenere solo parte della stringa intermedia sulla pila: i simboli che iniziano con la prima variabile nella stringa intermedia. Tutti i simboli terminali che compaiono prima della prima variabile sono subito abbinati con i simboli nella stringa di input.

La figura seguente mostra il PDA P .



Quello che segue è una descrizione informale di P .

1. Inserisce il simbolo marcatore $\$$ e la variabile iniziale sulla pila.
2. Ripete i seguenti passi.
 - (a) Se sulla cima della pila c'è il simbolo di una variabile A , sceglie non deterministicamente una delle regole per A e sostituisce A con la stringa sul lato destro della regola.
 - (b) Se sulla cima della pila c'è un simbolo terminale a , legge il simbolo seguente dall'input e lo confronta con a . Se essi sono uguali, ripete. Se essi non sono uguali, rifiuta su questo ramo del non determinismo.
 - (c) Se sulla cima della pila c'è il simbolo $\$$, entra nello stato accettante. In questo modo accetta l'input se esso è stato completamente letto.

Dimostrazione. Ora diamo i dettagli formali della costruzione del PDA $P = (Q, \Sigma, \Gamma, \delta, q_{start}, F)$. Per rendere più chiara la costruzione, usiamo una notazione abbreviata per la funzione di transizione. Questa notazione fornisce un modo per scrivere un'intera stringa sulla pila in un passo della macchina. Possiamo simulare quest'azione introducendo stati aggiuntivi per scrivere la stringa un simbolo alla volta.

Siano q ed r stati del PDA e siano $a \in \Sigma_\epsilon$ ed $s \in \Gamma_\epsilon$. Supponiamo di volere che il PDA vada da q ad r quando legge a ed elimina s . Inoltre, vogliamo che esso inserisca l'intera stringa $u = u_1 \cdots u_l$ sulla pila simultaneamente. Possiamo eseguire quest'azione introducendo nuovi stati $q_1, \dots, q_{l-1}$ e definendo

la funzione di transizione come segue:

$$\begin{aligned}
&\delta(q, a, s) \text{ contiene } (q_1, u_l), \\
&\delta(q_1, \epsilon, \epsilon) = \{(q_2, u_{l-1})\}, \\
&\delta(q_2, \epsilon, \epsilon) = \{(q_3, u_{l-2})\}, \\
&\quad \vdots \\
&\delta(q_{l-1}, \epsilon, \epsilon) = \{(r, u_1)\}
\end{aligned}$$

Useremo la notazione $(r, u) \in \delta(q, a, s)$ per denotare che quando q è lo stato in cui si trova l'automa, a è il prossimo simbolo di input ed s è il simbolo sulla cima della pila, il PDA può leggere a ed eliminare s , poi inserire la stringa u nella pila e passare nello stato r . La figura seguente ne mostra la realizzazione.

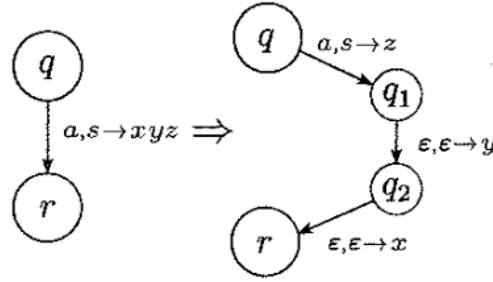


Figura 2.1: Implementazione dell'abbreviazione $(r, xyz) \in \delta(q, a, s)$

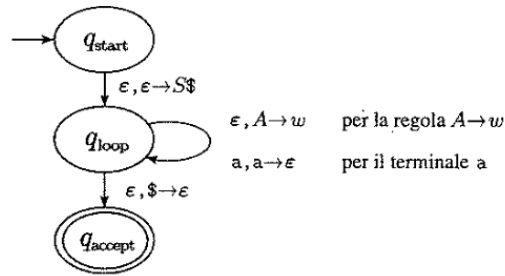
Gli stati di P sono $Q = \{q_{start}, q_{loop}, q_{accept}\} \cup E$, dove E è l'insieme degli stati necessari per realizzare l'abbreviazione appena descritta. Lo stato iniziale è q_{start} . L'unico stato accettante è q_{accept} .

La funzione di transizione è definita come segue. Cominciamo inizializzando la pila inserendo i simboli $\$$ e S , realizzando così il passo 1 nella descrizione informale: $\delta(q_{start}, \epsilon, \epsilon) = \{(q_{loop}, S\$)\}$. Poi aggiungiamo le transizioni per il ciclo principale del passo 2.

In primo luogo, trattiamo il caso (a) in cui la cima della pila contiene una variabile. Poniamo $\delta(q_{loop}, \epsilon, A) = \{(q_{loop}, w) \mid \text{dove } A \rightarrow w \text{ è una regola in } R\}$.

In secondo luogo, trattiamo il caso (b) in cui la cima della pila contiene un terminale. Poniamo $\delta(q_{loop}, a, a) = \{(q_{loop}, \epsilon)\}$.

Infine, trattiamo il caso (c) in cui il marcatore scelto per indicare la pila vuota \$ è sulla cima della pila. Poniamo $\delta(q_{loop}, \epsilon, \$) = \{(q_{accept}, \epsilon)\}$. Il diagramma di stato è mostrato nella seguente figura.



□

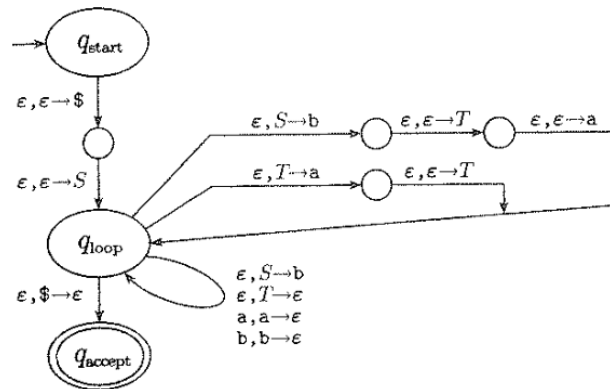
Esempio

Costruiamo un PDA P_1 dalla seguente CFG G .

$$S \rightarrow aTb \mid b$$

$$T \rightarrow Ta \mid \epsilon$$

La funzione di transizione è mostrata nel diagramma seguente.



Ora dimostriamo la direzione inversa del teorema **equivalenza delle CFG con i PDA**. Per la direzione diretta, abbiamo dato una procedura per trasformare una

CFG in PDA. Ora vogliamo dare una procedura per andare nella direzione opposta: trasformare un PDA in una CFG. Progettiamo la grammatica per simulare l'automa. Questo compito è impegnativo perchè "programmare" un automa è più facile che "programmare" una grammatica.

LEMMA 2.2

Se un linguaggio è riconosciuto da un automa a pila, allora esso è context-free.

Abbiamo un PDA P e vogliamo costruire una CFG G che genera tutte le stringhe che P accetta. In altre parole, G dovrebbe generare una stringa se quella stringa fa andare il PDA dal suo stato iniziale a uno stato accettante.

Per raggiungere questo risultato, progettiamo una grammatica che fa un pò in più. Per ciascuna coppia di stati p e q in P , la grammatica avrà una variabile A_{pq} . Questa variabile genera tutte le stringhe che possano portare P da p con pila vuota a q con pila vuota. Si osservi che tale stringhe possono anche condurre P da p a q , indipendentemente dal contenuto della pila in p , lasciando la pila in q nella stessa condizione in cui era in p . Innanzitutto, semplifichiamo il nostro compito modificando leggermente P per munirlo delle seguenti tre caratteristiche.

1. Ha un unico stato accettante, q_{accept} .
2. Svuota la sua pila prima di accettare.
3. Ciascuna transizione inserisce un simbolo sulla pila o ne elimina uno dalla pila, ma non fa entrambe le azioni contemporaneamente.

Dare a P le caratteristiche 1 e 2 è facile. Per munirlo della caratteristica 3, sostituiamo ciascun transizione che contemporaneamente elimina ed inserisce simboli con una sequenza di due transizioni che attraversa un nuovo stato, e sostituiamo ogni transizione che non elimina nè inserisce simboli con una sequenza di due transizioni che inserisce e poi elimina un simbolo arbitrario della pila.

Per progettare G in modo che A_{pq} generi tutte le stringhe che portano P da p a q , iniziando e terminando con una pila vuota, dobbiamo capire come P agisce su queste stringhe. Per ognuna di tali stringhe x , la prima mossa di P su x deve essere un push, poichè ogni mossa è un push o un pop e P non può eliminare da una pila vuota. Analogamente, l'ultima mossa su x deve essere un pop perchè alla fine la pila deve essere vuota.

Durante la computazione di P su x si presentano due eventualità. O il simbolo eliminato alla fine è il simbolo che era stato inserito all'inizio, oppure no. Nel

primo caso, la pila potrebbe essere vuota solo all'inizio e alla fine della computazione di P su x . Altrimenti, il simbolo inizialmente inserito deve essere stato eliminato in qualche punto prima della fine di x e quindi la pila si svuota in questo punto. Simuliamo la prima possibilità con la regola $A_{pq} \rightarrow aA_{rs}b$, dove a è l'input letto nella prima mossa, b è l'input letto nell'ultima mossa, r è lo stato che segue p ed s è lo stato che precede q . Simuliamo la seconda possibilità con la regola $A_{pq} \rightarrow A_{pr}A_{rq}$ dove r è lo stato in cui la pila diventa vuota.

Dimostrazione. Sia $P = (Q, \Sigma, \Gamma, \delta, q_0, \{q_{accept}\})$ e costruiamo G . L'insieme delle variabili di G è $\{A_{pq} \mid p, q \in Q\}$. La variabile iniziale è $A_{q_0, q_{accept}}$. Ora descriviamo le regole di G nei seguenti tre punti.

1. $\forall p, q, r, s \in Q, u \in \Gamma$ e $a, b \in \Sigma_\epsilon$, se $\delta(p, a, \epsilon)$ contiene (r, u) e $\delta(s, b, u)$ contiene (q, ϵ) , poni la regola $A_{pq} \rightarrow aA_{rs}b$ in G
2. $\forall p, q, r \in Q$ poni la regola $A_{pq} \rightarrow A_{pr}A_{rq}$ in G
3. Infine, $\forall p \in Q$ poni la regola $A_{pp} \rightarrow \epsilon$ in G .

Ora proviamo che questa costruzione funziona dimostrando che A_{pq} genera x se e solo se x porta P da p con la pila vuota a q con la pila vuota. Consideriamo ogni direzione di "se e solo se" come un enunciato separato.

PROPOSIZIONE 2.3

Se A_{pq} genera x , allora x può portare P da p con pila vuota a q con la pila vuota.

Dimostriamo questo enunciato per induzione sul numero dei passi nella derivazione di x da A_{pq} . **Base:** La derivazione è in un solo passo.

Una derivazione in un solo passo deve usare una regola il cui lato destro non contenga variabili. Le uniche regole in G con nessuna variabile sul lato destro sono $A_{pp} \rightarrow \epsilon$. Ovviamente, l'input ϵ porta P da p con la pila vuota a p con la pila vuota e quindi la base è dimostrata.

Passo induttivo: Assumiamo che l'enunciato sia vero per le derivazioni di lunghezza al più k , dove $k \geq 1$, e proviamo che esso è vero per le derivazioni di lunghezza $k + 1$.

Supponiamo che $A_{pq} \xRightarrow{*} x$ in $k + 1$ passi. Il primo passo in questa derivazione è $A_{pq} \Rightarrow aA_{rs}b$ oppure $A_{pq} \Rightarrow A_{pr}A_{rq}$. Trattiamo questi due casi separatamente.

Nel primo caso, consideriamo la parte y di x che A_{rs} genera, quindi $x = ayb$. Poichè $A_{rs} \xRightarrow{*} y$ in k passi, l'ipotesi induttiva ci dice che P può andare da r con la pila vuota a s con la pila vuota. Poichè $A_{pq} \rightarrow aA_{rs}b$ è una regola di G , $\delta(p, a, \epsilon)$ contiene (r, u) e $\delta(s, b, u)$ contiene (q, ϵ) , per qualche simbolo di pila u . Quindi, se P inizia in p con la pila vuota, dopo aver letto a può andare nello stato r e inserire u in cima alla pila. Poi, la lettura della stringa y può portarlo in s e lasciare u sulla pila. In seguito dopo aver letto b può andare nello stato q ed eliminare u dalla pila. Pertanto, x può portare P da p con la pila vuota a q con la pila vuota.

Nel secondo caso, consideriamo le parti y e z di x che A_{pr} e A_{rq} rispettivamente generano, quindi $x = yz$. Poichè $A_{pr} \xRightarrow{*} y$ in al più k passi e $A_{rq} \xRightarrow{*} z$ in al più k passi, l'ipotesi induttiva ci dice che y può portare P da p a r e z può portare P da r a q , con la pila vuota all'inizio e alla fine della computazione. Quindi x può portare P da p con la pila vuota a q con la pila vuota. Questo completa la prova del passo induttivo.

PROPOSIZIONE 2.4

Se x può portare P da p con la pila vuota a q con la pila vuota, A_{pq} genera x .

Dimostriamo questo enunciato per induzione sul numero dei passi nella computazione di P da p a q con pile vuote sull'input x .

Base: La computazione è in 0 passi.

Se una computazione è in 0 passi, essa inizia e termina nello stesso stato p . Quindi dobbiamo mostrare che $A_{pp} \xRightarrow{*} x$. P non può leggere alcun carattere in 0 passi, quindi $x = \epsilon$. Per costruzione, G ha la regola $A_{pp} \rightarrow \epsilon$, perciò la base è dimostrata.

Passo induttivo: Assumiamo l'enunciato vero per le computazioni di lunghezza al più k , dove $k \geq 0$, e dimostriamo che è vero per le computazioni di lunghezza $k + 1$.

Supponiamo che P abbia una computazione dove x porta da p a q con pile vuote in $k + 1$ passi. O la pila è vuota solo all'inizio e alla fine di questa computazione oppure essa si svuota anche altrove.

Nel primo caso, il simbolo che è stato inserito nella prima mossa deve essere lo stesso simbolo che è stato rimosso dall'ultima mossa. Chiamiamo u questo simbolo. Sia a il simbolo di input letto nella prima mossa, b il simbolo di input letto nell'ultima mossa, r lo stato dopo la prima mossa ed s lo stato prima

dell'ultima mossa. Allora $\delta(p, a, \epsilon)$ contiene (r, u) e $\delta(s, b, u)$ contiene (q, ϵ) e quindi la regola $A_{pq} \rightarrow aA_{rs}b \in G$.

Sia y la parte di x senza a e b , quindi $x = ayb$. L'input y può portare P da r a s senza toccare il simbolo u che è sulla cima della pila e quindi P può andare da r con pila vuota a s con pila vuota sull'input y . Abbiamo eliminato il primo e l'ultimo passo dei $k + 1$ passi della computazione iniziale su x , perciò la computazione su y ha $k - 1$ passi. Quindi l'ipotesi induttiva ci dice che $A_{rs} \xRightarrow{*} y$. Allora $A_{pq} \xRightarrow{*} x$.

Nel secondo caso, sia r uno stato in cui la pila si svuota oltre che all'inizio o alla fine della computazione su x . Allora le parti della computazione da p ad r e da r a q contengono al più k passi. Sia y l'input letto nella prima parte e sia z l'input letto nella seconda parte. L'ipotesi induttiva ci dice che $A_{pr} \xRightarrow{*} y$ e $A_{rq} \xRightarrow{*} z$. Poichè la regola $A_{pq} \rightarrow A_{pr}A_{rq}$ è in G , $A_{pq} \xRightarrow{*} x$ e la dimostrazione è completata. $\square$

2.3 Linguaggi non context-free

In questa sezione presentiamo una tecnica per dimostrare che alcuni linguaggi non sono context-free. Precedentemente, per dimostrare che un linguaggio non era regolare, abbiamo utilizzato il pumping lemma. A questo punto presentiamo un pumping lemma simile per i linguaggi context-free. Esso stabilisce che per ogni linguaggio context-free esiste un particolare valore chiamato la *lunghezza del pumping* tale che tutte le stringhe più lunghe nel linguaggio possono essere iterate. Questa volta il significato di iterazione è più complesso.

2.3.1 Il pumping lemma per i linguaggi context-free

Teorema 2.7: Il pumping lemma per i linguaggi context-free

Se A è un linguaggio context-free, allora esiste un numero p (lunghezza del pumping) tale che, se s è qualsiasi stringa in A di lunghezza almeno p , allora s può essere divisa in cinque parti $s = uvxyz$ che soddisfano le condizioni.

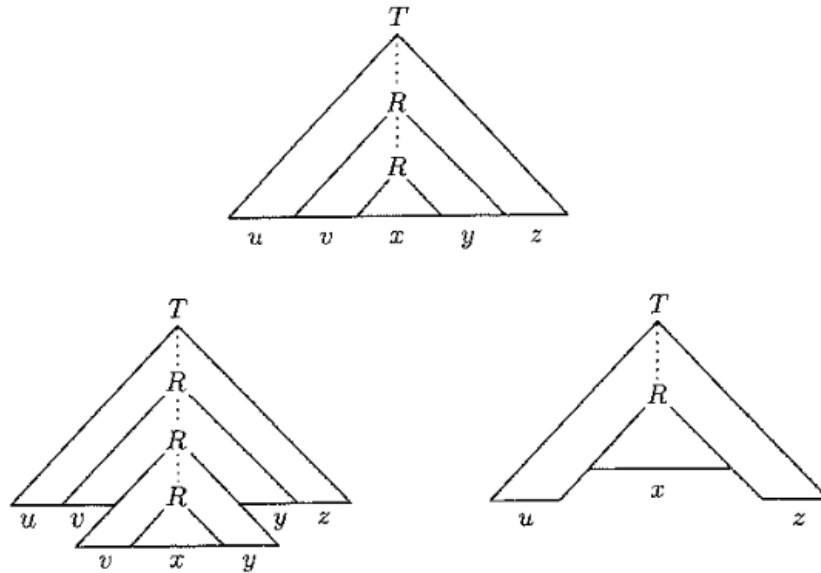
1. $\forall i \geq 0, uv^i xy^i z \in A$
2. $|vy| > 0$
3. $|vxy| \leq p$

Quando s è divisa in $uvxyz$, la condizione 2 afferma che v oppure y non è la stringa vuota. Altrimenti il teorema sarebbe banalmente vero. La condizione

3 afferma che le parti v , x e y insieme hanno lunghezza al più p . Questa condizione tecnica a volte è utile per dimostrare che alcuni linguaggi non sono context-free.

Sia A un CFL e sia G una CFG che lo genera. Dobbiamo dimostrare che ogni stringa sufficientemente lunga s in A può essere iterata e restare in A . L'idea dietro questa strategia è semplice.

Sia s una stringa molto lunga in A . Poichè s è in A , essa è derivabile da G e quindi ha un albero sintattico. L'albero sintattico per s deve essere molto alto perchè s è molto lunga. Cioè l'albero sintattico deve contenere un cammino lungo dalla variabile alla radice dell'albero a uno dei simboli terminali su una foglia. Per il principio della piccioni, qualche simbolo di variabile R si deve ripetere in questo cammino lungo. Come mostra la figura seguente, questa ripetizione ci permette di sostituire il sottoalbero sotto la seconda occorrenza di R con il sottoalbero sotto la prima occorrenza di R e ottenere ancora un albero sintattico consentito. Pertanto, possiamo dividere s in cinque parti $uvxyz$ e possiamo replicare il secondo e il quarto pezzo e ottenere una stringa ancora nel linguaggio. In altre parole, $uv^i xy^i z$ è in A per ogni $i \geq 0$.



Dimostrazione. Sia G una CFG per il CFL A . Sia b il massimo numero di simboli nel lato destro di una regola (assumiamo che sia almeno 2). Sappiamo

che, in ogni albero sintattico costruito usando questa grammatica, un nodo non può avere più di b figli. In altre parole, ci sono al più b foglie in un passo dalla variabile iniziale; ci sono al più b^2 foglie in 2 passi dalla variabile iniziale; e ci sono al più b^h foglie in h passi dalla variabile iniziale. Quindi, se l'altezza dell'albero sintattico è al più h , la lunghezza della stringa generata è al più b^h . Viceversa, se una stringa generata ha lunghezza maggiore o uguale a $b^h + 1$, ciascuno dei suoi alberi sintattici deve avere un'altezza maggiore o uguale a $h + 1$.

Sia $|V|$ il numero delle variabili in G . Poniamo p , la lunghezza del pumping, uguale a $b^{|V|+1}$. Ora se s è una stringa in A e la sua lunghezza è maggiore o uguale a p , il suo albero sintattico deve avere altezza maggiore o uguale a $|V| + 1$, poichè $b^{|V|+1} \geq b^{|V|} + 1$.

Per vedere come iterare tale stringa s , sia τ uno dei suoi alberi sintattici. Se s ha diversi alberi sintattici, scegliamo un albero sintattico τ che abbia il più piccolo numero di nodi. Sappiamo che τ deve avere altezza maggiore o uguale a $|V| + 1$, quindi il suo cammino più lungo dalla radice a una foglia ha lunghezza almeno $|V| + 1$. Questo cammino ha almeno $|V| + 2$ nodi; uno etichettato da un terminale, gli altri etichettati da variabili. Quindi questo cammino ha almeno $|V| + 1$ variabili. Poichè G ha solo $|V|$ variabili, qualche variabile R è presente più di una volta su questo cammino. Per un utilizzo successivo, scegliamo R in modo che una variabile che si ripete tra le $|V| + 1$ variabili più in basso su questo cammino.

Dividiamo s in $uvxyz$. Ogni occorrenza di R ha un sottoalbero sotto essa che genera una parte della stringa s . L'occorrenza più in alto di R ha un sottoalbero più grande e genera vxy , mentre l'occorrenza più in basso genera solo x come sottoalbero più piccolo. Entrambi questi sottoalberi sono generati dalla stessa variabile, quindi possiamo sostituire l'uno con l'altro e ottenere ancora un albero sintattico corretto. Sostituire ripetutamente il più piccolo con il più grande fornisce gli alberi sintattici per le stringhe $uv^i xy^i z$ per ogni $i > 1$. Sostituire il più grande con il più piccolo genera la stringa uxz . Questo dimostra la condizione 1 del pumping lemma. Dimostriamo le condizioni 2 e 3.

Per ottenere la condizione 2, dobbiamo essere sicuri che v e y non sono entrambe ϵ . Se lo fossero, l'albero sintattico ottenuto sostituendo il più piccolo sottoalbero al più grande avrebbe meno nodi di τ e genererebbe ancora s . Questo non è possibile perchè abbiamo scelto τ in modo che sia un albero sintattico per s con il più piccolo numero di nodi. Questa è la ragione per aver selezionato così τ .

Per ottenere la condizione 3, dobbiamo essere sicuri che vxy ha lunghezza al più p . Nell'albero sintattico per s l'occorrenza più in alto di R genera vxy . Abbiamo scelto R in modo che entrambe le occorrenze di essa cadano nelle $|V| + 1$ variabili più in basso del cammino e abbiamo scelto il più lungo cammino nell'albero sintattico, in modo che il sottoalbero in cui R genera vxy sia alto al più $|V| + 1$. Un albero con questa altezza può generare una stringa di lunghezza al più $b^{|V|+1} = p$ $\square$

Esempio

Usiamo il pumping lemma per mostrare che il linguaggio $B = \{a^n b^n c^n \mid n \geq 0\}$ non è context-free.

Assumiamo che B sia un CFL e giungiamo a una contraddizione. Sia p la lunghezza del pumping per B la cui esistenza è garantita dal pumping lemma. Scegliamo la stringa $s = a^p b^p c^p$. Ovviamente s è un elemento di B e di lunghezza almeno p . Il pumping lemma afferma che s può essere iterata, ma noi mostriamo che non può esserlo. In altre parole, mostriamo che, non importa come dividiamo s in $uvxyz$, una delle tre condizioni del pumping lemma è violata.

In primo luogo, la condizione 2 stabilisce che v o y non è vuota. Allora consideriamo due casi, a seconda che le sottostringhe v e y contengano più di un tipo di simbolo dell'alfabeto o no.

1. Quando entrambe v e y contengono solo un tipo di simboli dell'alfabeto, v non contiene entrambi i simboli a e b o entrambi i simboli b e c e lo stesso vale per y . In questo caso, la stringa uv^2xy^2z non può contenere lo stesso numero di a , b e c . Quindi, essa non può essere un elemento di B . Abbiamo raggiunto la contraddizione.
2. Quando v o y contengono più di un tipo di simbolo, uv^2xy^2z può contenere un equal numero dei tre simboli dell'alfabeto ma non nell'ordine corretto. Perciò essa non può essere un elemento di B e si verifica un assurdo.

Uno di questi casi deve verificarsi. Poichè entrambi i casi conducono a un assurdo, la contraddizione è inevitabile. Quindi l'assunzione che B sia un CFL deve essere falsa. Pertanto abbiamo provato che B non è CFL.

2.4 Linguaggi context-free deterministici

Come abbiamo detto precedentemente, gli automi finiti deterministici e gli automi finiti non deterministici sono equivalenti dal punto di vista del potere riconoscitivo di linguaggi. Invece gli automi a pila non deterministici sono più potenti della loro controparte deterministica. Ci sono linguaggi context-free che non possono essere riconosciuti da automi a pila deterministici ma da automi a pila non deterministici.

I linguaggi che sono riconosciuti da automi a pila deterministici (DPDA) sono chiamati linguaggi context-free deterministici (DCFL). Questa sottoclasse dei linguaggi context-free è importante per le applicazioni, come il progetto di parser nei compilatori per i linguaggi di programmazione. Nel definire un DPDA, ci atteniamo al principio base del determinismo: in ogni passo della sua computazione, il DPDA ha al più un modo di procedere in base alla sua funzione di transizione. Definire i DPDA è più complicato che definire i DFA perché i DPDA possono leggere un simbolo di input senza eliminare un simbolo dalla pila e viceversa. Di conseguenza, permettiamo ϵ -mosse nella funzione di transizione di un DPDA. Queste ϵ -mosse assumono due forme: *mosse ϵ -input* che corrispondono a $\delta(q, \epsilon, x)$ e *mosse ϵ -pila* che corrispondono a $\delta(q, a, \epsilon)$. Una mossa può assumere entrambe le forme con $\delta(q, \epsilon, \epsilon)$. Se un DPDA può fare una ϵ -mossa in una certa situazione, gli è vietato fare una mossa in quella stessa situazione che comporta elaborare un simbolo invece di ϵ .

La definizione formale è la seguente.

DEFINIZIONE 2.5: AUTOMA A PILA DETERMINISTICO

Un *automa a pila deterministico* è una sestupla $(Q, \Sigma, \Gamma, \delta, q_0, F)$ dove Q, Σ, Γ ed F sono tutti insiemi finiti e

1. Q è l'insieme degli stati,
2. Σ è l'alfabeto dell'input,
3. Γ è l'alfabeto della pila,
4. $\delta : Q \times \Sigma_\epsilon \times \Gamma_\epsilon \rightarrow (Q \times \Gamma_\epsilon) \cup \{\emptyset\}$ è la funzione di transizione,
5. $q_0 \in Q$ è lo stato iniziale ed
6. $F \subseteq Q$ è l'insieme degli stati accettanti.

La funzione di transizione δ deve soddisfare la seguente condizione. $\forall q \in Q, a \in \Sigma$ e $x \in \Gamma$, esattamente uno dei valori

$$\delta(q, a, x), \delta(q, a, \epsilon), \delta(q, \epsilon, x) \text{ e } \delta(q, \epsilon, \epsilon)$$

è diverso dal $\emptyset$.

La funzione di transizione può dare in uscita una singola mossa della forma (r, y) o può indicare nessuna azione dando in uscita $\emptyset$.

Un DPDA ha esattamente una mossa lecita in ogni situazione in cui la sua pila non è vuota. Se la pila è vuota, un DPDA può fare mosse solo se la funzione di transizione specifica una mossa che elimina ϵ . Altrimenti, il DPDA non ha mosse lecite ed esso rifiuta senza leggere il resto dell'input.

Il linguaggio di un DPDA è chiamato un *linguaggio context-free deterministico*.

LEMMA 2.5

Ogni DPDA ha un DPDA equivalente che legge sempre l'intera stringa di input.

Un DPDA può non riuscire a leggere tutta la stringa di input se prova a eliminare simboli da una pila vuota o se effettua una sequenza infinita di mosse ϵ -input. Chiamiamo la prima situazione *hanging* e la seconda situazione *looping*. Risolviamo il problema dell'hanging inizializzando la pila con un simbolo speciale. Se questo simbolo è in seguito eliminato dalla pila prima della fine dell'input, il DPDA legge fino alla fine dell'input e rifiuta.

Risolviamo il problema del looping identificando le situazioni di looping, cioè quelle da cui nessun ulteriore simbolo è più letto, e riprogrammando il DPDA in modo che esso legga e rifiuti l'input invece di essere in looping. Dobbiamo sistemare queste modifiche per risolvere il caso in cui l'hanging o il looping si presentino sull'ultimo simbolo dell'input. Se il DPDA entra in uno stato accettante in un qualsiasi momento dopo che ha letto l'ultimo simbolo, il DPDA modificato accetta invece di rifiutare.

Dimostrazione. Sia $P = (Q, \Sigma, \Gamma, \delta, q_0, F)$ un DPDA. Innanzitutto aggiungiamo un nuovo stato iniziale q_{start} , un ulteriore stato accettante q_{accept} , un nuovo stato q_{reject} , e anche altri nuovi stati come descritto nel seguito.

Eseguiamo le seguenti variazioni per ogni $r \in Q, a \in \Sigma_\epsilon$ e $x, y \in \Gamma_\epsilon$.

In primo luogo modifichiamo P in modo che, non appena entra in uno stato accettante, resta in stati accettanti fino a quando non legge il simbolo di input

successivo. Aggiungiamo un nuovo stato accettante q_a per ogni $q \in Q$. Per ogni $q \in Q$, se $\delta(q, \epsilon, x) = (r, y)$, poniamo $\delta(q_a, \epsilon, x) = (r_a, y)$ e poi se $q \in F$, modifichiamo anche δ ponendo $\delta(q, \epsilon, x) = (r_a, y)$. Per ogni $q \in Q$ e $a \in \Gamma$, se $\delta(q, a, x) = (r, y)$ poniamo $\delta(q_a, a, x) = (r, y)$. Sia F' l'insieme dei nuovi e vecchi stati accettanti.

Poi, modifichiamo P in modo che rifiuti quando prova a eliminare simboli da una pila vuota, inizializzando la pila con un nuovo simbolo speciale $\$$. Se successivamente P rileva $\$$ in uno stato non accettante, entra in q_{reject} e scandisce l'input fino alla fine. Se P rileva $\$$ in uno stato accettante, entra in q_{accept} . In seguito, se resta input non letto, entra in q_{reject} e scandisce l'input fino alla fine. Formalmente, poniamo $\delta(q_{start}, \epsilon, \epsilon) = (q_0, \$)$. Per $x \in \Gamma$ e $\delta(q, a, x) \neq \emptyset$, se $q \notin F'$ allora poniamo $\delta(q, a, \$) = (q_{reject}, \epsilon)$, e se $q \in F'$ allora poniamo $\delta(q, a, \$) = (q_{accept}, \epsilon)$. Per $a \in \Sigma$, poniamo $\delta(q_{reject}, a, \epsilon) = (q_{reject}, \epsilon)$ e $\delta(q_{accept}, a, \epsilon) = (q_{reject}, \epsilon)$.

Infine, modifichiamo P in modo che rifiuti invece di fare una sequenza infinita di mosse ϵ -input prima della fine dell'input. Per ogni $q \in Q$ e $x \in \Gamma$, chiamiamo (q, x) una *situazione di looping* se, quando P è nello stato q con $x \in \Gamma$ sulla cima della pila, non elimina mai nulla sotto x e non legge mai un simbolo di input. Diciamo che la situazione è di looping è *accettante* se P entra in uno stato accettante durante le sue mosse successive, altrimenti, essa è di *rifiuto*. Se (q, x) è una situazione di looping accettante, poniamo $\delta(q, \epsilon, x) = (q_{accept}, \epsilon)$, mentre se (q, x) è una situazione di looping di rifiuto, poniamo $\delta(q, \epsilon, x) = (q_{reject}, \epsilon)$. $\square$

2.4.1 Proprietà dei DCFL

Esamineremo proprietà di chiusura e di non chiusura della classe dei DCFL e le useremo per esibire un CFL che non è un DCFL.

Teorema 2.8

La classe dei DCFL è chiusa rispetto al complemento.

Scambiando gli stati accettanti e non accettanti di un DFA si ottiene un nuovo DFA che riconosce il complemento del linguaggio. La stessa strategia funziona per i DPDA tranne che per un problema. Il DPDA può accettare il suo input entrando sia in stati accettanti che in stati non accettanti in una sequenza di mosse alla fine della stringa di input. Scambiando stati accettanti e non accettanti l'automa modificato accetterebbe ancora in questo caso. Risolviamo questo problema modificando il DPDA in modo da limitare quando si può presentare l'accettazione.

Per ogni simbolo di input, il DPDA modificato può entrare in uno stato accettante solo quando è sul punto di leggere il simbolo seguente. In altre parole, solo gli stati “di lettura”, stati in cui viene sempre letto un simbolo di input, possono essere stati accettanti.

Dimostrazione. Modifichiamo P come descritto nella prova del [lemma 2.5](#) e sia $(Q, \Sigma, \Gamma, \delta, q_0, F)$ la macchina risultante. Questa macchina legge sempre l'intera stringa di input. Inoltre, una volta entrata in uno stato accettante, resta in stati accettanti fino a quando legge il successivo simbolo di input. Abbiamo bisogno di identificare gli stati di lettura. Se il DPDA nello stato q legge un simbolo di input $a \in \Sigma$ senza eliminare simboli dalla pila, cioè $\delta(q, a, \epsilon) \neq \emptyset$, allora q sarà uno stato di lettura. Invece, se esso legge e rimuove anche un simbolo dalla pila, la decisione di lettura può dipendere solo dal simbolo eliminato, quindi dividiamo questo passo in due: una rimozione e poi una lettura. Quindi se $\delta(q, a, x) = (r, y)$ con $a \in \Sigma$ e $x \in \Gamma$, aggiungiamo un nuovo stato q_x e modifichiamo δ ponendo $\delta(q, \epsilon, x) = (q_x, \epsilon)$ e $\delta(q_x, a, \epsilon) = (r, y)$. Definiamo q_x come uno stato di lettura. Gli stati q_x non eliminano mai simboli dalla pila, quindi la loro azione è indipendente dal contenuto della pila. Definiamo q_x come uno stato accettante se $q \in F$. Infine, togliamo il ruolo di stato accettante a ogni stato che non è uno stato di lettura. Il DPDA modificato è equivalente a P , ma esso entra al più una volta in uno stato accettante per ogni simbolo di input, quando è in procinto di leggere il simbolo seguente. Ora, scambiamo accettazione e non accettazione tra questi stati di lettura. Il DPDA risultante riconosce il linguaggio complemento. $\square$

Questo teorema implica che alcuni CFL non sono DCFL. Ogni CFL il cui complemento non è un CFL non è un DCFL. Quindi $A = \{a^i b^j c^k \mid i \neq j \text{ o } j \neq k \text{ con } i, j, k \geq 0\}$ è un CFL ma non un DCFL. Altrimenti $\overline{A}$ sarebbe un CFL.

Parte II

Teoria della computabilità

Capitolo 3

La tesi di Church-Turing

3.1 Macchine di Turing

La *macchina di Turing* è simile ad un automa finito, ma con una memoria illimitata e senza restrizioni, una macchina di Turing è un modello molto più preciso di un computer. Una macchina di Turing può fare tutto ciò che può fare un computer reale.

Una macchina di Turing utilizza un nastro infinito come propria memoria illimitata. Ha una testina che è in grado di leggere e scrivere simboli ed è libera di muoversi lungo il nastro. Inizialmente il nastro contiene solo la stringa di input e tutto il resto è vuoto. Se la macchina deve memorizzare un'informazione la può scrivere sul nastro. Per leggere l'informazione che ha scritto, la macchina può spostare la testina su di essa.

Il seguente elenco riassume le differenze tra automi finiti e macchine di Turing.

1. Una macchina di Turing può sia scrivere che leggere sul nastro.
2. La testina di lettura-scrittura può muoversi sia verso sinistra che verso destra.
3. Il nastro è infinito.
4. Gli stati speciali di accettazione e rifiuto hanno effetto immediato.

3.1.1 Definizione formale di macchina di Turing

Il cuore della funzione di una macchina di Turing è la funzione di transizione δ perchè essa ci dice come la macchina effettua un passo. Per una macchina di

Turing, δ prende la forma $Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$. Cioè, quando la macchina occupa un certo stato q e la testa punta alla casella del nastro contenente un simbolo a , e se $\delta(q, a) = (r, b, L)$, la macchina di Turing scrive il simbolo b al posto di a , e passa nello stato r . La terza componente è L oppure R ed indica se la testa si sposta verso sinistra o verso destra dopo la scrittura.

DEFINIZIONE 3.1: MACCHINA DI TURING

Una *macchina di Turing* è una 7-upla, $(Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$, dove Q, Σ, Γ sono tutti insiemi finiti e

1. Q è l'insieme degli stati,
2. Σ è l'alfabeto di input non contenente il simbolo **blank** $\sqcup$,
3. Γ è l'alfabeto del nastro, con $\sqcup \in \Gamma$ e $\Sigma \subseteq \Gamma$,
4. $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ è la funzione di transizione,
5. $q_0 \in Q$ è lo stato iniziale,
6. $q_{accept} \in Q$ è lo stato di accettazione e
7. q_{reject} è lo stato di rifiuto, con $q_{accept} \neq q_{reject}$.

Una macchina di Turing computa nel seguente modo. Inizialmente M riceve il suo input $w = w_1 w_2 \dots w_n \in \Sigma^*$ sulle n celle più a sinistra del nastro, mentre il resto del nastro è vuoto, contiene solo simboli blank. La testina parte dalla posizione più a sinistra del nastro. Quando M inizia la computazione, questa procede secondo le regole descritte dalla funzione di transizione. Se M tenta di spostare la testina a sinistra quando si trova all'estremità sinistra del nastro, allora la testina rimane nello stesso posto per quella mossa, anche se la funzione di transizione indica L . La computazione prosegue fino a quando la macchina raggiunge lo stato di accettazione oppure di rifiuto, a tal punto si ferma. Se nessuno dei due stati viene raggiunto, M va avanti per sempre. Durante la computazione di una macchina di Turing, si verificano cambiamenti dello stato corrente, del contenuto corrente del nastro, e della posizione corrente della testina. Un'impostazione di questi tre elementi è chiamata una **configurazione** della macchina di Turing. Vediamo come sono rappresentate le configurazioni. Per uno stato q e due stringhe u e v sull'alfabeto Γ del nastro, scriviamo uqv per indicare la configurazione, dove lo stato corrente è q , il contenuto del nastro è uv e la posizione attuale della testa è il primo simbolo di v . Dopo l'ultimo simbolo di v , il nastro contiene

solo simboli $\sqcup$. Per esempio, $1011q_701111$ rappresenta la configurazione in cui il nastro contiene 101101111 , lo stato corrente è q_7 e la testina è posizionata sul secondo 0.

Si dice che la configurazione C_1 *produce* C_2 se la macchina di Turing può passare da C_1 a C_2 in un unico passo. Supponiamo di avere a, b e c in Γ , così come u e v in Γ^* e gli stati q_i e q_j . In tal caso $uaq_i bv$ e $uq_j acv$ sono due configurazioni. Diciamo che

$$uaq_i bv \text{ produce } uq_j acv$$

se nella funzione di transizione $\delta(q_i, b) = (q_j, c, L)$. Per uno spostamento a destra, diciamo che

$$uaq_i bv \text{ produce } uacq_j v$$

se $\delta(q_i, b) = \delta(q_j, c, R)$.

La *configurazione iniziale* di M su input w è la configurazione $q_0 w$ che indica che la macchina è nello stato iniziale q_0 con la testina nella posizione più a sinistra sul nastro. In una *configurazione di accettazione* lo stato della configurazione è q_{accept} . In una *configurazione di rifiuto* lo stato della configurazione è q_{reject} . Le configurazioni di accettazione e rifiuto sono *configurazioni di arresto* e non producono ulteriori configurazioni.

L'insieme di stringhe che M accetta rappresenta il *linguaggio di M* , o il *linguaggio riconosciuto da M* , denotato con $L(M)$.

DEFINIZIONE 3.2: LINGUAGGIO TURING-RICONOSCIBILE

Un linguaggio si dice *Turing-riconoscibile* se esiste una macchina di Turing che lo riconosce.

Quando attiviamo una macchina di Turing su un input, tre risultati sono possibili. La macchina può *accettare*, *rifiutare* oppure andare in *loop*. Per *loop* si intende semplicemente che la macchina non si ferma. Un ciclo, comporta un comportamento semplice o complesso che non porta ad uno stato di arresto. Una macchina di Turing M può non accettare un input sia raggiungendo lo stato q_{reject} e rifiutando oppure andando in loop. A volte è difficile distinguere una macchina in loop o una macchina che ci sta impiegando molto tempo. Per questo motivo si preferiscono macchine di Turing che si fermano su ogni singolo input; macchine che non cicloranno mai. Una tale macchina è detta *decisore* perchè prende in ogni caso una decisione.

DEFINIZIONE 3.3: LINGUAGGIO TURING-DECIDIBILE

Un linguaggio si dice *Turing-decidibile* o semplicemente *decidibile* se esiste una macchina di Turing che lo decide.

3.1.2 Esempi di macchine di Turing

Esempio

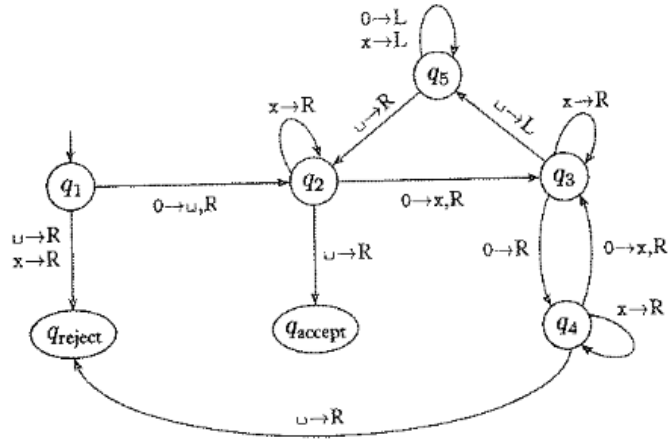
In questo esempio descriviamo una macchina di Turing M_2 che decide $A = \{0^{2^n} \mid n \geq 0\}$.

M_2 = “su input w :

1. Muove la testina da sinistra a destra, cancellando ogni secondo 0.
2. Se al passo 1, il nastro conteneva un solo 0, accetta.
3. Se al passo 1, il nastro conteneva più di un singolo 0 ed il loro numero era dispari, rifiuta.
4. Riporta la testa all'estremità sinistra del nastro.
5. Va al passo 1. ”

Ogni iterazione del passo 1 dimezza il numero di 0. Quando la macchina si muove lungo il nastro nel passo 1, ricorda se il numero di 0 è pari o dispari. Se tale numero è dispari e maggiore di 1, il numero di 0 iniziale in input non poteva essere una potenza di 2. Pertanto la macchina, su questa istanza, rifiuta. Tuttavia, se il numero di 0 è 1, il numero iniziale doveva essere potenza di 2. Quindi, in questo caso la macchina accetta. La descrizione formale di $M_2 = (Q, \Sigma, \Gamma, \delta, q_1, q_{accept}, q_{reject})$ è la seguente.

- $Q = \{q_1, q_2, q_3, q_4, q_5, q_{accept}, q_{reject}\}$
- $\Sigma = \{0\}$
- $\Gamma = \{0, x, \sqcup\}$
- δ è descritto mediante un diagramma di stato.
- Gli stati iniziale, di accettazione e di rifiuto sono rispettivamente q_1, q_{accept} e q_{reject} .



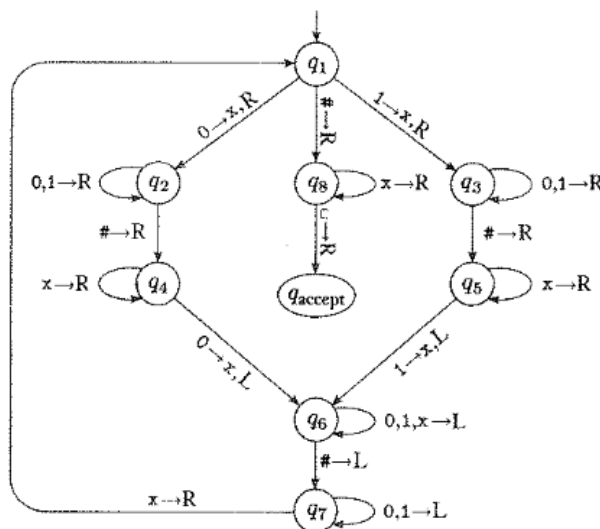
In questo diagramma di stato, nella transizione da q_1 a q_2 compare l'etichetta $0 \rightarrow \sqcup, R$. Questa etichetta indica che quando siamo nello stato q_1 e la testina legge 0, la macchina si porta nello stato q_2 , scrive $\sqcup$ e sposta la testina a destra. Per chiarezza utilizziamo l'abbreviazione $0 \rightarrow R$ per indicare la transizione da q_3 a q_4 , il che significa che la macchina si sposta verso destra durante la lettura di 0 quando si trova nello stato q_3 ma non modifica il nastro. Questa macchina inizia scrivendo un simbolo blank sullo 0 più a sinistra del nastro in modo da poter identificare l'estrema sinistra del nastro del passo 4. Diamo un esempio di esecuzione di questa macchina su input 0000. La configurazione iniziale è $q_1 0000$. La sequenza delle configurazioni della macchina è la seguente.

$q_1 0000$	$\sqcup q_5 x 0 x \sqcup$	$\sqcup x q_5 x x \sqcup$
$\sqcup q_2 000$	$q_5 \sqcup x 0 x \sqcup$	$\sqcup q_5 x x x \sqcup$
$\sqcup x q_3 00$	$\sqcup q_2 x 0 x \sqcup$	$q_5 \sqcup x x x \sqcup$
$\sqcup x 0 q_4 0$	$\sqcup x q_2 0 x \sqcup$	$\sqcup q_2 x x x \sqcup$
$\sqcup x 0 x q_3 \sqcup$	$\sqcup x x q_3 x \sqcup$	$\sqcup x q_2 x x \sqcup$
$\sqcup x 0 q_5 x \sqcup$	$\sqcup x x x q_3 \sqcup$	$\sqcup x x q_2 x \sqcup$
$\sqcup x q_5 0 x \sqcup$	$\sqcup x x q_5 x \sqcup$	$\sqcup x x x q_2 \sqcup$
		$\sqcup x x x \sqcup q_{accept}$

Esempio

In questo esempio viene descritta formalmente la macchina di Turing $M_1 = (Q, \Sigma, \Gamma, \delta, q_1, q_{accept}, q_{reject})$ per il linguaggio $B = \{w\#w \mid w \in \{0, 1\}^*\}$.

- $Q = \{q_1, \dots, q_8, q_{accept}, q_{reject}\}$
- $\Sigma = \{0, 1, \#\}$ e $\Gamma = \{0, 1, \#, x, \sqcup\}$
- Descriviamo δ mediante il diagramma di stato.
- Gli stati iniziale, di accettazione e di rifiuto sono rispettivamente $q_1, q_{accept}, q_{reject}$



L'etichetta $0, 1 \rightarrow R$ sulla transizione che va da q_3 a se stesso. Tale etichetta significa che la macchina rimane in q_3 ed effettua una mossa a destra quando legge uno 0 oppure un 1 nello stato q_3 . Non cambia il simbolo sul nastro. La fase 1 è attuata mediante gli stati da q_1 a q_7 , e la fase 2 degli stati rimanenti.

3.2 Varianti di macchine di Turing

Esistono diverse alternative delle macchine di Turing, comprese le versioni multinastro o non deterministiche. Sono dette *varianti* del modello macchina di Turing. Il modello originale e le sue ragionevoli varianti hanno tutti lo stesso potere computazionale, quindi, riconoscono la stessa classe di linguaggi. Chiamiamo *robustezza* questa invarianza ad alcune variazioni nella definizione. Per illustrare la robustezza del modello di macchina di Turing cerchiamo di variare il tipo di funzione transizione consentito. Nella definizione di **macchina di Turing** data precedentemente, la funzione di transizione forza la testina a spostarsi verso sinistra o destra ad ogni passo; la testina non può semplicemente restare ferma. Supponiamo di aver permesso alla macchina di Turing la capacità di restare ferma. La funzione di transizione avrebbe allora la forma: $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, S\}$.

3.2.1 Macchina di Turing multinastro

Una *macchina di Turing multinastro* è come una normale macchina di Turing con vari nastri. Ogni nastro ha la sua testina per la lettura e la scrittura. Inizialmente l'input si trova sul nastro 1, mentre gli altri nastri sono vuoti. La funzione di transizione viene modificata per consentire la lettura, la scrittura e lo spostamento della testina contemporaneamente su alcuni o tutti i nastri. Formalmente,

$$\delta : Q \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{L, R, S\}^k$$

dove k è il numero di nastri. L'espressione

$$\delta(q_i, a_1, \dots, a_k) = (q_j, b_1, \dots, b_k, L, R, \dots, L)$$

significa che, se la macchina si trova nello stato q_i e le testine da 1 a k leggono i simboli da a_1 ad a_k , la macchina va nello stato q_j , scrive i simboli da b_1 a b_k e muove ogni testina a sinistra o a destra, o la fa restare ferma, come specificato.

Teorema 3.1: Equivalenza con le macchine di Turing multinastro

Per ogni macchina di Turing multinastro esiste una macchina di Turing a nastro singolo equivalente.

Dimostrazione. Mostriamo come convertire una macchina di Turing multinastro M in una TM S equivalente a nastro singolo. L'idea chiave è quella di mostrare come simulare M con S . Supponiamo che M abbia k nastri. Allora S simula l'effetto di k nastri memorizzando le loro informazioni sul suo sin-

golo nastro. Essa utilizza il nuovo simbolo $\#$ come delimitatore per separare i contenuti dei diversi nastri. Oltre al contenuto dei nastri, S deve tenere traccia delle posizioni delle testine. Lo fa scrivendo un simbolo con un punto sopra per contrassegnare la posizione cui si troverebbe la testina di quel nastro. I simboli del nastro puntati sono semplicemente simboli nuovi che sono stati aggiunti all'alfabeto del nastro.

$S =$ "Su input $w = w_1 \dots w_n$:

1. Inizialmente S mette il suo nastro nel formato che rappresenta tutti i k nastri di M . Il nastro formattato contiene:

$$\# \dot{w}_1 w_2 \dots w_n \# \sqcup \# \sqcup \# \dots \#$$

2. Per simulare una singola mossa, S scansione il suo nastro dal primo $\#$, che segna l'estremità sinistra, al $(k + 1) \#$, che segna l'estremità di destra, per determinare i simboli puntati dalle testine virtuali. Successivamente S fa un secondo passaggio per aggiornare i nastri in accordo alla funzione di transizione di M .
3. Se in qualsiasi momento S sposta una delle testine virtuali a destra su un $\#$, questa azione significa che M ha spostato la testina corrispondente sulla part di nastro vuota non letta in precedenza. Quindi S scrive un simbolo blank in questa cella del nastro e sposta il contenuto del nastro, da questa cella fino al simbolo $\#$ più a destra, di un'unità a destra.

Poi prosegue la simulazione come prima".

□

Corollario 3.1

Un linguaggio è Turing-riconoscibile se e solo se qualche macchina di Turing multinastro lo riconosce.

Dimostrazione. Un linguaggio Turing-riconoscibile è riconosciuto da una macchina di Turing ordinaria, che è un caso particolare di una macchina di Turing multinastro. Questo prova un verso di questo corollario. L'altro verso segue dal teorema di **equivalenza con le macchine di Turing multinastro**. □

3.2.2 Macchina di Turing non deterministica

In una macchina di Turing non deterministica, in un qualsiasi punto della computazione la macchina può procedere effettuando varie scelte. La funzione di transizione di una macchina di Turing non deterministica è della forma:

$$\delta : Q \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma \times \{L, R\})$$

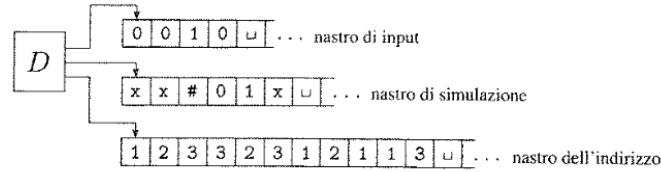
La computazione di una macchina di Turing non deterministica è un albero i cui rami corrispondono alle diverse scelte possibili previste per la macchina.

Teorema 3.2: Equivalenza con le macchine di Turing non deterministiche

Per ogni macchina di Turing non deterministica esiste una macchina di Turing deterministica equivalente.

Possiamo simulare qualsiasi TM non deterministica N con una TM deterministica D . L'idea di base della simulazione consiste nel provare tutte le possibili scelte che può fare N durante la sua computazione non deterministica. Se D trova lo stato di accettazione su qualsiasi di questi rami, allora D accetta. Altrimenti, la simulazione di D non terminerà. Possiamo guardare la computazione di N su un input w come ad un albero. Ogni ramo rappresenta una scelta non deterministica. Ogni nodo dell'albero è una configurazione di N . La TM D esplora quest'albero alla ricerca di una configurazione di accettazione. Non conviene fare esplorare a D utilizzando una depth-first search in quanto D potrebbe rimanere per sempre ad esplorare un eventuale cammino infinito e non trovare una configurazione accettante in quel cammino. Quindi D deve esplorare l'albero secondo l'algoritmo di ricerca breadth-first search. Questo metodo garantisce che D visita ogni nodo dell'albero finché non incontra una configurazione di accettazione.

Dimostrazione. La TM D deterministica che simula ha tre nastri. Dal teorema di **equivalenza con le macchine di Turing multinastro**, questa disposizione equivale ad avere un singolo nastro. La macchina D utilizza i suoi tre nastri in maniera particolare, come illustrato nella seguente figura. Il nastro 1 contiene sempre la stringa di input e non viene mai modificato. Il nastro 2 mantiene solo una copia del nastro di N corrispondente a qualche diramazione della sua computazione non deterministica. Il nastro 3 tiene traccia della posizione di D nell'albero delle computazioni di N .



Consideriamo la rappresentazione dei dati sul nastro 3. Ogni nodo dell'albero può avere al massimo b figli, dove b è la dimensione del più grande insieme di scelte possibili date dalla funzione di transizione di N . Ad ogni nodo della struttura assegniamo un indirizzo che è una stringa sull'alfabeto $\Gamma_b = \{1, 2, \dots, b\}$. Assegniamo l'indirizzo 231 al nodo a cui si arriva partendo dalla radice, spostandosi al suo secondo figlio, spostandosi ancora da tale nodo al suo terzo figlio ed infine spostandosi al primo figlio di quest'ultimo nodo. Ogni simbolo della stringa ci dice quale deve essere la scelta successiva, durante la simulazione di un passo in una ramificazione di una computazione non deterministica di N . A volte un simbolo può non corrispondere ad una scelta se sono disponibili troppe poche scelte per una configurazione. In tal caso l'indirizzo non è valido e non corrisponde a nessun nodo. Il nastro 3 contiene una stringa su Γ_b . Essa rappresenta la ramificazione della computazione di N dalla radice al nodo indirizzato da tale stringa, a meno che l'indirizzo non sia valido. La stringa vuota è l'indirizzo della radice dell'albero. Descriviamo ora D .

1. Inizialmente il nastro 1 contiene l'input w e i nastri 2 e 3 sono vuoti.
2. Copia il nastro 1 sul nastro 2 ed inizializza la stringa sul nastro 3 a ϵ .
3. Utilizza il nastro 2 per simulare N con input w su una ramificazione della sua computazione non deterministica. Prima di ogni passo di N , consulta il simbolo successivo sul nastro 3 per determinare quale scelta fare tra quelle consentite della funzione di transizione di n . Se non rimangono più simboli sul nastro 3 o se questa scelta deterministica non è valida, interrompe questo cammino andando alla fase 4. Va alla fase 4 anche quando si verifica una configurazione di rifiuto. Se incontra una configurazione di accettazione, allora accetta l'input.

4. Sostituisce la stringa sul nastro 3 con la stringa successiva rispetto all'ordine sulle stringhe. Simula la ramificazione successiva della computazione di N andando al passo 2.

□

Corollario 3.2

Un linguaggio è Turing-riconoscibile se e solo se esiste una macchina di Turing non deterministica che lo riconosce.

Dimostrazione. Qualsiasi TM deterministica è automaticamente una TM non deterministica e quindi una direzione di questo teorema è già dimostrata. L'altra direzione segue dal teorema di **equivalenza con le macchine di Turing non deterministiche**. □

Possiamo semplificare la dimostrazione del teorema di **equivalenza con le macchine di Turing non deterministiche** in modo che se N si ferma sempre su tutte le ramificazioni durante la computazione, D si ferma sempre. Chiameremo una macchina non deterministica *decisore* se tutte le ramificazioni si fermano su ogni input.

Corollario 3.3

Un linguaggio è decidibile se e solo se esiste una macchina di Turing non deterministica che lo decide.

3.2.3 Enumeratori

Per indicare un linguaggio Turing-riconoscibile è possibile anche utilizzare l'espressione *linguaggio ricorsivamente enumerabile* per indicare un linguaggio Turing-riconoscibile. QUesto termine deriva da una macchina di Turing chiamata *enumeratore*. Un enumeratore è una macchina di Turing con una stampante collegata. La macchina di Turing può utilizzare tale stampante come dispositivo di output per stampare stringhe. Ogni volta che la macchina di Turing vuole aggiungere una stringa alla lista, invia la stringa alla stampante.

Un enumeratore E inizia con un nastro di input vuoto. Se l'enumeratore non si ferma, esso può stampare un elenco infinito di stringhe. Il linguaggio enumerato da E è la collezione di tutte le stringhe del linguaggio in qualsiasi ordine, eventualmente anche con ripetizioni.

Teorema 3.3: Equivalenza con gli enumeratori

Un linguaggio è Turing-riconoscibile se e solo se esiste un enumeratore che lo enumera.

Dimostrazione. Come prima cosa, dimostriamo che se abbiamo un enumeratore E che enumera un linguaggio A , allora esiste una TM M che riconosce A . La TM M funziona come segue.

M = “ Su input w :

1. Esegue E . Ogni volta che E genera una stringa, la confronta con w .
2. Se w appare nell’output di E , accetta.”

M accetta quelle stringhe che compaiono sulla lista di E . Ora occupiamoci dell’altra direzione. Se la TM M riconosce un linguaggio A , possiamo costruire il seguente enumeratore E per A . Sia $s_1, s_2, \dots$ una lista di tutte possibili stringhe in Σ^* .

E = “Ignora l’input.

1. Ripete i seguenti passi per $i = 1, 2, \dots$
2. Esegue M per i passi su ogni input $s_1, s_2, \dots, s_i$.
3. Se una qualche computazione accetta, stampa la corrispondente s_j .

Se M accetta una particolare stringa s , alla fine s apparirà nella lista generata da E . In realtà, essa apparirà nella lista un numero infinito di volte, perchè M ritorna all’inizio su ogni stringa per ogni ripetizione del passo 1.

Questa procedura fa sì che M lavori in parallelo su tutte le possibili stringhe di input. $\square$

3.3 La definizione di algoritmo

Parlando in maniera informale, un *algoritmo* è un insieme di istruzioni semplici per l’esecuzione di un certo compito. Gli algoritmi svolgono anche un ruolo importante in matematica. Anche se gli algoritmi hanno avuto una lunga storia nel campo della matematica, la nozione di algoritmo non è stata formalizzata con precisione fino al ventesimo secolo.

3.3.1 I problemi di Hilbert

Nel 1900, il matematico David Hilbert al Congresso Internazionale dei Matematici a Parigi, identificò 23 problemi matematici da risolvere per il secolo successivo. Il decimo problema riguardava gli algoritmi.

Prima di descrivere il problema, introduciamo i polinomi. Un *polinomio* è una somma di termini, dove ogni termine è il prodotto di alcune variabili ed una costante, chiamata *coefficiente*.

Una *radice* di un polinomio è un'assegnazione di valori alle sue variabili per cui il valore del polinomio è 0.

Il decimo problema di Hilbert consiste nell'ideare un algoritmo per verificare se un polinomio abbia o meno una radice intera (ovvero le radici sono tutte numeri interi). Hilbert non usò il termine algoritmo, piuttosto "un processo in base al quale esso può essere determinato da un numero finito di operazioni". Hilbert ha chiesto esplicitamente che fosse "progettato" un algoritmo.

Non esiste un algoritmo per questo problema; si tratta di un problema non risolvibile algebricamente. Per poter provare che un algoritmo non esiste è necessario avere una definizione di algoritmo chiara. La definizione è arrivata nel 1936 da parte di Alonzo Church ed Alan Turing.

Church utilizzò un sistema di notazione detto λ -calcolo per definire gli algoritmi. Turing lo fece con le sue macchine. È stato dimostrato che queste definizioni sono equivalenti. Questa connessione è chiamata **tesi di Church-Turing**.

La tesi di Church-Turing fornisce la definizione di algoritmo necessaria per risolvere il decimo problema di Hilbert.

Riscriviamo il decimo problema di Hilbert nella nostra terminologia.

$$D = \{p \mid p \text{ è un polinomio avente radice intera}\}$$

Il decimo problema di Hilbert chiede se l'insieme D è decidibile. La risposta è negativa. Di contro possiamo dimostrare che D è Turing-riconoscibile. Prima di fare ciò, consideriamo un problema più semplice. Si tratta di un analogo del decimo problema di Hilbert per i polinomi che hanno un'unica variabile, come per esempio $4x^3 - 2x^2 + x - 7$. Sia:

$$D_1 = \{p \mid p \text{ è un polinomio su } x \text{ avente una radice intera}\}$$

Ecco una TM M_1 che riconosce D_1 :

M_1 = "Su input $\langle p \rangle$: dove p è un polinomio sulla variabile x .

1. Valuta p con x posta successivamente ai valori 0, 1, -2, 2, -2, 3, -3, Se in un qualsiasi momento la valutazione del polinomio è 0, accetta".

Se p ha una radice intera, M_1 ad un certo punto la trova e accetta. Se p non ha radice intera, M_1 sarà in esecuzione per sempre. Nel caso di più variabili,

possiamo presentare una TM M simile che riconosce D . Ora M considera tutte le possibili impostazioni delle variabili con valori interi.

Sia M_1 che M sono riconoscitori, ma non decisori. Possiamo convertire M_1 in un decisore per D_1 , perchè possiamo calcolare dei limiti all'intervallo dei valori in cui possono trovarsi le radici di un polinomio a singola variabile e limitare la ricerca a questo intervallo di valori. Se non si trova una radice all'interno di questi limiti, la macchina rifiuta. Il teorema Matijasevič mostra che non è possibile calcolare limiti analoghi per polinomi a più variabili.

Capitolo 4

Decidibilità

4.1 Linguaggi decidibili

4.1.1 Problemi decidibili relativi a linguaggi regolari

Iniziamo con alcuni problemi computazionali relativi ad automi finiti. Forniamo algoritmi per testare se un automa finito accetta o meno una stringa, se il linguaggio di un automa finito è vuoto e se due automi finiti sono equivalenti.

Il *problema dell'accettazione* per DFA consistente nel testare se un particolare automa finito deterministico accetta una data stringa, può essere espresso come un linguaggio, A_{DFA} . Questo linguaggio contiene le codifiche di tutti i DFA con le stringhe che essi accettano. Definiamo

$$A_{DFA} = \{ \langle B, w \rangle \mid B \text{ è un DFA che accetta la stringa di input } w \}$$

Il problema di verifica se un DFA B accetta l'input w coincide con il problema di verificare se $\langle B, w \rangle$ è un elemento del linguaggio A_{DFA} . Mostrare che il linguaggio è decidibile equivale a mostrare che il problema computazionale è decidibile.

Teorema 4.1

A_{DFA} è un linguaggio decidibile.

Per dimostrare questo teorema dobbiamo presentare una TM M che decide A_{DFA} . $M =$ “ Su input $\langle B, w \rangle$, dove B è un DFA e w è una stringa

1. Simula B su input w

2. Se la simulazione termina in uno stato di accettazione, accetta, altrimenti rifiuta. ”

Dimostrazione. Per dimostrare che A_{DFA} è decidibile costruiamo una TM M che ha:

- una codifica di B sul primo nastro (regole);
- una codifica di w sul secondo nastro;
- una codifica dello stato corrente di B sul terzo nastro.

M cerca la transizione giusta sul primo nastro e modifica stato corrente e carattere osservato. Se lo stato è finale, allora M accetta, altrimenti rifiuta. $\square$

Possiamo dimostrare un teorema simile per automi a stati finiti non deterministici. Dato:

$$A_{NFA} = \{ \langle B, w \rangle \mid B \text{ è un NFA che accetta la stringa di input } w \}$$

Teorema 4.2

A_{NFA} è un linguaggio decidibile.

Dimostrazione. Presentiamo una TM M che decide A_{NFA} . Potremmo progettare N in modo che operi come M , simulando un NFA invece di un DFA. Invece, procediamo in modo diverso per illustrare una nuova idea: N usa M come sottoprocedura. Poichè M è progettata per funzionare con un DFA, N prima converte l’NFA che riceve come input in un DFA e poi lo passa ad M . $N =$ “Su input $\langle B, w \rangle$, dove B è un NFA e w è una stringa:

1. Converte l’NFA B in un DFA C equivalente;
2. Esegue la TM M su input $\langle C, w \rangle$
3. Se M accetta, *accetta*, altrimenti, *rifiuta*”.

$\square$

In modo analogo possiamo determinare se un’espressione regolare genera una data stringa. Consideriamo

$$A_{REX} = \{ \langle R, w \rangle \mid R \text{ è un’espressione regolare che genera la stringa } w \}$$

Teorema 4.3

A_{REX} è un linguaggio decidibile.

Dimostrazione. La seguente TM P decide A_{REG} .

$P =$ “ Su input $\langle R, w \rangle$, dove R è un’espressione regolare e w è una stringa:

1. Converti l’espressione regolare R in un NFA A equivalente;
2. Esegui la TM N su input $\langle A, w \rangle$
3. Se N accetta, *accetta*, altrimenti, *rifiuta*”.

□

Questi tre teoremi dicono che, ai fini della decidibilità, è equivalente presentare alla macchina di Turing, un DFA, un NFA, oppure un’espressione regolare, perchè la macchina è in grado di convertire una codifica nell’altra.

Ora affrontiamo un diverso tipo di problema concernente gli automi a stati finiti: il *test del vuoto* per il linguaggio di un automa finito. Nella dimostrazione seguente, dobbiamo determinare se un automa finito accetta una qualche stringa. Consideriamo:

$$E_{DFA} = \{ \langle A \rangle \mid A \text{ è un DFA e } L(A) = \emptyset \}$$

Teorema 4.4

E_{DFA} è un linguaggio decidibile.

Dimostrazione. Il DFA accetta almeno una stringa se e solo se dallo stato iniziale può raggiungere uno stato di accettazione percorrendo il verso delle frecce del DFA. Per verificare questa condizione possiamo progettare una TM T tale che:

$T =$ “ Su input $\langle A \rangle$, dove A è un DFA:

1. Marca lo stato iniziale di A .
2. Ripete fino a quando non vengono più marcati nuovi stati:
 - 2.1. Marca qualsiasi stato che ha una transizione proveniente da uno stato già marcato.
3. Se nessuno stato di accettazione risulta marcato, *accetta*, altrimenti, *rifiuta*”.

□

Il prossimo teorema afferma che determinare se due DFA riconoscono lo stesso

linguaggio è decidibile. Sia:

$$EQ_{DFA} = \{ \langle A, B \rangle \mid A \text{ e } B \text{ sono DFA e } L(A) = L(B) \}$$

Teorema 4.5

EQ_{DFA} è un linguaggio decidibile.

Dimostrazione. Per dimostrare questo teorema costruiamo un nuovo DFA C a partire da A e B , dove C accetta solo quelle stringhe che sono accettate da A o da B , ma non da entrambi. In particolare, se A e B riconoscono lo stesso linguaggio, C non accetterà nulla. Il linguaggio di C è:

$$L(C) = (L(A) \cap \overline{L(B)}) \cup (\overline{L(A)} \cap L(B))$$

Questa espressione è denotata come *differenza simmetrica* di $L(A)$ e $L(B)$. La differenza simmetrica è utile perché $L(C) = \emptyset \iff L(A) = L(B)$. Siamo in grado di costruire da A e da B con le costruzioni fatte per dimostrare che la classe dei linguaggi regolari risulta chiusa rispetto alle operazioni di complemento, unione e intersezione.

Costruiamo $F =$ “Su input $\langle A, B \rangle$, dove A e B sono DFA:

1. Costruisce il DFA C come descritto.
2. Esegue la TM M su input $\langle C \rangle$.
3. Se T accetta, *accetta*, altrimenti, *rifiuta*.

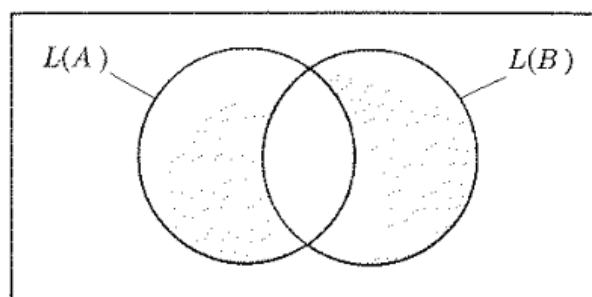


Figura 4.1: Differenza simmetrica di $L(A)$ e $L(B)$

□

4.1.2 Problemi decidibili relativi a linguaggi context-free

Descriviamo ora algoritmi per determinare se una CFG genera una particolare stringa e per determinare se il linguaggio di una CFG è vuoto.

Consideriamo

$$A_{CFG} = \{ \langle G, w \rangle \mid G \text{ è una CFG che genera la stringa } w \}$$

Teorema 4.6

A_{CDG} è un linguaggio decidibile.

Per la CFG G e la stringa w vogliamo determinare se G genera w . Un'idea è quella di utilizzare G per passare attraverso tutte le derivazioni per determinare se ne esiste una di w . Questo non funziona, poichè bisognerebbe provare infinite derivazioni. Se G non genera w , questo algoritmo potrebbe non fermarsi. Tale idea dà una macchina di Turing che è un riconoscitore, non un decisore, per A_{CFG} . Per rendere questa macchina di Turing un decisore occorre assicurare che l'algoritmo provi solo un numero finito di derivazioni. Precedentemente, abbiamo dimostrato che, se G fosse in forma normale di Chomsky, qualsiasi derivazione di w avrebbe $2n - 1$ passi, dove n è la lunghezza di w .

Dimostrazione. Diamo la TM S per A_{CFG} .

$S =$ “ Su input $\langle G, w \rangle$, dove G è una CFG e w è una stringa:

1. Converti G in una grammatica equivalente in forma normale di Chomsky.
2. Lista tutte le derivazioni di $2n - 1$ passi, dove n è la lunghezza di w ; tranne se $n = 0$, in tal caso lista tutte le derivazioni di un passo.
3. Se una di tali derivazioni genera w , *accetta*, altrimenti, *rifiuta*.”

□

Il problema di determinare se una CFG genera una particolare stringa è correlato al problema della compilazione dei linguaggi di programmazione. L'algoritmo nella TM S è molto inefficiente e non sarebbe mai utilizzato in pratica.

Ricordiamo che le CFG e i PDA sono equivalenti, quindi tutto ciò che vale per la decidibilità dei problemi concernenti le CFG vale anche per i PDA. Torniamo ora al problema del test del vuoto per il linguaggio di una CFG. Come abbiamo fatto per i DFA, possiamo dimostrare che il problema di determinare se una CFG

genera almeno una stringa è decidibile. Sia:

$$E_{CFG} = \{ \langle G \rangle \mid G \text{ è una CFG ed } L(G) = \emptyset \}$$

Teorema 4.7

E_{CFG} è un linguaggio decidibile.

Per progettare un algoritmo per questo problema, potremmo tentare di utilizzare la TM S e passare attraverso tutte le possibili w . Però le w da testare sono infinite, quindi questo metodo potrebbe non terminare mai. Per determinare se il linguaggio di una grammatica è vuoto, abbiamo bisogno di testare se la variabile iniziale può generare una stringa di terminali. L'algoritmo fa questo risolvendo un problema più generale. Determina per ogni variabile se essa è in grado di generare una stringa di terminali. Quando l'algoritmo ha determinato che una variabile può generare qualche stringa di terminali, l'algoritmo tiene traccia di queste informazioni marcando tale variabile. Dappiamo, l'algoritmo marca tutti i simboli terminali della grammatica. Poi, scandisce tutte le regole della grammatica. Se trova una regola che consente a qualche variabile di essere sostituita da una stringa di simboli, che sono già tutti marcati, l'algoritmo sa che anche questa variabile può essere marcata. L'algoritmo continua in questo modo finché non può marcare ulteriori variabili. La TM R implementa questo algoritmo.

Dimostrazione. $R =$ “ Su input $\langle G \rangle$, dove G è una CFG:

1. Marca tutti i simboli terminali in G .
2. Ripete fin quando nessuna nuova variabile viene marcata:
 - 2.1. Marca una qualsiasi variabile A tale che G ha una regola $A \rightarrow U_1 U_2 \cdots U_k$ ed ogni simbolo $U_1, \dots, U_k$ è già stato marcato.
3. Se la variabile iniziale non è segnata, *accetta*, altrimenti, *rifiuta*.

□

Consideriamo ora il problema di determinare se due grammatiche context-free generano lo stesso linguaggio. Sia:

$$EQ_{CFG} = \{ \langle G, H \rangle \mid G \text{ e } H \text{ sono CFG e } L(G) = L(H) \}$$

Potremmo utilizzare lo stesso procedimento utilizzato per dimostrare che EQ_{DFA} è decidibile. Ma questo procedimento non va bene per le CFG perchè la classe

dei linguaggi context-free non è chiusa per complemento o intersezione. Infatti, EQ_{CFG} non è decidibile.

Ora mostriamo che i linguaggi context-free possono essere decisi dalle macchine di Turing.

Teorema 4.8

Ogni linguaggio context-free è decidibile.

Sia A un CFL. Il nostro obiettivo è dimostrare che A è decidibile. Dimostriamo questo teorema con la TM S per decidere A_{CFG} .

Dimostrazione. Sia G una CFG per A , progettiamo una TM M_G che decide A . Costruiamo una copia di G in M_G . Essa funziona come segue.

$M_G =$ “Su input w :

1. Esegue la TM S su input $\langle G, w \rangle$.
2. Se questa macchina accetta, *accetta*; altrimenti, *rifiuta*.

□

Quest'ultimo teorema fornisce il collegamento finale nelle relazioni tra le quattro principali classi di linguaggi che abbiamo descritto finora: regolari, context-free, decidibili e Turing-riconoscibili.

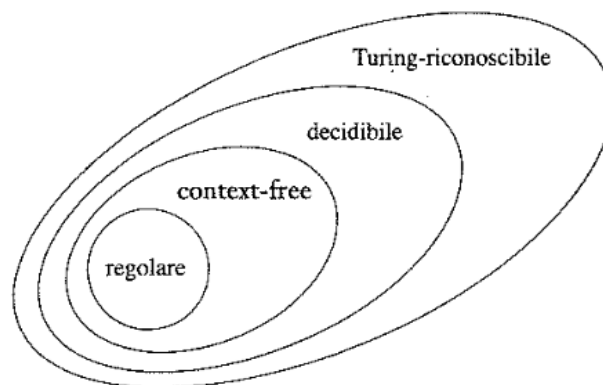


Figura 4.2: Le relazioni tra le classi di linguaggi

4.2 Indecidibilità

In questa sezione dimostriamo uno dei teoremi più importanti della teoria della computazione: esiste un problema specifico che è algoritmicamente irrisolvibile. Affrontiamo il primo problema che stabilisce l'indecidibilità di uno specifico linguaggio: il problema di determinare se una macchina di Turing accetta una determinata stringa in input. Chiamiamo tale linguaggio A_{TM} . Sia:

$$A_{TM} = \{ \langle M, w \rangle \mid M \text{ è una TM e } M \text{ accetta } w \}$$

Teorema 4.9

A_{TM} è indecidibile.

Prima di dimostrare questo teorema, osserviamo che A_{TM} è Turing-riconoscibile. Quindi questo teorema mostra che i riconoscitori sono più potenti dei decisori. Richiedere che una TM si fermi su ogni input limita le tipologie di linguaggi che possono essere riconosciuti. La seguente macchina di Turing U riconosce A_{TM} . $U =$ " Su input $\langle M, w \rangle$ dove M è una TM e w è una stringa:

1. Simula M su input w
2. Se durante la computazione M entra nello stato di accettazione, *accetta*; se M entra nello stato di rifiuto, *rifiuta*. "

Si noti che questa macchina cicla su input $\langle M, w \rangle$ se M cicla su w , questo è il motivo per cui non decide A_{TM} .

4.2.1 Il metodo della diagonalizzazione

La dimostrazione dell'indecidibilità di A_{TM} utilizza una tecnica chiamata *diagonalizzazione*.

DEFINIZIONE 4.1: RICHIAMO SULLE FUNZIONI

Supponiamo di avere gli insiemi A e B ed una funzione f da A a B . Diciamo che f è *iniettiva*, se essa non mappa mai due elementi diversi in uno stesso punto, cioè se $f(a) \neq f(b)$ ogniqualvolta $a \neq b$.

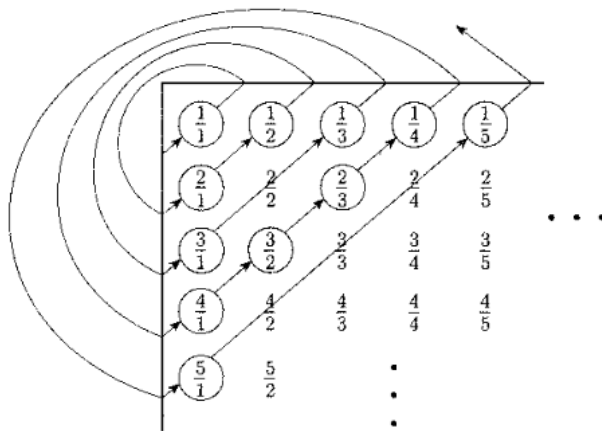
Diciamo che f è *suriettiva* se tocca ogni elemento di B - cioè se per ogni $b \in B$ esiste un $a \in A$ tale che $f(a) = b$.

Diciamo che A e B hanno *stessa cardinalità* se esiste una funzione iniettiva e suriettiva $f : A \rightarrow B$. Una funzione che è sia iniettiva che suriettiva è detta funzione *biettiva*. In una funzione biettiva ogni elemento di A viene

100

100

1

L
Z
i
eT
n

100%

100

Supponiamo che la biezione f esista. Prendiamo $f(1) = 3.14159 \dots$, $f(2) = 55.5555 \dots$, $f(3) = \dots$, e così via, tanto per dare alcuni valori per f . Allora f accoppia il numero 1 con $3.14159 \dots$, il numero 2 con $55.5555 \dots$, e così via. La tabella seguente mostra alcuni valori di un'ipotetica biezione f tra $\mathbb{N}$ e $\mathbb{R}$.

n	$f(n)$
1	3.14159...
2	55.55555...
3	0.12345...
4	0.50000...
$\vdots$	$\vdots$

Costruiamo il numero x desiderato, dandone la rappresentazione decimale. Si tratta di numero compreso tra 0 e 1, quindi tutte le sue cifre significative sono cifre frazionarie che seguono la virgola decimale. Il nostro obiettivo è quello di garantire che $x \neq f(n)$ per ogni n . Per garantire $x \neq f(1)$, scegliamo la prima cifra di x come una qualsiasi cifra diversa dalla prima cifra decimale 1 di $f(1) = 3.14159 \dots$. Arbitriamente, scegliamola pari a 4. Per assicurarci che $x \neq f(2)$, scegliamo la seconda cifra di x diversa dalla seconda cifra decimale 5 di $f(2)$. Arbitriamente, scegliamola pari a 6. Continuando in questo modo lungo la diagonale della tavola per f , otteniamo tutte le cifre di x . Sappiamo che x non è $f(n)$ per ogni n perchè differisce da $f(n)$ nell' n -esima cifra frazionaria. $\square$

Il teorema precedente ha un'importante applicazione nella teoria della computazione. Esso dimostra che alcuni linguaggi non sono decidibili e neppure Turing riconoscibili, per la ragione che l'insieme dei linguaggi è non numerabile mentre l'insieme di tutte le macchine di Turing è numerabile. Poichè ogni macchina di Turing è in grado di riconoscere un solo linguaggio e ci sono più linguaggi che macchine di Turing, alcuni linguaggi non sono riconosciuti da una qualche macchina di Turing.

Corollario 4.1

Alcuni linguaggi non sono Turing-riconoscibili.

Dimostrazione. Per dimostrare che l'insieme di tutte le macchine di Turing è numerabile dobbiamo prima osservare che l'insieme di tutte le stringhe Σ^* è numerabile, per ogni alfabeto Σ . Avendo solo un numero finito di stringhe di

ogni lunghezza, possiamo formare una lista di Σ^* scrivendo tutte le stringhe di lunghezza 0, lunghezza 1, lunghezza 2, e così via.

L'insieme di tutte le macchine di Turing è numerabile perchè ogni macchina di Turing M può essere codificata con una stringa $\langle M \rangle$. Se ci limitiamo a tralasciare quelle stringhe che non sono la codifica di una macchina di Turing, possiamo ottenere una lista di tutte le macchine di Turing. Per mostrare che l'insieme di tutti i linguaggi è non numerabile, dobbiamo prima osservare che l'insieme delle sequenze binarie infinite è non-numerabile.

Una *sequenza binaria infinita* è una sequenza senza fine di 0 e di 1. Sia $\mathcal{B}$ l'insieme di tutte sequenze binarie infinite. Possiamo dimostrare che $\mathcal{B}$ è non numerabile utilizzando una dimostrazione mediante diagonalizzazione. Sia $\mathcal{L}$ l'insieme di tutti i linguaggi sull'alfabeto Σ . Mostriamo che $\mathcal{L}$ è non-numerabile dando una corrispondenza con $\mathcal{B}$, dimostrando così che i due insiemi hanno la stessa cardinalità. Sia $\Sigma^* = \{s_1, s_2, s_3, \dots\}$. Ogni linguaggio $A \in \mathcal{L}$ ha un'unica sequenza in $\mathcal{B}$. Il bit i -esimo della sequenza è un 1 se $s_i \in A$ ed è uno 0 se $s_i \notin A$; questa è chiamata la *sequenza caratteristica* di A .

La funzione $f : \mathcal{L} \rightarrow \mathcal{B}$, dove $f(A)$ è uguale alla sequenza caratteristica di A , è sia iniettiva che suriettiva, quindi è una biezione. Pertanto, essendo $\mathcal{B}$ non numerabile, anche $\mathcal{L}$ è non numerabile.

Abbiamo così dimostrato che l'insieme di tutti i linguaggi non può essere messo in corrispondenza biunivoca con l'insieme di tutte le macchine di Turing. Concludiamo, quindi, che alcuni linguaggi non sono riconosciuti da una macchina di Turing. $\square$

4.2.2 Un linguaggio indecidibile

Dimostriamo l'indcidibilità del linguaggio:

$$A_{TM} = \{\langle M, w \rangle \mid M \text{ è una TM ed } M \text{ accetta } w\}$$

Dimostrazione. Assumiamo che A_{TM} è decidibile, per poi ottenere una contraddizione. Supponiamo che H sia un decisore per A_{TM} . Sull'input $\langle M, w \rangle$, dove M è una TM e w è una stringa, H si ferma ed accetta se M accetta w . Inoltre, H si ferma e rifiuta se M non accetta w . In altri termini, assumiamo che H è una TM, dove

$$H(\langle M, w \rangle) = \begin{cases} \text{accetta} & \text{se } M \text{ accetta } w \\ \text{rifiuta} & \text{se } M \text{ rifiuta } w \end{cases}$$

Ora costruiamo una nuova macchina di Turing D avente H come sottoprocedura. Questa nuova TM chiama H per determinare cosa fa M quando l'input

di M è la sua stessa descrizione $\langle M \rangle$. Una volta che D ha determinato quest'informazione, essa fa il contrario. Cioè, rifiuta se M accetta ed accetta se M non accetta. Diamo ora una descrizione di D .

$D =$ “ Su input $\langle M \rangle$, dove M è una TM:

1. Esegue H su input $\langle M, \langle M \rangle \rangle$
2. Da in output l'opposto di ciò che H dà in output. Cioè, se H accetta, *rifiuta*; e se H rifiuta, *accetta*.

Se D viene eseguita con la sua stessa descrizione $\langle D \rangle$, otteniamo:

$$D(\langle D \rangle) = \begin{cases} accetta, & \text{se } D \text{ non accetta } \langle D \rangle \\ rifiuta & \text{se } D \text{ accetta } \langle D \rangle \end{cases}$$

Indipendentemente da ciò che D fa, essa è costretta a fare il contrario, il che ovviamente è una contraddizione. Quindi, nè la TM D nè la TM H possono esistere. $\square$

Rivediamo i passi di questa prova. Supponiamo che la TM H decida A_{TM} . Quindi usiamo H per costruire una TM D che prende in input $\langle M \rangle$, tale che D accetta $\langle M \rangle$ esattamente quando M non accetta il suo input $\langle M \rangle$, Infine, eseguiamo D su se stessa. Quindi, le macchine eseguono le seguenti azioni, dove l'ultima riga fornisce la contraddizione.

- H accetta $\langle M, w \rangle$ esattamente quando M accetta w .
- D rifiuta $\langle M \rangle$ esattamente quando M accetta $\langle M \rangle$
- D rifiuta $\langle D \rangle$ esattamente quando D accetta $\langle D \rangle$.

4.2.3 Un linguaggio non Turing-riconoscibile

Nella sezione precedente abbiamo dimostrato che un linguaggio è non decidibile. Ora mostriamo un linguaggio che non è neppure Turing-riconoscibile. Si noti che A_{TM} non basta allo scopo, perchè abbiamo dimostrato che A_{TM} è Turing riconoscibile. Il teorema seguente mostra che, se un linguaggio ed il suo complemento sono entrambi Turing-riconoscibili, il linguaggio è decidibile. Quindi, per qualsiasi linguaggio non decidibile, o esso non è Turing-riconoscibile oppure il suo complemento non è Turing-riconoscibile. Diciamo che un linguaggio è *coTuring riconoscibile* se esso è il complemento di un linguaggio Turing-riconoscibile.

Teorema 4.11

Un linguaggio è decidibile se e solo se è Turing-riconoscibile e coTuring-riconoscibile.

Dimostrazione. Abbiamo due direzioni da dimostrare. In primo luogo, se A è decidibile, possiamo facilmente vedere che sia A che il suo complemento $\bar{A}$ sono Turing-riconoscibili. Qualsiasi linguaggio decidibile è Turing-riconoscibile, e il complemento di un linguaggio decidibile è a sua volta decidibile. Per la direzione inversa, se entrambi A e $\bar{A}$ sono Turing-riconoscibili, indichiamo con M_1 il riconoscitore per A e con M_2 il riconoscitore per $\bar{A}$. La seguente macchina di Turing M è un decisore per A .

$M =$ “ Su input w :

1. Esegue sia M_1 che M_2 su input w parallelo.
2. Se M_1 accetta, *accetta*; se M_2 accetta, *rifiuta*.

L'esecuzione di due macchine in parallelo significa che M ha due nastri, uno per simulare M_1 e l'altro per simulare M_2 . In questo caso M alterna la simulazione di un passo di M_1 con un passo di M_2 e continua finché una delle due accetta. Ora dimostriamo che M decide A . Ogni stringa w è in A , oppure in $\bar{A}$. Pertanto una tra M_1 e M_2 deve accettare w . Poiché M si ferma ogni volta che M_1 accetta oppure M_2 accetta, allora M si ferma sempre quindi è un decisore. Inoltre, accetta tutte le stringhe in A e respinge tutte le stringhe in $\bar{A}$. Quindi M è un decisore per A , e pertanto A è decidibile. $\square$

Corollario 4.2

$\overline{A_{TM}}$ non è Turing-riconoscibile.

Dimostrazione. Sappiamo che A_{TM} è Turing-riconoscibile. Se anche $\overline{A_{TM}}$ fosse Turing-riconoscibile, A_{TM} sarebbe decidibile. Ma, precedentemente, abbiamo dimostrato che A_{TM} non è decidibile, quindi $\overline{A_{TM}}$ non può essere Turing-riconoscibile. $\square$

Capitolo 5

Riducibilità

Nel capitolo precedente, abbiamo presentato esempi di problemi risolvibili tramite macchina di Turing e dato un esempio di problema, A_{TM} , che è computazionalmente irrisolvibile. In questo capitolo esaminiamo altri problemi irrisolvibili e per fare ciò introduciamo un metodo, detto *riducibilità*, per dimostrare che alcuni problemi sono computazionalmente irrisolvibili.

Una *riduzione* è un modo di convertire un problema in un altro problema in modo tale che una soluzione al secondo problema può essere usata per risolvere il primo problema. In termini della teoria della computabilità, se A è riducibile a B e B è decidibile, allora anche A è decidibile. Equivalentemente, se A è indecidibile e riducibile a B , B è indecidibile. Quest'ultima parte è la chiave per dimostrare che alcuni problemi sono indecidibili.

5.1 Problemi indecidibili dalla teoria dei linguaggi

Abbiamo già stabilito l'indecidibilità di A_{TM} , il problema di determinare se una macchina di Turing accetta un dato input. Consideriamo ora un problema affine, $HALT_{TM}$, il problema di determinare se una macchina di Turing si ferma (accettando o rifiutando) su un dato input. Questo problema è generalmente noto come il *problema della fermata*. Usiamo l'indecidibilità di A_{TM} per dimostrare l'indecidibilità del problema della fermata riducendo A_{TM} a $HALT_{TM}$. Sia

$$HALT_{TM} = \{ \langle M, w \rangle \mid M \text{ è una TM e } M \text{ si ferma su input } w \}$$

Teorema 5.1

$HALT_{TM}$ è indecidibile.

Questa dimostrazione è per assurdo. Assumiamo che $HALT_{TM}$ è decidibile ed utilizziamo questa assunzione per dimostrare che A_{TM} è decidibile, contraddicendo il teorema. L'idea chiave è quella di mostrare che A_{TM} è riducibile a $HALT_{TM}$.

Supponiamo di avere una TM R che decide $HALT_{TM}$. Usiamo quindi R per costruire S , una TM che decide A_{TM} .

Dimostrazione. Assumiamo al fine di ottenere una contraddizione che la TM R decide $HALT_{TM}$. Costruiamo la TM S per decidere A_{TM} , che opera come segue.

$S =$ “ Su input $\langle M, w \rangle$, una codifica di una TM M ed una stringa w :

- Esegue la TM R su input $\langle M, w \rangle$.
- Se R rifiuta, *rifiuta*.
- Se R accetta, simula M su w finché non si ferma.
- Se M ha accettato, *accetta*; se M ha rifiutato, *rifiuta*”.

Chiaramente, se R decide $HALT_{TM}$, allora S decide A_{TM} . Poiché A_{TM} è indecidibile, allora anche $HALT_{TM}$ deve essere indecidibile. $\square$

Presentiamo altri teoremi e le loro dimostrazioni come ulteriori esempi dell'uso della riducibilità per dimostrare l'indecidibilità. Sia

$$E_{TM} = \{ \langle M \rangle \mid M \text{ è una TM e } L(M) = \emptyset \}$$

Teorema 5.2

E_{TM} è indecidibile.

Seguiamo lo schema adottato nel teorema precedente. Assumiamo che E_{TM} è decidibile e mostriamo che A_{TM} è decidibile, una contraddizione. Sia R una TM che decide E_{TM} . Utilizziamo R per costruire la TM S che decide A_{TM}

Dimostrazione. Descriviamo la macchina M_1 .

$M_1 =$ “ Su input x :

1. Se $x \neq w$, *rifiuta*
2. Se $x = w$, esegue M su input w e *accetta* se M accetta”.

Questa macchina ha la stringa w come parte della sua descrizione. Essa verifica se $x = w$ nel modo ovvio, attraverso la scansione dell'input e confrontandolo carattere per carattere con w per determinare se coincidono. Mettendo insieme tutto questo, assumiamo che la TM R decide E_{TM} e costruiamo la TM S che decide A_{TM} come segue.

$S =$ “ Su input $\langle M, w \rangle$, una codifica di una TM M e una stringa w :

1. Usa la descrizione di M e w per costruire la TM M_1 descritta sopra.
2. Esegue R su input $\langle M_1 \rangle$
3. Se R accetta, *rifiuta*; se R rifiuta, *accetta*.”

S deve essere in grado di calcolare una descrizione di M_1 da una descrizione di M e w . Può farlo, perchè ha bisogno solo di aggiungere ad M alcuni stati in più che svolgono il test $x = w$. Se R fosse un decisore per E_{TM} , S sarebbe un decisore per A_{TM} . Un decisore per A_{TM} non può esistere, quindi sappiamo che E_{TM} deve essere indecidibile. $\square$

Un altro problema interessante è quello di determinare se una macchina di Turing riconosce un linguaggio che può essere riconosciuto anche da un modello di calcolo più semplice. Ad esempio, sia $REGULAR_{TM}$ il problema di determinare se una data macchina di Turing ha un automa finito equivalente. Questo problema equivale a determinare se la macchina di Turing riconosce un linguaggio regolare. Sia

$$REGULAR_{TM} = \{ \langle M \rangle \mid M \text{ è una TM ed } L(M) \text{ è un linguaggio regolare.} \}$$

Teorema 5.3

$REGULAR_{TM}$ è indecidibile.

Come sempre, assumiamo che $REGULAR_{TM}$ sia deciso da una TM R e utilizziamo questa assunzione per costruire una TM S che decide A_{TM} . L'idea è che S prenda il suo input $\langle M, w \rangle$ e modifichi M in modo che la risultante TM riconosca un linguaggio regolare se e solo se M accetta w .

Chiamiamo M_2 la macchina così modificata. Progettiamo M_2 in modo che riconosca un linguaggio non regolare $\{0^n 1^n \mid n \geq 0\}$ se M non accetta w , e riconosca il linguaggio regolare Σ^* se M accetta w . Dobbiamo specificare come S può costruire una tale M_2 da M a w . Qui, M_2 accetta automaticamente tutte le stringhe in $\{0^n 1^n \mid n \geq 0\}$. Inoltre, se M accetta w , M_2 accetta tutte le altre stringhe.

Non costruiamo M_2 per essere eseguita su qualche input, ma la costruiamo con il solo scopo di dare in input la sua descrizione al decisore per $REGULAR_{TM}$ che abbiamo assunto esistere. Quando tale decisore dà la sua risposta, possiamo usarla per rispondere se M accetta w o meno. Quindi possiamo decidere A_{TM} , una contraddizione.

Dimostrazione. Definiamo R come una TM che decide $REGULAR_{TM}$ e costruiamo una TM S che decide A_{TM} . Allora S funziona come segue.

$S =$ “ Su input $\langle M, w \rangle$, dove una M è una TM e w una stringa:

1. Costruisce la seguente TM M_2 .
 $M_2 =$ “ Su input x :
 - 1.1. Se x ha la forma $0^n 1^n$, accetta.
 - 1.2. Se x non ha tale forma, esegue M su input w e accetta se M accetta w ”.
2. Esegue R su input $\langle M_2 \rangle$
3. Se R accetta, accetta; se R rifiuta, rifiuta”.

□

Il seguente teorema mostra che verificare l'equivalenza di due macchine di Turing è un problema indecidibile. Sia

$$EQ_{TM} = \{ \langle M_1, M_2 \rangle \mid M_1 \text{ ed } M_2 \text{ sono TM ed } L(M_1) = L(M_2) \}$$

Teorema 5.4

EQ_{TM} è indecidibile.

Mostriamo che se EQ_{TM} fosse decidibile allora anche E_{TM} sarebbe decidibile eseguendo una riduzione da E_{TM} a EQ_{TM} . L'idea è semplice. E_{TM} è il problema di determinare se il linguaggio di una TM è vuoto. EQ_{TM} è il problema di determinare se i linguaggi delle due TM sono uguali. Se uno di questi linguaggi è $\emptyset$, dobbiamo determinare se il linguaggio dell'altra macchina è vuoto, cioè il problema E_{TM} .

Dimostrazione. Consideriamo una TM R che decide EQ_{TM} e costruiamo la TM S che decide E_{TM} come segue.

$S =$ “Su input $\langle M \rangle$, dove M è una TM:

1. Esegue R su input $\langle M, M_1 \rangle$, dove M_1 è una TM che rifiuta ogni input.

2. Se R accetta, *accetta*; se R rifiuta, *rifiuta*”.

Se R decide EQ_{TM} , S decide E_{TM} . Ma sappiamo che E_{TM} è indecidibile, quindi anche EQ_{TM} deve essere indecidibile. $\square$

Parte III

Teoria della complessità

Capitolo 6

Complessità di tempo

Anche quando un problema è decidibile, e quindi in linea di principio computazionalmente risolvibile, può non essere risolvibile in pratica se la soluzione richiede una quantità eccessiva di tempo di memoria.

6.1 Misure di complessità

Consideriamo il linguaggio $A = \{0^k 1^k \mid k \geq 0\}$. Ovviamente, A è un linguaggio decidibile. Calcoliamo di quanto tempo necessita una macchina di Turing a nastro singolo per decidere A . Esaminiamo la seguente TM M_1 per A .

$M_1 =$ “Su input w :

1. Scandisce il nastro e *rifiuta* se trova uno 0 a destra di un 1.
2. Ripete se il nastro contiene almeno uno 0 ed almeno un 1:
 - 2.1. Scandisce il nastro, cancellando uno 0 ed un 1.
3. Se rimane almeno uno 0 dopo che ogni simbolo 1 è stato cancellato, o se rimane almeno un 1 dopo che ogni simbolo 0 è stato cancellato, *rifiuta*. Altrimenti, se non rimangono né simboli 0 né simboli 1, *accetta*.”

Analizzeremo l'algoritmo della TM M_1 che decide A per determinare la quantità di tempo che impiega. Si analizza il **caso peggiore**, dove si valuta il tempo di esecuzione massimo tra tutti gli input di una determinata lunghezza. Si analizza il **caso medio**, dove si considera la media dei tempi di esecuzione su tutti gli input di una determinata lunghezza.

DEFINIZIONE 6.1: COMPLESSITÀ DI TEMPO

Sia M una macchina di Turing deterministica che si ferma su tutti gli input. Il *tempo di esecuzione* o *complessità di tempo* di M è la funzione $f : \mathbb{N} \rightarrow \mathbb{N}$, dove $f(n)$ è il numero massimo di passi che M utilizza su un qualsiasi input di lunghezza n . Se $f(n)$ è il tempo di esecuzione di M , diciamo che M ha tempo di esecuzione $f(n)$ e che M è una macchina di Turing di tempo $f(n)$.

6.1.1 Notazione O-grande ed o-piccola

Poichè il tempo di esecuzione di un algoritmo è spesso un'espressione complessa, solitamente ci limitiamo ad ottenerne una stima. L'utile metodo di stima, detto *analisi asintotica*, permette di valutare il tempo di esecuzione dell'algoritmo quando viene eseguito su grandi input. Lo facciamo prendendo in considerazione solo il termine di ordine maggiore dell'espressione del tempo di esecuzione dell'algoritmo, trascurando sia il coefficiente di tale termine, che tutti i termini di ordine inferiore. La *notazione asintotica*, viene anche chiamata *notazione O-grande*.

DEFINIZIONE 6.2: O-GRANDE

Siano f e g funzioni, $f, g : \mathbb{N} \rightarrow \mathbb{R}^+$. Si dice che $f(n) = O(g(n))$ se esistono interi positivi c e n_0 tali che per ogni $n \geq n_0$

$$f(n) \leq cg(n)$$

Quando $f(n) = O(g(n))$, diciamo che $g(n)$ è un *limite superiore* per $f(n)$, o più precisamente, che $g(n)$ è un *limite superiore asintotico* per $f(n)$, per sottolineare che stiamo ignorando le costanti.

$\mathbb{R}^+$ è l'insieme dei numeri reali non negativi.

Esempio

Sia $f_1(n) = 5n^3 + 2n^2 + 22n + 6$. Selezionando il termine di coefficiente maggiore e trascurando il suo coefficiente, si ottiene $f_1(n) = O(n^3)$.

Verifichiamo che questo risultato soddisfa la definizione formale di **O-grande**. Lo facciamo ponendo $c = 6$ e $n_0 = 10$. Quindi $5n^3 + 2n^2 + 22n + 6 \leq 6n^3$ per ogni $n \geq 10$. Inoltre, $f_1(n) = O(n^4)$ perchè $n^4 > n^3$ e quindi è un limite superiore asintotico per f_1 .

La notazione *O-grande* ha una controparte chiamata **notazione o-piccolo**. La notazione *O-grande* dice che una funzione è asintoticamente *non più grande*. Per dire che una funzione è asintoticamente *più piccola* di un'altra, usiamo la

notazione o -piccolo. La differenza tra le notazioni O -grande e o -piccolo è analoga alla differenza tra $\leq$ e $<$.

DEFINIZIONE 6.3: o -PICCOLO

Siano f e g funzioni $f, g : \mathbb{N} \rightarrow \mathbb{R}^+$. Diciamo che $f(n) = o(g(n))$ se

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

In altri termini, $f(n) = o(g(n))$ significa che per ogni numero reale $c > 0$, esiste un numero n_0 , tale che $f(n) < cg(n)$ per ogni $n \geq n_0$.

6.1.2 Analisi degli algoritmi

Vogliamo stabilire una notazione per classificare i linguaggi in base alla loro necessità di tempo.

DEFINIZIONE 6.4: CLASSE DI COMPLESSITÀ DI TEMPO

Sia $t : \mathbb{N} \rightarrow \mathbb{R}^+$ una funzione. La *classe di complessità di tempo*, $\text{TIME}(t(n))$, è definita come l'insieme di tutti i linguaggi che sono decisi da una macchina di Turing in tempo $O(t(n))$.

Torniamo al linguaggio $A = \{0^k 1^k \mid k \geq 0\}$ e la macchina TM M_1 che decide A . $M_1 =$ “Su input w :

1. Scandisce il nastro e *rifiuta* se trova uno 0 a destra di un 1.
2. Ripete se il nastro contiene almeno uno 0 ed almeno un 1:
 - 2.1. Scandisce il nastro, cancellando uno 0 ed un 1.
3. Se rimane almeno uno 0 dopo che ogni simbolo 1 è stato cancellato, o se rimane almeno un 1 dopo che ogni simbolo 0 è stato cancellato, *rifiuta*. Altrimenti, se non rimangono né simboli 0 né simboli 1, *accetta*.”

La fase 1 richiede $2n$ passi, che nella notazione O -grande diventa $O(n)$. Nelle fasi 2 e 3, la macchina esegue ripetutamente la scansione del nastro e cancella uno 1 e un 1 ad ogni scansione. Ogni scansione utilizza $O(n)$ passo. Poiché ogni scansione elimina due simboli, possono verificarsi al più $n/2$ scansioni. Così, il tempo totale impiegato dalle fasi 2 e 3 è di $n/2 O(n) = O(n^2)$ passi. Quindi $A \in \text{TIME}(n^2)$.

Cerchiamo una TM che migliora il tempo di esecuzione e consideriamo la macchina M_2 .

$M_2 =$ “Su input w :

1. Scandisce il nastro e *rifiuta* se trova uno 0 a destra di un 1.
2. Ripete finchè il nastro contiene almeno uno 0 ed un 1.
 - 2.1. Scandisce il nastro, controllando se il numero totale di 0 e di 1 rimasti è pari o dispari. Se è dispari, *rifiuta*.
 - 2.2. Scandisce nuovamente il nastro, cancellando prima ogni secondo 0 a partire dal primo 0, poi cancellando ogni secondo 1 a partire dal primo 1.
3. Se nessun 0 e nessun 1 rimangono sul nastro, *accetta*.

Altrimenti, *rifiuta*”.

Analizziamo il tempo di esecuzione di M_2 . Osserviamo che ogni fase impiega un tempo $O(n)$. Determiniamo il numero di volte in cui ognuna viene eseguita. Le fasi 1 e 5 vengono eseguite una volta, impiegando un tempo totale di $O(n)$. La fase 4 scarta almeno metà dei simboli 0 e 1 ogni volta che viene eseguita, quindi si verificano al massimo $1 + \log_2 n$ iterazioni del ciclo prima di averli cancellati tutti. Così il tempo tale delle fasi 2, 3, 4 è $(1 + \log_2 n)O(n)$ o $O(n \log n)$. Il tempo di esecuzione di M_2 è $O(n) + O(n \log n) = O(n \log n)$. Quindi $A \in \text{TIME}(n \log n)$.

6.2 La classe P

6.2.1 Tempo polinomiale

DEFINIZIONE 6.5: LA CLASSE P

P è la classe dei linguaggi che sono decidibili in tempo polinomiale su una macchina di Turing deterministica a singolo nastro, In altre parole,

$$P = \bigcup_k \text{TIME}(n^k)$$

La classe P ha un ruolo centrale nella teoria della complessità ed è importante perchè

1. P è invariante per tutti i modelli di calcolo che sono polinomialmente equivalenti ad una macchina di Turing deterministica a nastro singolo.
2. P corrisponde approssimativamente alla classe dei problemi che sono realisticamente risolvibili su un computer.

Il punto 1 indica che P è una classe matematicamente robusta. Non è influenzata dai paritolari del modello di computazione che stiamo usando. Il punto 2 indica che P è importante da un punto di vista pratico. Quando un problema è in P, abbiamo un metodo per risolverlo che viene eseguito in tempo n^k , per una qualche costante k . Ovviamente, un tempo di esecuzione di n^{100} è improbabile che sia di qualche utilità pratica.

Esempi di problemi in P Quando analizziamo un algoritmo per dimostrare che esso viene eseguito in tempo polinomiale, abbiamo bisogno di fare due cose. In primo luogo, dobbiamo dare un limite superiore polinomiale sul numero di fasi che l'algoritmo esegue quando viene eseguito su un input di lunghezza n . Poi, dobbiamo esaminare le singole fasi nella descrizione dell'algoritmo per essere sicuri che ognuna può essere implementata in tempo polinomiale su un modello deterministico ragionevole. Quando entrambi i compiti sono stati completati, possiamo concludere che l'algoritmo ha un tempo di esecuzione polinomiale perchè abbiamo dimostrato che viene eseguito per un numero polinomiale di fasi, ciascuna delle quali può essere eseguita in tempo polinomiale, e la composizione di polinomi è ancora un polinomio.

Analizziamo problemi computazionali che contengono la codifica di un grafo. Una codifica ragionevole di un grafo consiste in un elenco dei suoi nodi e archi. Un altro è la *matrice di adiacenza*, dove l'elemento (i, j) -esimo è 1 se c'è un arco dal nodo i al nodo j e 0 se non c'è. Quando analizziamo algoritmi sui grafi, il tempo di esecuzione può essere calcolato in termini del numero di nodi invece della dimensione della rappresentazione del grafo. Così, se analizziamo un algoritmo e mostriamo che il suo tempo di esecuzione è polinomiale nel numero di nodi, sappiamo che esso è anche polinomiale nella dimensione dell'input.

Il primo problema riguarda grafi orientati. Un grafo orientato G contiene nodi s e t , come mostriamo nella seguente figura. Il problema *PATH* consiste nel determinare, se esiste, un cammino diretto da s a t . Sia

$$PATH = \{ \langle G, s, t \rangle \mid G \text{ è un grafo orientato che contiene un cammino diretto da } s \text{ a } t \}$$

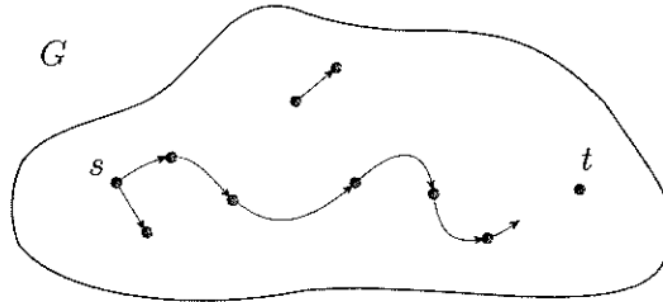


Figura 6.1: Esiste un cammino da s a t ?

Teorema 6.1

$PATH \in P$

Dimostrazione. Un algoritmo in tempo polinomiale M per $PATH$ funzione come segue.

$M =$ “Su input $\langle G, s, t \rangle$, dove G è un grafo orientato con nodi s e t :

1. Marca il nodo s .
2. Ripete il seguente passo fino a quando nessun nuovo nodo viene marcato:
 - 2.1. Scansiona tutti gli archi di G . Se trova un arco (a, b) che va da un nodo a marcato ad un nodo b non marcato, marca il nodo b .
3. Se t è marcato, *accetta*. Altrimenti, *rifiuta*.”

Ora analizziamo questo algoritmo per dimostrare che lavora in tempo polinomiale. Ovviamente, le fasi 1 e 4 sono eseguite una solta volta. La fase 3 è eseguita al più m volte perchè ogni volta, tranne l'ultima, marca un nodo aggiuntivo in G . Quindi il numero totale di fasi utilizzate è al massimo $1 + 1 + m$, che dà un polinomio nella dimensione di G .

Le fasi 1 e 4 sono facilmente implementate in tempo polinomiale su un qualsiasi modello deterministico ragionevole. La fase 3 richiede una scansione dell'input ed un test che verifichi se certi nodi sono marcati o meno, ed è anch'essa facilmente implementabile in tempo polinomiale. Quindi M è un algoritmo di tempo polinomiale per $PATH$. $\square$

Consideriamo un altro esempio di algoritmo avente tempo polinomiale. Diciamo che due numeri sono *relativamente primi* se 1 è il più grande numero intero che li divide entrambi. Per esempio, 10 e 21 sono relativamente primi, anche se nessuno dei due è un numero primo. Sia *RELPRIME* il problema di verificare se due numeri sono relativamente primi. Quindi

$$RELPRIME = \{\langle x, y \rangle \mid x \text{ e } y \text{ sono relativamente primi}\}$$

Teorema 6.2

RELPRIME $\in P$.

Questo problema si può risolvere con antico procedimento numerico, chiamato *algoritmo di Euclide*, per il calcolo del massimo comun divisore.

Dimostrazione. L'algoritmo euclideo *E* è il seguente.

E = "Su input $\langle x, y \rangle$, dove x e y sono numeri naturali in binario:

1. Ripete finchè $y = 0$:
 - 1.1. Pone $x = x \bmod y$
 - 1.2. Scambia x e y .
2. Output x ".

L'algoritmo *R* risolve *RELPRIME*, utilizzando *E* come sottoprocedura.

R = "Su input $\langle x, y \rangle$, dove x e y sono numeri naturali in binario:

1. Esegue *E* su $\langle x, y \rangle$.
2. Se il risultato è 1, *accetta*. Altrimenti, *rifiuta*.

Chiaramente, se *E* lavora in tempo polinomiale, così fa *R* e quindi abbiamo solo bisogno di analizzare *E* per valutarne il tempo e la correttezza. Per analizzare la complessità di tempo di *E*, per prima cosa mostriamo che ogni esecuzione della fase 2, taglia il valore di x di almeno la metà. Dopo l'esecuzione della fase 2, risulta $x < y$ a causa della funzione $\bmod$. Dopo la fase 3, risulta $x > y$ perchè i due valori vengono scambiati. Così, quando la fase 2 viene successivamente eseguita, $x > y$. Se $x/2 \geq y$, allora $x \bmod y \leq y \leq x/2$ e x diminuisce di almeno la metà. Se $x/2 < y$, allora $x \bmod y = x - y < x/2$ e x diminuisce di almeno la metà. I valori di x e y sono scambiati ogni volta che viene eseguita la fase 3, così ognuno dei valori originali di x e y è ridotto di almeno la metà ad ogni iterazione del ciclo. Quindi, il numero di volte in cui le

fasi 2 e 3 sono eseguite risulta pari a $\min 2 \log_2 x, 2 \log_2 y$. Questi logaritmi sono proporzionali alle lunghezze delle rappresentazioni, il che dà il numero di fasi eseguite pari a $O(n)$. $\square$

L'ultimo esempio di algoritmo polinomiale mostra che ogni linguaggio context-free è decidibile in tempo polinomiale.

Teorema 6.3

Ogni linguaggio context-free è un elemento di P.

Dimostrazione. Il seguente algoritmo D dimostra il teorema. Sia $G = (\Sigma, Q, P, S, F)$ una CFG in forma normale di Chomsky che genera il CFL L . Si supponga che S sia la variabile iniziale.

$D =$ "Su input $w = w_1 \cdots w_n$:

```

1  if  $w = \epsilon$  and  $S \rightarrow \epsilon \in P$  then
2      accetta.
3  else:
4      rifiuta.
5  for  $i=1$  to  $n$ : // Esamina sottostringhe di lunghezza 1
6      if  $A \rightarrow b \in P$  and  $b = w_i$  then
7          table( $i, i$ ) =  $A$ 
8  for  $l = 2$  to  $n$ : //  $l$  e' la lunghezza della sottostringa
9      for  $i = 1$  to  $n - l + 1$ : //  $i$  posizione iniziale della
10         sottstringa
11          $j = i + l - 1$  //  $j$  posizione finale della
12         sottstringa
13         for  $k = i$  to  $j - 1$ : //  $k$  posizione di separazione
14         for each  $A \rightarrow BC$ :
15             if  $B \in \text{table}(i, k)$  and  $C \in \text{table}(k + 1, j)$  then
16                 table( $i, j$ ) =  $A$ 
17  if  $S \in \text{table}(1, n)$  then:
18      accetta
19  else:
20      rifiuta

```

Ogni fase è facilmente implementata in modo da essere eseguita in tempo polinomiale. Le fasi 4 e 5 sono eseguite al più nv , volte dove v è il numero di variabili in G ed è un costante fissa indipendente da n , quindi queste fasi sono eseguite $O(n)$ volte. La fase 6 (riga 8) è eseguita al più n volte, la fase 7 (riga 9) è eseguita al più n volte. Ogni volta che la fase 7 viene eseguita, le fasi 8

(riga 9) e 9 (riga 11) sono eseguite al più n volte. Ogni volta che la fase 9 viene eseguita, la fase 10 (riga 12) è eseguita r volte, dove r è il numero di regole di G , ed è un'altra costante. Quindi la fase 11 (riga 13) dell'algoritmo viene eseguita $O(n^3)$ volte. La somma totale mostra che D esegue $O(n^3)$ fasi. $\square$

6.3 La classe NP

Consideriamo il problema del *cammino Hamiltoniano* in un grafo orientato G è un cammino orientato che attraversa ogni nodo del grafo esattamente una volta. Consideriamo il problema di verificare se un grafo orientato contiene un cammino Hamiltoniano che collega due nodi specificati, come mostrato nella seguente figura. Sia

$$HAMPATH = \{ \langle G, s, t \rangle \mid G \text{ è un grafo orientato} \\ \text{con un cammino Hamiltoniano da } s \text{ a } t \}$$

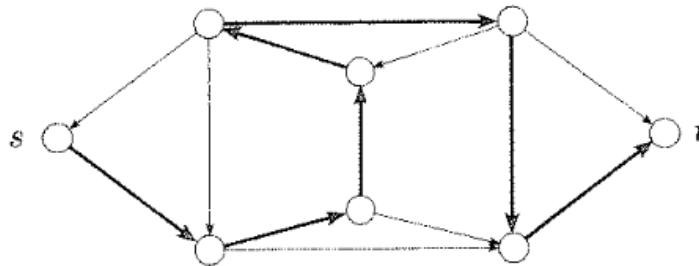


Figura 6.2: HAMPATH

Nessuno sa se *HAMPATH* è risolvibile in tempo polinomiale. Il problema *HAMPATH* ha una caratteristica chiamata *verificabilità polinomiale* che è importante per capire la sua complessità. Anche se non conosciamo una maniera veloce per determinare se un grafo contiene un cammino Hamiltoniano, se un tale cammino è stato scoperto in qualche modo, si potrebbe facilmente convincere qualcun altro della sua esistenza semplicemente esibendolo. In altre parole, *verificare* l'esistenza di un cammino Hamiltoniano può essere molto più facile che *determinare* la sua esistenza.

Un altro problema polinialmente verificabile è l'essere composto. Ricordiamo che un numero naturale è **composto** se è il prodotto di due numeri interi maggiori

di 1. Sia

$$COMPOSITES = \{x \mid x = pq, \text{ per gli interi } p, q > 1\}$$

Alcuni problemi potrebbero non essere polinomialmente verificabili. Per esempio, consideriamo $\overline{HAMPATH}$. Anche se riuscissimo a determinare che un grafo non ha un cammino Hamiltoniano, non conosciamo alcun modo per verificarne l'inesistenza senza utilizzare lo stesso algoritmo avente tempo esponenziale usato per la determinazione originaria. Una definizione formale è la seguente

DEFINIZIONE 6.6: VERIFICATORE

Un **verificatore** per un linguaggio A è un algoritmo V , dove

$$A = \{w \mid V \text{ accetta } \langle w, c \rangle \text{ per qualche stringa } c\}$$

Misuriamo il tempo di un verificatore solo in termini della lunghezza di w , quindi un *verificatore in tempo polinomiale* viene eseguito in tempo polinomiale nella lunghezza di w . Un linguaggio A è *polinomialmente verificabile* se ammette un verificatore in tempo polinomiale.

Un verificatore usa ulteriori informazioni, rappresentate ad simbolo c nella definizione di **verificatore**. Questa informazione è chiamata *certificato* o *prova* di appartenenza ad A . Ad esempio, per il problema $HAMPATH$, un certificato per una stringa $\langle G, s, t \rangle \in HAMPATH$ è semplicemente un cammino Hamiltoniano da s a t . Per il problema $COMPOSITES$, un certificato per il numero x è semplicemente uno dei suoi divisori.

DEFINIZIONE 6.7: LA CLASSE NP

La classe NP è la classe dei linguaggi che ammettono un verificatore in tempo polinomiale.

La classe NP è importante perchè contiene molti problemi di interesse pratico. Come abbiamo visto, $HAMPATH$ e $COMPOSITES$ sono elementi di NP. Inoltre, $COMPOSITES$ è anche elemento di P, che è un sottoinsieme di NP; ma dimostrare questo risultato più forte è molto più difficile. Il termine NP proviene da **tempo polinomiale non deterministico** e deriva da una caratterizzazione alternativa che utilizza macchine di Turing non deterministiche di tempo polinomiale.

Quella che segue è una macchina di Turing non deterministica che decide il problema $HAMPATH$ in tempo polinomiale non deterministico.

$N_1 =$ "Su input $\langle G, s, t \rangle$, dove G è un grafo orientato con nodi s e t :

1. Scrive una lista di m numeri, $p_1, \dots, p_m$, dove m è il numero di nodi in G . Ogni numero nella lista è scelto in modo non deterministico tra 1 e m .
2. Controlla se vi sono ripetizioni nella lista. Se ne trova una, *rifiuta*.
3. Controlla se $s = p_1$ e $t = p_m$. Se una delle due è falsa, *rifiuta*.
4. Per ogni i tra 1 e $m - 1$, controlla se (p_i, p_{i+1}) è un arco di G . Se non lo è, *rifiuta*. Altrimenti tutti i test sono stati superati, quindi *accetta*.”

Nella fase 1 la selezione non deterministica richiede chiaramente un tempo polinomiale. Nelle fasi 2 e 3, ogni parte è un semplice controllo, quindi entrambe richiedono tempo polinomiale. Infine, anche la fase 4 richiede tempo polinomiale. Quindi, questo algoritmo viene eseguito in tempo polinomiale non deterministico.

Teorema 6.4

Un linguaggio è in NP se esso viene deciso in tempo polinomiale da una macchina di Turing non deterministica.

Dimostrazione. Per la parte diretta di questo teorema, assumiamo $A \in NP$ e mostriamo che A è deciso da NTM N avente tempo polinomiale. Sia V il verificatore polinomiale per A la cui esistenza è assicurata dalla definizione della *La classe NP*. Si supponga che V è una TM avente tempo di esecuzione n^k e costruiamo N come segue.

$N =$ “Su input w di lunghezza n :

1. In maniera non deterministica seleziona una stringa di c di lunghezza al più n^k .
2. Esegue V su input $\langle w, c \rangle$.
3. Se V accetta, *accetta*; altrimenti, *rifiuta*.”

Per dimostrare la direzione inversa del teorema, supponiamo che A è deciso da una NTM N avente tempo polinomiale e costruiamo come segue un verificatore V avente tempo polinomiale.

$V =$ “Su input $\langle w, c \rangle$, dove w e c sono stringhe:

1. Simula N su input w , trattando ogni simbolo di c come la descrizione di una scelta non deterministica da formulare ad ogni passo.
2. Se questo ramo di computazione di N accetta, *accetta*; altrimenti, *rifiuta*.”



Definiamo la classe di complessità di tempo non deterministico $\mathbf{NTIME}(t(n))$ in modo analogo alla classe di complessità di tempo deterministico $\mathbf{TIME}(t(n))$.

DEFINIZIONE 6.8: CLASSE NTIME

$\mathbf{NTIME}(t(n)) = \{L \mid L \text{ è un linguaggio deciso da una macchina di Turing non deterministica di tempo } O(t(n))\}$

Corollario 6.1

$$NP = \bigcup_k \mathbf{NTIME}(n^k)$$

6.3.1 Esempi di problemi in NP

Una **clique** in un grafo non orientato è un sottografo, in cui ogni due nodi sono collegati da un arco. Una **k -clique** è una clique che contiene k nodi. La seguente figura illustra un grafo con una 5-clique.

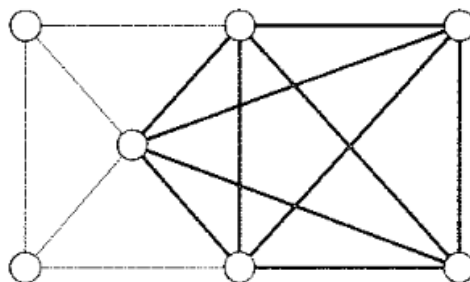


Figura 6.3: Un grafo con una 5-clique

Il problema della clique consiste nel determinare se un grafo contiene una clique di una dimensione specificata. Sia

$$CLIQUE = \{\langle G, k \rangle \mid G \text{ è un grafo non orientato contenente una } k\text{-clique}\}$$

Teorema 6.5

$$CLIQUE \in NP$$

Dimostrazione. Il seguente algoritmo è un verificatore V per $CLIQUE$.

$V = \text{“Su input } \langle G, k, c \rangle \text{:”}$

1. Controlla se c è un sottografo con k nodi in G .
2. Controlla se G contiene tutti gli archi tra i nodi in c .
3. Se entrambe le condizioni sono verificate, *accetta*; altrimenti, *rifiuta*.

□

Un'altra dimostrazione può essere la seguente.

Dimostrazione. Definiamo la TM N nel seguente modo.

$N =$ “Su input $\langle G, k \rangle$, dove G è un grafo:

1. Non deterministicamente seleziona un sottoinsieme c di k nodi in G .
2. Controlla se G contiene tutti gli archi tra i nodi di c .
3. Se sì, *accetta*; altrimenti, *rifiuta*.”

□

Come altro esempio, consideriamo il problema di aritmetica sugli interi *SUBSET-SUM*. Ci viene dato un insieme di numeri $x_1, \dots, x_k$ ed un valore obiettivo t . Vogliamo determinare se l'insieme contiene un sottoinsieme che somma a t . Quindi,

$$SUBSET-SUM = \{ \langle S, t \rangle \mid S = \{x_1, \dots, x_k\}, \text{ e per qualche } \{y_1, \dots, y_l\} \subseteq \{x_1, \dots, x_k\}, \text{ si ha } \sum y_i = t \}$$

Teorema 6.6

$SUBSET-SUM \in NP$.

Dimostrazione. L'algoritmo seguente è un verificatore V per *SUBSET-SUM*.

$V =$ “Su input $\langle \langle S, t \rangle, c \rangle$:

1. Verifica se c è una collezione di numeri la cui somma è t .
2. Verifica se S contiene tutti i numeri in c .
3. Se le condizioni sono entrambe vere, *accetta*; altrimenti, *rifiuta*.”

□

Una dimostrazione alternativa è la seguente.

Dimostrazione. Definiamo la NTM N come segue.

$N =$ “Su input $\langle S, t \rangle$:

1. In modo non deterministico seleziona un sottoinsieme c dei numeri in S .
2. Verifica se c è una collezione di numeri la cui somma è t .
3. Se la condizione è vera, *accetta*; altrimenti, *rifiuta*.”

□

Si osservi che i complementi di questi insiemi, non sono elementi di NP. Verificare che qualcosa non è presente sembra essere più difficile di verificare che è presente. Definiamo una classe di complessità separata, denominata **coNP**, che contiene i linguaggi che sono il complemento di un linguaggio in NP. Non sappiamo se coNP è diversa da NP.

6.3.2 $P = NP$

Come abbiamo detto, NP è la classe dei linguaggi che sono risolubili in tempo polinomiale con una macchina di Turing non deterministica. P è la classe dei linguaggi per cui l'appartenenza può essere decisa in tempo polinomiale. Quindi, riassumendo:

- P = la classe dei linguaggi per cui l'appartenenza è *decidibile* velocemente.
- NP = la classe dei linguaggi per cui l'appartenenza è *verificabile* velocemente.

Abbiamo presentato i linguaggi *HAMPATH* e *CLIQUE* che sono elementi di NP, ma non è nota l'appartenenza a P. Il potere della verificabilità polinomiale sembra essere molto maggiore di quello della decidibilità polinomiale. Ma, P e NP potrebbero essere uguali. Non siamo in grado di dimostrare l'esistenza di un solo linguaggio in NP che non è in P.

Il miglior metodo deterministico attualmente noto per decidere se un linguaggio è in NP utilizza tempo esponenziale. In altre parole, possiamo dimostrare che

$$NP \subseteq EXPTIME = \bigcup_k TIME(2^{n^k})$$

ma non sappiamo se NP è contenuta in una classe di complessità deterministica più piccola.

6.4 NP-completezza

Un progresso importante sulla questione $P \neq NP$ è dovuto al lavoro di Stephen Cook e Leonid Levin che scoprirono vari problemi appartenenti a NP la cui complessità individuale è correlata a quella dell'intera classe. Se esistesse un algoritmo di tempo polinomiale per uno qualsiasi di essi, tutti i problemi in NP diventerebbero risolvibili in tempo polinomiale. Questi problemi vengono detti **NP-completi**.

Il problema NP-completo che analizziamo è il **problema della soddisfacibilità**. Una **formula booleana** è un'espressione che coinvolge variabili e operazioni booleane. Una formula booleana è **soddisfacibile** se qualche assegnamento di 0 e di 1 alle variabili fa sì che la formula valga 1. La seguente formula booleana

$$\phi = (\bar{x} \wedge y) \vee (x \wedge \bar{z})$$

è soddisfacibile perchè l'assegnamento $x = 0, y = 1$ e $z = 0$ fa sì che ϕ valga 1. Il problema della soddisfacibilità consiste nel verificare se una formula booleana è soddisfacibile. Sia

$$SAT = \{\langle \phi \rangle \mid \phi \text{ è una formula booleana soddisfacibile}\}$$

Teorema 6.7: SAT polinomiale

$$SAT \in P \iff P = NP$$

6.4.1 Riducibilità in tempo polinomiale

Precedentemente, abbiamo definito il concetto di riduzione di un problema ad un altro. Quando il problema A si riduce al problema B , una soluzione per B può essere usata per risolvere A . Ora definiamo una versione della riducibilità che tiene conto dell'efficienza di computazione. Quando il problema A è riducibile *efficientemente* al problema B , una soluzione efficiente per B può essere usata per risolvere A efficientemente.

DEFINIZIONE 6.9: FUNZIONE CALCOLABILE IN TEMPO POLINOMIALE

Una funzione $f : \Sigma^* \rightarrow \Sigma^*$ è una **funzione calcolabile in tempo polinomiale** se esiste una TM M che si arresta con $f(\omega)$ soltanto sul suo nastro, quando ha iniziato con un qualsiasi input ω .

DEFINIZIONE 6.10: RIDUCIBILITÀ IN TEMPO POLINOMIALE

Il linguaggio A è **riducibile mediante una funzione in tempo polinomiale**, o semplicemente **riducibile in tempo polinomiale**, al linguaggio B , deno-

tato con $A \leq_P B$, se esiste una funzione calcolabile in tempo polinomiale $f : \Sigma^* \rightarrow \Sigma^*$ dove, per ogni ω ,

$$\omega \in A \iff f(\omega) \in B$$

La funzione f è chiamata **riduzione di tempo polinomiale** di A a B .

Teorema 6.8

Se $A \leq_P B$ e $B \in P$, allora $A \in P$.

Dimostrazione. Sia M l'algoritmo di tempo polinomiale che decide B ed f la riduzione di tempo polinomiale di A a B . Descriviamo un algoritmo di tempo polinomiale N che decide A come segue.

$N =$ "Su input ω :

1. Calcola $f(\omega)$.
2. Esegui M con input $f(\omega)$ e dai in output qualsiasi cosa M da in output."

Risulta $\omega \in A$ ogni volta che $f(\omega) \in B$ perchè f è una funzione di riduzione di A a B . Pertanto, M accetta $f(\omega)$ ogni volta che $\omega \in A$. Inoltre, N computa in tempo polinomiale perchè ciascuna delle sue due fasi viene eseguita in tempo polinomiale. $\square$

Prima di mostrare una riduzione di tempo polinomiale, introduciamo $3SAT$, un caso speciale di problema di soddisfacibilità per cui tutte le formule sono in forma speciale. Un **letterale** è una variabile booleana o una variabile booleana negata, come x o $\bar{x}$. Una **clausola** consiste in diversi letterali connessi tramite operatori $\vee$, come $(x_1 \vee \bar{x}_2 \vee \bar{x}_3 \vee \bar{x}_4)$. Una formula booleana è in **forma normale congiuntiva** o **cnf**, se comprende diverse clausole connesse tramite operatori $\wedge$, come

$$(x_1 \vee \bar{x}_2 \vee \bar{x}_3 \vee \bar{x}_4) \wedge (x_3 \vee \bar{x}_5 \vee x_6) \wedge (x_3 \vee \bar{x}_6)$$

Essa è una **formula 3cnf** se tutte le clausole hanno tre letterali. Sia $3SAT = \{\langle \phi \rangle \mid \phi \text{ è una formula 3cnf soddisfacibile}\}$.

Teorema 6.9

$3SAT$ è riducibile in tempo polinomiale a $CLIQUE$.

Dimostrazione. Sia ϕ una formula con k clausola come

$$\phi = (a_1 \vee b_1 \vee c_1) \wedge (a_2 \vee b_2 \vee c_2) \wedge \cdots \wedge (a_k \vee b_k \vee c_k)$$

La riduzione f genera la stringa $\langle G, k \rangle$, dove G è un grafo non orientato

definito come segue.

I nodi in G sono organizzati in k gruppi di tre nodi ciascuno, detti **triple**, $t_1, \dots, t_k$. Ciascun tripla corrisponde ad una delle clausole in ϕ , e ciascun nodo in una tripla corrisponde ad un letterale nella clausola associata. Si etichetti ciascun nodo di G con il suo letterale corrispondente in ϕ . Gli archi di G connettono tutti meno due tipi di coppie di nodi in G . Non ci sono archi presenti tra nodi nella stessa tripla, e non ci sono archi presenti tra nodi con etichette complementari, come x_2 ed $\bar{x}_2$.

Dimostriamo che ϕ è soddisfacibile se e solo se G ha una clique di dimensione k . Si supponga che ϕ abbia un assegnamento che la soddisfa. In questo assegnamento soddisfacente, almeno un letterale è vero in ogni clausola. In ciascuna tripla di G , si selezioni un nodo corrispondente al letterale vero nell'assegnamento soddisfacente. Se più di un letterale è vero in una particolare clausola, si scelga uno dei letterali veri arbitrariamente. I nodi appena selezionati formano una clique di dimensione k . Il numero di nodi selezionati è k perchè ne è stato scelto uno per ognuna delle k triple. Ciascuna coppia dei nodi selezionati è collegata da un arco perchè nessuna coppia rientra in una delle eccezioni descritte in precedenza. Non possono essere della stessa tripla perchè ne è stato scelto uno per tripla. Non possono avere etichette complementari perchè i letterali associati erano entrambi veri nell'assegnamento soddisfacente. Pertanto, G contiene una clique di dimensione k .

Si supponga che G abbia una clique di dimensione k . Nessuna coppia di nodi di una clique occorre nella stessa tripla perchè i nodi della stessa tripla non sono connessi da archi. Pertanto, ciascuna delle k triple contiene esattamente uno dei k nodi della clique. Si assegnino valori di verità alle variabili di ϕ , in modo tale che ciascun letterale che etichetta un nodo della clique risulti vero. Questa operazione è sempre possibile perchè due nodi etichettati in modo complementare non sono connessi da un arco, e, quindi, non possono essere entrambi nella clique. L'assegnamento così definito per le variabili soddisfa ϕ perchè ciascuna tripla contiene un nodo della clique, e quindi, ciascuna clausola contiene un letterale a cui è stato assegnato *true*. Pertanto, ϕ è soddisfacibile. $\square$

Quindi, se *CLIQUE* è risolvibile in tempo polinomiale lo è anche *3SAT*.

6.4.2 Definizione di NP-completezza

DEFINIZIONE 6.11: NP-COMPLETEZZA

Un linguaggio B è **NP-completo** se soddisfa due condizioni:

1. B appartiene a NP, e
2. ogni A appartenente a NP è riducibile in tempo polinomiale a B .

Teorema 6.10

Se B è NP-completo e $B \in P$, allora $P = NP$

Dimostrazione. Il teorema discende direttamente dalla definizione di riducibilità in tempo polinomiale. $\square$

Teorema 6.11

Se B è NP-completo e $B \leq_P C$ per $C \in NP$, allora C è NP-completo.

Dimostrazione. Già sappiamo che C appartiene a NP, quindi dobbiamo far vedere che ogni A appartenente a NP è riducibile in tempo polinomiale a C , poichè B è NP-completo, ogni linguaggio appartenente a NP è riducibile in tempo polinomiale a B , e B a sua volta è riducibile in tempo polinomiale a C . Le riduzioni in tempo polinomiale possono essere composte; cioè, se A è riducibile in tempo polinomiale a B e B è riducibile in tempo polinomiale a C , allora A è riducibile in tempo polinomiale a C . Pertanto, ogni linguaggio appartenente a NP è riducibile in tempo polinomiale a C . $\square$

6.4.3 Il Teorema di Cook e Levin

Una volta che disponiamo di un problema NP-completo, possiamo ottenerne altri attraverso riduzioni polinomiali da esso. Tuttavia, determinare il primo problema NP-completo è più difficile. Lo facciamo provando che SAT è NP-completo.

Teorema 6.12: Teorema di Cook e Levin

SAT è NP-completo.

Dimostrazione. Prima di tutto, mostriamo che SAT appartiene a NP. Una macchina non deterministica di tempo polinomiale può ipotizzare un assegnamento per una data formula ϕ ed accettare se l'assegnamento soddisfa ϕ . Successivamente, prendiamo un qualsiasi linguaggio A appartenente ad NP e facciamo vedere che A è riducibile in tempo polinomiale a SAT . Sia N una NTM che decide A in tempo n^k per qualche costante k . La nozione che segue

aiuta a descrivere la riduzione.

Un **tableau** per N su ω è una tabella $n^k \times n^k$ le cui righe sono le configurazioni di una diramazione della computazione di N su input ω , come mostrato nella figura seguente.

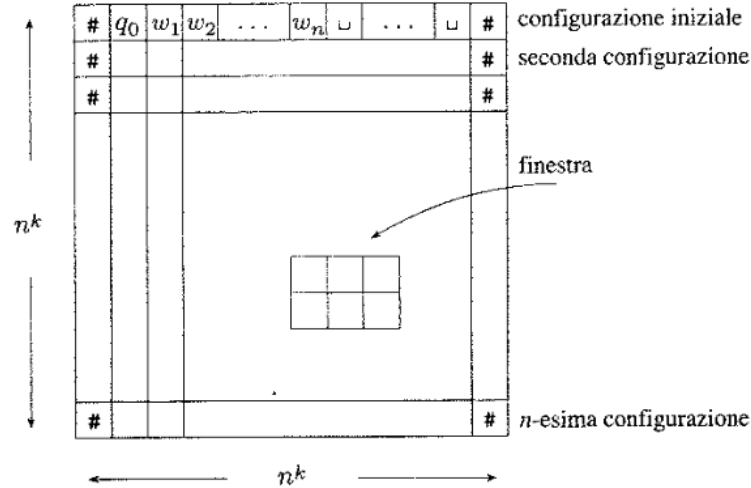


Figura 6.4: Un tableau

Per convenienza assumiamo che ciascuna configurazione inizi e finisca con un simbolo $\#$. Pertanto, la prima e l'ultima colonna di un tableau sono tutti $\#$. La prima riga di un tableau è la configurazione iniziale di N su ω , e ciascuna riga di segue dalla precedente in accordo alla funzione di transizione di N . Un tableau è **accettante** se qualche riga del tableau è una configurazione accettante.

Ogni tableau accettante per N su ω corrisponde ad una diramazione della computazione accettante di N su ω . Quindi, il problema di stabilire se N accetta ω è equivalente al problema di stabilire se esiste un tableau accettante per N su ω .

Passiamo ora alla descrizione di tempo polinomiale f di A a SAT . Su input ω , la riduzione produce una formula ϕ . Iniziamo descrivendo le variabili di ϕ . Siano Q e Γ l'insieme degli stati e l'alfabeto del nastro di N , rispettivamente. Sia $C = Q \cup \Gamma \cup \{\#\}$. Per ciascun i e j tra 1 ed n^k e per ciascun $s \in C$, introduciamo una variabile $x_{i,j,s}$.

Ciascuna delle $\binom{n^k}{2}$ entrate di un tableau è chiamata **cella**. La cella in riga i e colonna j viene denotata con $cell[i, j]$ e contiene un simbolo di C . Rappresentiamo i contenuti delle celle con le variabili di ϕ . Se $x_{i,j,s} = 1$, significa che $cell[i, j]$ contiene s .

Progettiamo a questo punto ϕ in modo tale che un assegnamento che soddisfa le variabili di ϕ corrisponda ad un tableau accettante per N su ω . La formula ϕ è l'AND di quattro parti: $\phi_{cell} \wedge \phi_{start} \wedge \phi_{move} \wedge \phi_{accept}$. Descriviamo ciascuna parte, una per volta.

La prima cosa che dobbiamo garantire al fine di ottenere una corrispondenza tra un assegnamento ed un tableau è che l'assegnamento ponga ad 1 esattamente una variabile per ciascuna cella. La formula ϕ_{cell} garantisce questo requisito in termini di operazioni booleane:

$$\phi_{cell} = \bigwedge_{1 \leq i, j \leq n^k} \left[\left(\bigvee_{s \in C} x_{i,j,s} \right) \wedge \left(\bigwedge_{s, t \in C, s \neq t} (\overline{x_{i,j,s}} \vee \overline{x_{i,j,t}}) \right) \right]$$

La prima parte della formula stabilisce che almeno una variabile assume valore 1 nella cella corrispondente. La seconda parte stabilisce che non più di una variabile assume valore 1.

Qualsiasi assegnamento alle variabili che soddisfa ϕ (e di conseguenza ϕ_{cell}) deve avere esattamente una variabile ad 1 per ogni cella. Pertanto, qualsiasi assegnamento che soddisfa ϕ specifica un simbolo in ciascuna cella della tabella. Le parti ϕ_{start} , ϕ_{move} e ϕ_{accept} garantiscono che questi simboli corrispondano realmente ad un tableau accettante.

La formula ϕ_{start} assicura che la prima riga della tabella sia la configurazione iniziale di N su ω , richiedendo esplicitamente che le variabili siano ad 1:

$$\begin{aligned} \phi_{start} = & x_{1,1,\#} \wedge x_{1,2,q_0} \wedge \\ & x_{1,3,\omega_1} \wedge x_{1,4,\omega_2} \wedge \dots \wedge x_{1,n+2,\omega_n} \wedge \\ & x_{1,n+3,\sqcup} \wedge \dots \wedge x_{1,n^k-1,\sqcup} \wedge x_{1,n^k,\#} \end{aligned}$$

La formula ϕ_{accept} garantisce che una configurazione accettante sia presente nel tableau. Essa assicura che q_{accept} , il simbolo per lo stato di accettazione, compaia in una delle celle del tableau, richiedendo che una delle variabili corrispondenti sia a 1:

$$\phi_{accept} = \bigvee_{1 \leq i, j \leq n^k} x_{i,j,q_{accept}}$$

Infine, la formula ϕ_{move} garantisce che ciascuna riga del tableau corrisponda ad una configurazione che segue legittimamente dalla configurazione della riga precedente, in accordo alle regole di N . Lo fa assicurando che ogni finestra di celle 2×3 sia lecita. Diciamo che una finestra è **lecita** se non viole le azioni specificate della funzione di transizione di N . Per esempio, siano a, b e c elementi dell'alfabeto di nastro, e q_1 e q_2 stati di N . Si assuma che nello stato q_1 con la testina che legge una a , N scrive una b , resta nello stato q_1 , e muove a destra; e che quando nello stato q_1 con la testina che legge una b , N non deterministicamente

1. scrive una c , entra in q_2 , e muove a sinistra, oppure
2. scrive una a , entra in q_2 , e muove a destra.

In termini formali $\delta(q_1, a) = \{(q_1, b, R)\}$ e $\delta(q_1, b) = \{(q_2, c, L), (q_2, a, R)\}$. Esempi di finestre lecite per questa macchina sono le seguenti.

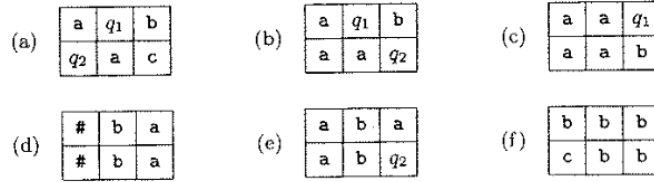


Figura 6.5: Esempi di finestre lecite

Esempi di finestre non lecite sono:

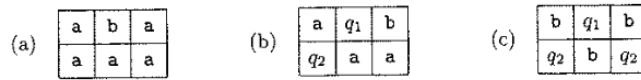


Figura 6.6: Esempi di finestre non lecite

PROPOSIZIONE 6.1

Se la riga superiore del tableau è la configurazione iniziale ed ogni finestra nel tableau è lecita, ciascuna riga del tableau è una configurazione che segue legittimamente dalla precedente.

Proviamo l'asserto considerando ogni coppia di configurazioni adiacenti nel tableau, indicate come la configurazione superiore e la configurazione inferiore. Nella configurazione superiore, ogni cella che contiene un simbolo di nastro e non è adiacente ad un simbolo di stato rappresenta la cella centrale superiore in una finestra la cui riga superiore non contiene stati. Pertanto, questo stesso simbolo deve comparire al centro in basso nella finestra. La finestra contenente il simbolo di stato nella cella centrale superiore garantisce che le tre posizioni corrispondenti vengano aggiornate consistentemente tramite la funzione di transizione.

Torniamo ora alla costruzione di ϕ_{move} . Essa garantisce che tutte le finestre nel tableau sono lecite. Ciascuna finestra contiene sei celle, che possono essere impostate in un numero fissato di modi per produrre una finestra lecita. La formula ϕ_{move} stabilisce che le impostazioni delle sei celle devono essere uno di questi modi, ossia

$$\phi_{move} = \bigwedge_{1 \leq i < n^k, 1 < j < n^k} (\text{la finestra } (i, j) \text{ è lecita})$$

che possiamo scrivere come:

$$\bigwedge_{a_1, \dots, a_6} (x_{i,j-1,a_1} \wedge x_{i,j,a_2} \wedge x_{i,j+1,a_3} \wedge x_{i+1,j-1,a_4} \wedge x_{i+1,j,a_5} \wedge x_{i+1,j+1,a_6})$$

Esaminiamo la dimensione di ϕ per analizzare la complessità della riduzione. Prima di tutto, stimiamo il numero di variabili che ha. Il tableau è una tabella $n^k \times n^k$, quindi contiene n^{2k} celle. Ciascuna cella ha l variabili associate ad essa, dove $l = |C|$. Poiché l dipende soltanto dalla TM N e non dalla lunghezza dell'input n , il numero totale di variabili è $O(n^{2k})$.

Stimiamo la dimensione di ciascuna delle parti di ϕ . La formula ϕ_{cell} scandisce ogni singola cella del tableau, quindi la sua dimensione è $O(n^{2k})$. La formula q_{start} scandisce la prima riga del tableau, quindi la sua dimensione è $O(n^k)$. Le formule q_{move} e q_{accept} contengono un frammento di dimensione fissata della formula per ciascuna cella del tableau, quindi la loro dimensione è $O(n^{2k})$. Pertanto, la dimensione complessiva di ϕ è $O(n^{2k})$. Tale limite è sufficiente perché dimostra che la dimensione di ϕ è polinomiale in n . Pertanto, possiamo costruire facilmente una riduzione che produce ϕ in tempo polinomiale dall'input ω . $\square$

Quindi, abbiamo dimostrato il **teorema di Cook e Levin**.

La NP-completezza può essere provata con una riduzione di tempo polinomiale da un linguaggio che è già noto essere NP-completo. Possiamo usare SAT per questo scopo; ma usare $3SAT$ è solitamente più facile.

Corollario 6.2

$3SAT$ è NP-completo.

Dimostrazione. Ovviamente $3SAT$ appartiene a NP, quindi dobbiamo provare solamente che tutti i linguaggi appartenenti a NP si riducono a $3SAT$ in tempo polinomiale. Un modo per farlo è facendo vedere che SAT si riduce in tempo polinomiale a $3SAT$. Invece, lo facciamo modificando la dimostrazione del **teorema di Cook e Levin**, in modo tale che produca direttamente una formula in cnf con tre letterali per clausola. La formula del **teorema di Cook e Levin** produce una formula è già quasi in cnf. La formula ϕ_{cell} è un grosso AND di sottoformule. Quindi, ϕ_{cell} è in cnf. La formula q_{start} è un grosso AND tra variabili, quindi è in cnf. La formula ϕ_{accept} è un grosso OR di variabili, ed è perciò una singola clausola. La formula ϕ_{move} è l'unica che non è in forma cnf, ma possiamo facilmente convertirla.

ϕ_{move} è un grosso AND di sottoformule. La legge distributiva, stabilisce che possiamo sostituire un OR di vari AND con un equivalente AND di vari OR. Il risultato è una formula che è in cnf.

Ora che abbiamo scritto la formula in cnf, la convertiamo in una con tre letterali per clausola. In ciascuna clausola che correntemente ha uno o due letterali, duplichiamo uno dei due letterali, fino a quando il numero totale diventa tre. Ciascuna clausola che ha più di tre letterali la dividiamo in più clausole e aggiungiamo ulteriori variabili per preservare la soddisfacibilità o non della clausola originale. Per esempio, sostituiamo la clausola $(a_1 \vee a_2 \vee a_3 \vee a_4)$ con l'espressione di due clausole $(a_1 \vee a_2 \vee z) \wedge (\bar{z} \vee a_3 \vee a_4)$. In generale, se la clausola contiene l letterali,

$$(a_1 \vee a_2 \vee \dots \vee a_l)$$

possiamo sostituirla con le $l - 2$ clausole

$$(a_1 \vee a_2 \vee z_1) \wedge (\bar{z}_1 \vee a_3 \vee z_2) \wedge (\bar{z}_2 \vee a_4 \vee z_3) \wedge \dots \wedge (\bar{z}_{l-3} \vee a_{l-1} \vee a_l)$$

È possibile verificare che la nuova formula è soddisfacibile se e solo se la formula originale lo era, quindi la dimostrazione è completa. $\square$